



DEPARTMENT OF
ELECTRICAL AND COMPUTER ENGINEERING

MARCOS VINICIUS BONFIM ROMÃO

BSc in Electrical and Computer Engineering

5G NETWORKS: DEPLOYMENT OF AN EXPERIMENTAL 5G CORE AND RADIO ACCESS NETWORK

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

September, 2026



DEPARTMENT OF
ELECTRICAL AND COMPUTER ENGINEERING

5G NETWORKS: DEPLOYMENT OF AN EXPERIMENTAL 5G CORE AND RADIO ACCESS NETWORK

MARCOS VINICIUS BONFIM ROMÃO

BSc in Electrical and Computer Engineering

Supervisor: Rodolfo Oliveira

Full Professor, NOVA University Lisbon

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

September, 2026

5G Networks: Deployment of an experimental 5G CORE and Radio Access Network

Copyright © Marcos Vinicius Bonfim Romão, NOVA School of Science and Technology, NOVA University Lisbon.

The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

ACKNOWLEDGEMENTS

First and foremost, I would like to express my love and affection for my parents, Luciana and Antonio, who have always placed their trust in me and supported me throughout my whole life. Without them, I would not have been able to reach this point in my life. I hope this dissertation, the achievement behind it, and the result of 17 years of study can make them proud. To my mother, I would like to express my gratitude for her unconditional love and support, for her curiosity in always asking me about my work and studies, and for always being there for me. To my father, I would like to express my gratitude for his guidance, his support, his ideas, and for always believing in me.

To my sister, Gabrielle, thanks for reading my drafts, providing valuable feedback, and asking about my work. Your questions made me explain things more clearly than I would have on my own. To my older sister, Débora, along with Leo and my nephews, thanks for your support and encouragement.

To my aunt, my uncle, and my cousins, who shared my home with me during my university career, helped me in many ways, kept me company, and supported me, I am thankful for their support.

To the rest of my family, on the other side of the world, I would like to express my thanks for their support and encouragement throughout my academic journey.

I am sincerely grateful to my supervisor, Professor Rodolfo Oliveira, for the opportunity to work on this dissertation and for his invaluable guidance and support throughout this research.

To my friends, who have been a source of support (and distraction), I would like to express my thanks for their friendship.

Without the support of everyone, I would not have been able to complete this dissertation. I am grateful for their presence throughout these five years.

The author acknowledges the support of *Instituto de Telecomunicações (IT)*, whose infrastructure and hardware resources were essential to the implementation and experimental validation of the work presented in this thesis. This work was funded by *FCT – Fundação para a Ciência e Tecnologia, I.P.*, under the projects UID/50008/2025 (DOI: [10.54499/UID/50008/2025](https://doi.org/10.54499/UID/50008/2025)) and 2022.08786.PTDC (DOI: [10.54499/2022.08786.PTDC](https://doi.org/10.54499/2022.08786.PTDC)).

ABSTRACT

Open-source software and commodity software-defined radios have made it possible to deploy a private 5G Standalone network without a commercial operator's budget. Deploying it, however, is only half the problem: a working network can still perform far below what its hardware allows, and calibrating and tuning it for the conditions it operates in is a challenge of its own, harder still without a reference to compare against.

This dissertation has focused on the implementation of a 5G Standalone network alongside a 4G one, to serve as a reference, sharing the same tools and Open5GS core. The 5G network is based on the srsRAN Project gNB and a USRP B210, and the 4G network on srsRAN 4G and a bladeRF xA4. The two were compared at three distances, tuning the 5G network over three rounds.

Against this baseline, the untuned 5G uplink was more than 20 times slower at 10 m. Rebalancing the TDD pattern from 3:1 to 2:2 brought it to the 4G level, and lowering the receiver gain removed a short-range overload, reaching 28.5 Mbps at 2 m, twice the 4G baseline, but at the cost of lower performance on longer distances. The 5G downlink reached 50.6 Mbps, over twice that of 4G. The idle round-trip time averaged 25 to 28 ms, but rose to seconds under TCP load due to an oversized default RLC queue.

These results show that an open-source 5G Standalone network can match or exceed a 4G network built with the same tools, but only once tuned to its hardware and environment, and that no fixed configuration suits every location. The implemented networks are now ready to support future experimental activities.

Keywords: 5G Standalone · Open-source · Software-Defined radio · 5G Testbed · Performance evaluation

RESUMO

O software open-source e os rádios definidos por software de baixo custo permitem implementar uma rede 5G privada em modo Standalone sem o orçamento de um operador comercial. Implementá-la, porém, é apenas metade do problema, pois uma rede funcional ainda pode ficar muito aquém do que o equipamento permite, e calibrá-la às condições em que opera é outro desafio, ainda mais difícil sem uma referência.

Esta dissertação focou-se na implementação de uma rede 5G Standalone, juntamente com uma rede 4G, como referência, partilhando o mesmo núcleo Open5GS. A rede 5G utiliza o srsRAN Project e uma USRP B210, e a rede 4G utiliza o srsRAN 4G e uma bladeRF xA4. As duas foram comparadas a três distâncias do rádio, com a rede 5G ajustada ao longo de três rondas.

Face a esta referência, o uplink 5G, sem ajustes, era mais de 20 vezes mais lento a 10 m. A mudança do padrão TDD de 3:1 para 2:2 igualou o débito ao da rede 4G, e a redução do ganho do recetor eliminou a saturação do equipamento a curta distância, atingindo 28,5 Mbps a 2 m, o dobro da 4G, mas à custa da diminuição do desempenho nas maiores distâncias. O downlink 5G atingiu 50,6 Mbps, mais do dobro da 4G. O tempo de ida e volta médio, de 25 a 28 ms sem carga, subiu para segundos sob tráfego TCP, devido a uma queue RLC sobredimensionada.

Estes resultados mostram que uma rede 5G Standalone open-source pode igualar ou superar uma rede 4G construída com as mesmas ferramentas, mas só depois de ajustada às condições em que se pretende operar, e que nenhuma configuração fixa serve a todas as localizações. As redes implementadas ficam disponíveis para auxiliar em trabalhos laboratoriais futuros.

Palavras-chave: 5G Standalone · Open-source · 5G Testbed · Software-Defined Radio · Avaliação de desempenho

DISCLOSURE OF USAGE OF GENERATIVE ARTIFICIAL INTELLIGENCE

The author, Marcos Vinicius Bonfim Romão, acknowledges the use of the following generative artificial intelligence tools in the development of this thesis. These technologies were utilized under strict supervision, and all AI-generated outputs were critically reviewed and verified for accuracy. The author accepts full responsibility for the validity and originality of the content, confirming that the use of these tools aligns with all institutional policies and standards of academic integrity.

Generative AI tools used: Anthropic Claude and Grammarly.

Activities supported by Generative Artificial Intelligence Tools:

Conceptualization

☐ Idea generation

☐ Defining the research objective

☐ Formulating research questions and hypotheses

☐ Feasibility assessment and risk evaluation

☐ Hypothesis viability assessment

☐ Simulated debate and argument testing

Literature Review

☐ Search and Discovery

☐ Literature summarization and synthesis

☐ Concept Mapping and Systematization

☐ Market/patent landscape analysis

☐ Gap Identification and Novelty Evaluation

☐ Cross-lingual literature comprehension

Methodology

☐ Experimental Design Optimization

☐ Development of experimental or research protocols

☐ Methodological Instrument Selection

Software Development and Automation

- | | | |
|---|---|--|
| ☒ Code Generation | ☐ Debugging and Repair | ☐ Algorithm design |
| ☐ Refactoring and Optimization | ☐ Process automation | ☐ Code documentation and comment generation |

Data Management

- | | | |
|--|--|--|
| ☐ Data collection | ☐ Data analysis | ☐ Synthetic data generation |
| ☐ Validation | ☐ Quantitative Plotting and Charting | ☐ De-identification and anonymization support |
| ☐ Data cleaning | ☒ Reproducibility and rerun checks | ☐ Audio-to-text transcription and diarization |
| ☒ Data curation and organization | ☐ Data labeling and annotation assistance | |

Visuals and Multimedia

- | | | |
|---|--|--|
| ☐ Synthetic Asset Generation | ☐ Image Enhancement and Editing | ☐ Diagrammatic and Schematic Design |
|---|--|--|

Writing and Editing

- | | | |
|--|---|--|
| ☒ Drafting Text | ☐ Formulation of conclusions | ☐ Citation formatting and bibliography management |
| ☒ Polishing and Editing | ☐ Tone Adjustment | ☐ Press releases and outreach materials |
| ☐ Abstract and Executive Summary Generation | ☐ Translation | ☐ Title Generation |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Quality Assurance

- | | | |
|--|---|--|
| ☒ Simulated Peer Review | ☐ Identification of limitations | ☐ Publication support |
| ☐ Consistency checking against field trends | ☐ Publication strategy and journal selection | |

Additional comment #1: Claude drafted some paragraphs across the document, which were then rewritten by the author.

Additional comment #2: At the end, Claude was used to find grammar errors and typos, and also to point out unclear sections and areas that weren't explored to their full potential.

Additional comment #3: Claude was used to scrape through the 12 GB of raw data generated from the networks and find patterns in the logs that point to useful data. This data

was then aggregated into CSV files, which were used by the author to make the necessary analysis.

Additional comment #4: Grammarly was used to rewrite sections of the document in a clearer, more grammatically correct manner.

CONTENTS

Abstract	v
Resumo	vi
Disclosure of Usage of Generative Artificial Intelligence	viii
Contents	xi
List of Figures	xv
List of Tables	xviii
Acronyms	xxi
1 Introduction	1
1.1 Motivation	1
1.2 Objectives and Contributions	3
1.3 Report Structure	4
2 Fundamentals of a 5G Network	5
2.1 Introduction to the 5G Architecture	5
2.2 Standalone vs Non-Standalone 5G Deployments	9
2.2.1 Non-Standalone (NSA) 5G Deployments	9
2.2.2 Standalone (SA) 5G Deployments	10
2.3 5G RAN	10
2.3.1 Next Generation Radio Access Network (NG-RAN) Architecture	11
2.3.2 RAN Interfaces	12
2.3.3 RAN Features	14

2.4	5G Core	17
2.4.1	Core Network Functions	18
2.4.2	Core Interfaces	21
2.5	Existing Open-Source Software for 5G Networks	21
2.5.1	Open-Source 5G Core Implementations	22
2.5.2	Open-Source 5G RAN Implementations	23
2.6	Summary	25
3	State of the Art	27
3.1	Single-Testbed Deployments and UE Integration	27
3.2	Comparative Evaluations of Open-Source Implementations	30
3.3	Automation, Scale, and Real-World Deployments	32
3.4	Overview of Reviewed Testbeds	35
4	Implementation	37
4.1	System Architecture Overview	37
4.2	Hardware Components	38
4.3	Core	40
4.4	Radio Access Network (RAN)	43
4.4.1	5G RAN	44
4.4.2	4G RAN	44
4.5	User Equipment (UE)	45
4.6	Metrics Collection	45
4.7	Summary	46
5	Methodology	48
5.1	Experimental Testbed	48
5.2	Experimental Methodology	51
5.3	Experiments	54
5.3.1	Test Round 1	54
5.3.2	Test Round 2	54
5.3.3	Test Round 3	55

6	Results	57
6.1	General comparison between 4G and 5G performance	57
6.2	Unloaded latency	60
6.3	Latency under load	65
6.4	TDD rebalance effects on uplink airtime and throughput	67
6.5	Radio link quality	71
6.5.1	Round 1: 4G and 5G	71
6.5.2	Rounds 2 and 3: the receiver gain trade-off	72
6.6	Core network performance	74
7	Conclusions	77
7.1	Summary and Main Findings	77
7.2	Limitations	79
7.3	Future Work	80
	Bibliography	82
	Appendix A: Testbed Setup	88
A.1	Network Connection (All Machines)	88
A.2	Core Machine (Machine 1)	91
A.2.1	Installing MongoDB	92
A.2.2	Installing Open5GS	93
A.2.3	Installing the WebUI for Open5GS	93
A.2.4	Configuring the Core Network Functions	94
A.2.5	Routing the UE Traffic	99
A.2.6	Adding the Subscribers	100
A.3	4G RAN Machine (Machine 4)	102
A.3.1	Setting up the Nuand drivers and firmware for the bladeRF xA4	102
A.3.2	Installing SoapySDR	103
A.3.3	srsRAN 4G	104
A.4	5G RAN Machine (Machine 2)	106
A.4.1	Installing UHD	106

A.4.2	Setting up the GPSDO	107
A.4.3	srsRAN Project (OCUDU)	110
A.4.4	Configuring the gNB	111
A.5	Running the Network	114
A.5.1	Preparing the RAN Machines	114
A.5.2	Starting the 4G Network	114
A.5.3	Starting the 5G Network	114
A.5.4	Verifying the Connection	115
A.6	Troubleshooting	118
Appendix B: Provisioning eSIM		120
B.1	Creating an eSIM profile	120
B.2	Installing the profile on the UE	122

LIST OF FIGURES

2.1	5G Network Architecture diagram.	6
2.2	Control and User Plane Separation.	7
2.3	5G NSA vs SA Deployments.	9
2.4	Next Generation Node B (gNodeB) Architecture.	12
2.5	5G RAN Protocol Stack.	13
2.6	5G RAN Interfaces.	13
2.7	Massive MIMO antennas.	14
2.8	5G Frequency Bands.	15
2.9	Subframe and slot structure in 4G and in 5G NR.	17
2.10	Downlink, uplink and flexible slots in a TDD period.	17
2.11	5G Core Network Architecture.	18
4.1	Network topology and static addresses of the four machines.	38
4.2	The laboratory bench, with the four machines.	39
4.3	The Nuand bladeRF xA4, connected and cabled on the bench. This board had no external synchronization source, although an SMA cable is present in the figure.	41
4.4	The B2901 and the Leo Bodnar GPSDO, as connected on the bench. Output 1 of the GPSDO provides a 1 PPS signal to the B2901's PPS IN port, while Output 2 provides a 10 MHz reference clock to the REF IN port.	41
4.5	The BT-200 LNA and BT-100 power amplifier bias tees, mounted on the bladeRF xA4's RX1 and TX1 ports respectively.	42

4.6	The testbed as deployed.	47
5.1	Plan view of the room, showing the radio position and the three UE test locations, drawn to scale.	49
5.2	Elevation view of the radio and UE heights.	50
5.3	Measured thickness of the two obstructions.	50
5.4	The three test locations, with the UE in place.	52
5.5	Slot and symbol structure of the 3:1 Time Division Duplexing (TDD) pattern used in Round 1.	53
5.6	Slot and symbol structure of the rebalanced 5-slot period of the 2:2 TDD pattern used from Round 2 onward.	53
5.7	The metal stand at Location B.	55
6.1	TCP downlink throughput at each test location.	58
6.2	UDP downlink throughput at each test location.	59
6.3	TCP uplink throughput at each test location.	60
6.4	UDP uplink throughput at each test location.	61
6.5	Round-trip time of each of the 30 ICMP echoes at the three test locations, with the 15.7 ms floor marked.	62
6.6	Wait for the next scheduling request opportunity.	64
6.7	TCP throughput across the three 5G rounds by configuration.	70
6.8	Two 5G registrations in the NGAP captures.	75
A.1	Output of <code>ip a</code> on the core machine.	89
A.2	Pings from the core machine to both RAN machines.	91
A.3	Output of the <code>systemctl status mongod</code> command, showing that the service is active and running.	93
A.4	The Open5GS WebUI's subscriber list, each subscriber identified by its IMSI.	100
A.5	The Open5GS WebUI's subscriber form, filled in for the testbed's UE.	101
A.6	The GPS Disciplined Oscillator (GPSDO)'s GPS antenna, placed on the windowsill outside the lab window for a clear view of the sky.	108
A.7	Output of <code>lbe142x.py detail</code> for the LBE-1421.	110

A.8	The CellularLab app configured for a TCP uplink test.	117
B.1	Simlessly Dashboard.	121
B.2	Generating an eSIM profile.	121
B.3	eSIM Activation Code.	122
B.4	Installing the eSIM profile on the UE.	123

LIST OF TABLES

2.1	5G Core Network Interfaces.	21
2.2	Overview of different open-source 5G Core implementations.	23
2.3	Comparison of the open-source 5G RAN projects.	25
3.1	Overview of network implementations in related work.	36
4.1	Specifications of testbed machines.	38
4.2	Software versions used on each machine throughout the test campaigns. . .	39
4.3	Main configuration values of the Open5GS core.	43
4.4	Configuration of the 4G and 5G RANs.	44
4.5	Comparison of the two candidate UEs.	45
4.6	Data sources recorded by the testbed during the test campaigns.	46
5.1	The three test locations.	51
5.2	Slot and symbol allocation per 5-slot, 70-symbol transmission period, for the two TDD patterns used across testing.	54
5.3	5G configuration and data validity across the three rounds.	56
6.1	Mean throughput by location, protocol and direction.	59
6.2	Round 3 round-trip time with no other traffic.	61
6.3	Ping interval against the fitted round-trip time ramp.	63
6.4	Round 3 TCP downlink: throughput, latency and retransmissions.	65
6.5	Round 1 uplink SINR and UE power headroom.	71
6.6	5G uplink at receive gain 70 and at 60.	72

6.7	5G registration and 4G attach times.	74
A.1	Static IP addresses of the testbed machines.	88
B.1	eSIM profile details.	120

LIST OF LISTINGS

5.1	<code>gnb.yaml</code> TDD configuration for the 3:1 pattern used in Round 1.	52
5.2	<code>gnb.yaml</code> TDD configuration for the rebalanced 2:2 pattern used from Round 2 onward.	53
6.1	Scheduling request fields from three consecutive <code>gnb.log</code> scheduler-metrics entries at Location A in Round 3.	64
6.2	Downlink data radio bearer configuration from <code>gnb.log</code> at Location A in Round 3.	66
6.3	First queue-full drop in <code>gnb.log</code> during the second Location A TCP downlink test in Round 3, 13.6 seconds into the test, shortened to the relevant fields.	66
6.4	<code>gnb.yaml</code> <code>cell_cfg</code> block used in Round 1.	68
6.5	Console trace from the Location A (2 m) TCP uplink test in Round 1. . .	68
6.6	Uplink fields extracted from three consecutive <code>gnb.log</code> scheduler-metrics entries for the Location A (2 m) TCP uplink test in Round 2 (2:2 pattern).	69

ACRONYMS

3GPP	3rd Generation Partnership Project (pp. 8–10 , 12 , 15–17 , 20 , 22 , 24 , 40 , 43)
5GC	5G Core (p. 19)
AF	Application Function (p. 20)
AMF	Access and Mobility Management Function (pp. 8 , 13 , 18 , 21 , 22 , 25 , 74–76 , 97 , 111 , 115 , 118)
API	Application Programming Interface (pp. 6 , 18 , 20 , 21)
AUSF	Authentication Server Function (pp. 19 , 20 , 25 , 74 , 96)
AWS	Amazon Web Services (p. 8)
BLER	Block Error Rate (pp. 72 , 73)
CI/CD	Continuous Integration/Continuous Delivery (pp. 32 , 36)
COTS	Commercial Off-The-Shelf (pp. 3 , 15 , 27–32 , 36 , 77 , 79)
CU	Centralized Unit (pp. 11 , 12 , 14 , 24 , 25 , 43)
CUPS	Control and User Plane Separation (pp. 5 , 7 , 10 , 18 , 25)
DN	Data Network (p. 21)
DU	Distributed Unit (pp. 11 , 12 , 14 , 24 , 25 , 43)
eMBB	enhanced Mobile Broadband (pp. 11 , 34)

eNodeB	Evolved Node B (pp. 11 , 46 , 69 , 72 , 77 , 95 , 105 , 114 , 118)
EPC	Evolved Packet Core (pp. 2 , 5 , 9 , 17–20 , 22 , 40 , 43 , 77 , 101 , 105)
EPS	Evolved Packet System (p. 8)
F1	F1 Interface (pp. 14 , 25)
F1-C	F1 Control Plane Interface (p. 14)
F1-U	F1 User Plane Interface (p. 14)
FCT	NOVA School of Science and Technology (p. 3)
FDD	Frequency Division Duplexing (pp. 15 , 24 , 25 , 44 , 78)
FPGA	Field-Programmable Gate Array (pp. 40 , 102 , 107)
GCP	Google Cloud Platform (p. 8)
gNodeB	Next Generation Node B (pp. xv , 2 , 11–14 , 16 , 24 , 25 , 27–30 , 32–34 , 43 , 45 , 46 , 52 , 61–68 , 72–75 , 77 , 78 , 80 , 97 , 98 , 106 , 108–111 , 114 , 115 , 118)
GPSDO	GPS Disciplined Oscillator (pp. xvi , 29 , 31 , 40 , 44 , 47 , 63 , 77 , 106–109 , 111 , 114 , 118)
GTP-U	GPRS Tunneling Protocol - User Plane (pp. 21 , 43 , 61 , 96 , 98)
HSS	Home Subscriber Server (p. 19)
IMSI	International Mobile Subscriber Identity (pp. 20 , 25 , 119 , 120)
IoT	Internet of Things (pp. 1 , 7 , 28)
LMF	Location Management Function (p. 20)
LNA	Low Noise Amplifier (pp. xv , 40 , 42 , 44 , 80)
LPWA	Low-Power Wide-Area (p. 7)
LTE	Long-Term Evolution (pp. 10 , 14 , 24 , 25 , 35 , 45)
MAC	Medium Access Control (pp. 10 , 12 , 46 , 61 , 112)
MCS	Modulation and Coding Scheme (pp. 68 , 72)
MIMO	Multiple Input Multiple Output (pp. 14 , 25 , 44)

MME	Mobility Management Entity (pp. 18 , 43 , 74 , 95 , 114 , 116 , 118)
mMTC	massive Machine Type Communications (p. 11)
mmWave	Millimeter Wave (p. 15)
N3IWF	Non-3GPP Interworking Function (p. 20)
NEF	Network Exposure Function (pp. 19 , 20 , 25)
NG	Next Generation (pp. 13 , 25 , 115)
NGAP	NG Application Protocol (pp. 21 , 43 , 46 , 74 , 75 , 112)
NG-C	NG Control Plane Interface (pp. 13 , 21)
NG-RAN	Next Generation Radio Access Network (pp. xi , 11 , 12)
NR	New Radio (pp. 14–17 , 25 , 43 , 45)
NRF	Network Repository Function (pp. 7 , 19 , 25 , 74 , 79 , 99)
NSA	Non-Standalone (pp. xi , 5 , 9 , 10 , 22 , 24 , 25 , 31 , 37 , 38 , 47 , 77 , 81 , 88)
NSSF	Network Slice Selection Function (pp. 20 , 96)
NWDAF	Network Data Analytics Function (p. 20)
OAI	OpenAirInterface (pp. 23 , 26 , 29–33 , 36)
OFDM	Orthogonal Frequency-Division Multiplexing (p. 16)
OSS	Open-Source Software (pp. 21 , 22 , 27 , 33)
OTA	Over-The-Air (pp. 32 , 36)
PCF	Policy Control Function (pp. 19 , 22 , 25 , 74 , 96)
PCRF	Policy and Charging Rules Function (p. 19)
PDCP	Packet Data Convergence Protocol (pp. 11 , 12 , 66 , 80)
PDU	Protocol Data Unit (pp. 18 , 31 , 32 , 34)
PFCP	Packet Forwarding Control Protocol (p. 21)
PGW	Packet Data Network Gateway (p. 43)
PHY	Physical Layer (pp. 12 , 30 , 61)
PLMN	Public Land Mobile Network (pp. 43 , 44 , 95 , 97 , 99 , 101 , 111 , 115 , 118)
PUSCH	Physical Uplink Shared Channel (pp. 63 , 64)

QoS	Quality of Service (pp. 11 , 15 , 20)
RAN	Radio Access Network (pp. xii , 1 , 4–6 , 8–12 , 15 , 20–26 , 28–40 , 42–48 , 50 , 51 , 55 , 71–73 , 77 , 79 , 80 , 88 , 90 , 91 , 102 , 104 , 106 , 111 , 113–115 , 118)
RF	Radio Frequency (pp. 28 , 29 , 40 , 44 , 46 , 118)
RLC	Radio Link Control (pp. 11 , 12 , 66 , 78 , 80)
RRC	Radio Resource Control (pp. 11 , 12 , 16)
RSP	Remote SIM Provisioning (p. 120)
RSRP	Reference Signal Received Power (p. 68)
RU	Radio Unit (pp. 12 , 25)
SA	Standalone (pp. xi , 2 , 3 , 5 , 9 , 10 , 23–25 , 27–29 , 31 , 35 , 77 , 79)
SBA	Service-Based Architecture (pp. 5–7 , 25 , 74 , 76 , 79)
SCEF	Service Capability Exposure Function (p. 20)
SCS	Subcarrier Spacing (pp. 16 , 17 , 29 , 68)
SCTP	Stream Control Transmission Protocol (p. 115)
SDR	Software-Defined Radio (pp. 2 , 3 , 24 , 27 , 28 , 30–34 , 36 , 39 , 40 , 43 , 44 , 46 , 73 , 103 , 104 , 118)
SGW	Serving Gateway (p. 43)
SINR	Signal-to-Interference-plus-Noise Ratio (pp. 68 , 71–73 , 76)
SMF	Session Management Function (pp. 18 , 21 , 22 , 25 , 43 , 98 , 116)
SMSF	Short Message Service Function (p. 20)
SR	Scheduling Request (p. 63)
SUCI	Subscription Concealed Identifier (pp. 20 , 25 , 31)
SUPI	Subscription Permanent Identifier (p. 20)
TDD	Time Division Duplexing (pp. xvi , 15–17 , 24 , 25 , 29 , 40 , 44 , 46 , 48 , 52–55 , 57 , 58 , 60 , 67–72 , 77–79 , 111)

UDM	Unified Data Management (pp. 19, 20, 25, 74, 96)
UDR	Unified Data Repository (pp. 19, 25, 96)
UE	User Equipment (pp. xii, xiv, xvi, 7, 8, 10, 12, 14–16, 18–21, 24, 27–36, 43, 45–52, 55, 60–63, 65, 66, 71–76, 78–81, 88, 89, 91, 98–101, 114–120, 122, 123)
UHD	USRP Hardware Driver (p. 39)
UPF	User Plane Function (pp. 13, 19–22, 25, 43, 61, 65, 74, 98)
URLLC	Ultra-Reliable Low-Latency Communications (pp. 7, 10, 11)
USRP	Universal Software Radio Peripheral (p. 27)
VM	Virtual Machine (p. 8)
Xn	Xn Interface (pp. 14, 25)

INTRODUCTION

This chapter introduces the work developed in this dissertation. Section 1.1 presents the motivation, Section 1.2 states the objectives and main contributions, and Section 1.3 outlines the structure of the remainder of the document.

1.1 Motivation

5G networks are no longer just an incremental evolution of mobile broadband; they have become the technological foundation for a new generation of applications that demand high reliability, ultra-low latency and massive connectivity. From industrial automation [20, 10, 7], immersive extended and augmented reality [39, 22, 35] to Internet of Things (IoT) applications [11, 41], these use cases require a network architecture that can meet their specific needs, placing enormous emphasis on reconfigurability of the network, softwarization across both the RAN and Core, and overall, a level of flexibility that rigid, vendor-specific infrastructure cannot provide.

Yet for a university research group, or any organization without a commercial operator's budget, deploying 5G to actually explore these use cases is usually out of reach. 5G infrastructure imposes a significant capital barrier: an IEEE Access study estimated that a full 5G rollout across the United Kingdom alone would cost between £30 billion and £50 billion, dwarfing the £2.5 billion mobile network operators spend annually on capital expenditure [30]. Beyond the raw cost, commercial 5G infrastructure is closed by design, so research conducted on it is constrained by reproducibility limitations for researchers

without access to the underlying, proprietary implementation. These financial and accessibility barriers have historically restricted real-world 5G Standalone (SA) research to a small circle of well-funded operators and vendors, slowing innovation and deployment diversity. They have also pushed research institutions and academic investigators instead towards open-source, software-defined alternatives, combined with commodity software-defined radio hardware [18, 38, 23]. Testbeds combining these open-source core platforms with software-defined radios on commodity servers have repeatedly demonstrated the feasibility of low-cost 5G network implementations [8], and these platforms make it possible to validate architectural choices, reproduce results and teach the internals of a technology that, in its commercial form, is deliberately opaque.

However, deploying a fully functional SA 5G network from open-source components alone is still a non-trivial undertaking, and getting it running is only half the problem: without a clear performance baseline, it is difficult to tell whether a given result reflects a genuine property of 5G, a limitation of the specific open-source stack, or simply a mis-configured radio. The existing literature tends to report on open-source 5G deployments in isolation, publishing throughput and latency figures with no directly comparable predecessor built with the same tools and on the same hardware to put those numbers in context. The literature has noted this lack of publicly available, comparable results [33]. A research group evaluating whether an open-source 5G testbed is viable for its needs is left to take those figures on faith.

This dissertation addresses that gap by building not one but two open-source, Software-Defined Radio (SDR)-based cellular networks side by side: a 5G SA network built on the srsRAN gNodeB [25] and Open5GS [27], and a 4G network built on srsRAN 4G [37] and the same Open5GS core, running its Evolved Packet Core (EPC) components. Sharing the core hardware and, as far as the technologies allow, the test methodology between the two generations makes it possible to treat the 4G network as a genuine baseline for the 5G one, rather than presenting 5G performance figures with nothing to compare them against. By relying exclusively on open-source software and commodity hardware, this work also aims to show that meaningful 5G SA experimentation does not require a commercial operator's budget, lowering the barrier to entry for 5G research at institutions in a similar position. Another objective of this dissertation is to leave, at its conclusion, a working 4G and 5G

network testbed at the university, so future projects can build on and explore it directly instead of starting from scratch.

1.2 Objectives and Contributions

This section outlines the primary objectives of this dissertation, as well as the main contributions expected from it.

The goals of this work are listed as follows:

- Review the architecture, protocols and open-source implementations available for 5G networks, and the equivalent 4G components needed to build a comparable baseline.
- Design and deploy an open-source, SDR-based 5G SA network and a 4G network, sharing the same core network hardware, using accessible Commercial Off-The-Shelf (COTS) equipment throughout.
- Define and execute a test methodology that empirically compares the two generations across multiple locations, traffic conditions, and directions.
- Diagnose configuration issues surfaced during testing, documenting the reasoning behind each change rather than only the final configuration.
- Leave a working, documented 4G and 5G testbed in place for future research and coursework at NOVA School of Science and Technology (FCT).

The main contributions of this dissertation are the following:

- A functional, dual-generation 4G and 5G testbed, built entirely from open-source software and COTS SDR hardware, reusable by future projects.
- An empirical, quantitative multi-round comparison of 4G and 5G performance (throughput, latency, and reliability) under matched distance and traffic conditions.
- A documented report of how specific configuration issues were diagnosed and corrected across successive test rounds, intended as a practical reference for others deploying a similar open-source stack.

1.3 Report Structure

The remainder of this document is structured as follows:

- Chapter 2 introduces the fundamental concepts and architecture of 5G networks, covering both the RAN and the core, as background for the chapters that follow.
- Chapter 3 surveys existing open-source 5G implementations and related testbed deployments described in the literature.
- Chapter 4 presents the testbed implementation: the hardware components, the core network design, the RAN design, and the interfaces connecting them.
- Chapter 5 details the methodology used to evaluate it across three successive test rounds and the changes made to the 5G RAN configuration.
- Chapter 6 presents the results of the performance evaluation, comparing the 4G baseline against the 5G network as it evolved across test rounds.
- Chapter 7 concludes the dissertation by summarizing the main findings, their limitations, and the work left open for the future.

FUNDAMENTALS OF A 5G NETWORK

This chapter introduces the theoretical background necessary to understand the design and evaluation of the 5G testbed developed in this project. Section 2.1 describes the architectural principles that distinguish 5G from previous mobile generations, such as the Service-Based Architecture, Control and User Plane Separation, network slicing, virtualization, and cloud-native network functions. Section 2.2 presents the two deployment models adopted by operators, Non-Standalone and Standalone, motivating the choice of a SA deployment for this work. Sections 2.3 and 2.4 detail, respectively, the architecture, components, and interfaces of the 5G RAN and Core network. Finally, Section 2.5 surveys the existing landscape of open-source software available for implementing 5G networks, covering both Core and RAN implementations.

2.1 Introduction to the 5G Architecture

The transition from 4G to 5G brought a fundamental redesign of how the network is built. While 4G's EPC was conceived as a set of tightly coupled, purpose-built network elements, 5G was designed from the ground up with cloud computing and IT infrastructure in mind, emphasizing modularity, virtualization, and standardized, API-driven communication between components. This shift enables the network to be deployed (and scaled, if necessary) with a flexibility that 4G architectures could not offer.

Figure 2.1 provides a high-level view of the 5G architecture, illustrating all components of the 5G Core, how their constituent network functions relate to one another, the 5G RAN's position on the network edge, and how it connects to the Core network. The remainder

of this section examines the key architectural principles that define this design, before Sections 2.3 and 2.4 detail the RAN and Core individually.

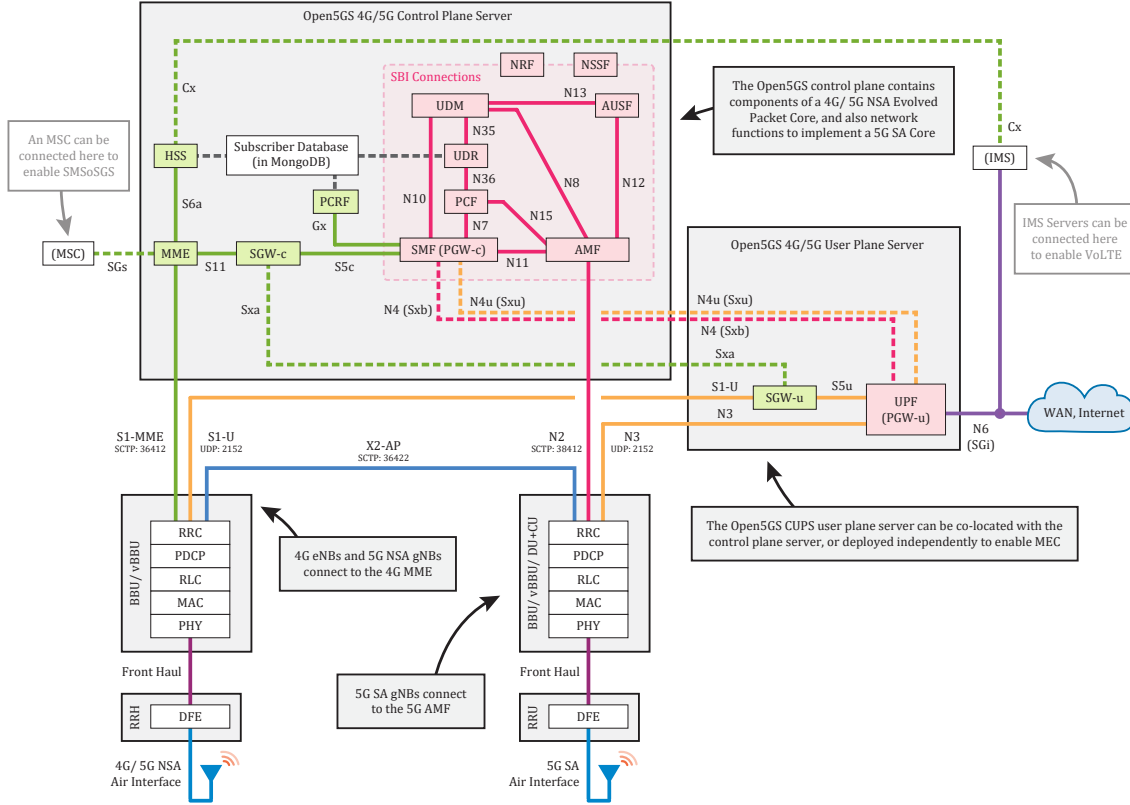


Figure 2.1: 5G Network architecture diagram from the Open5GS project [27]. Used under the AGPL-3.0 license.

The 5G architecture represents a significant evolution from previous generations of mobile networks. Compared to these, 5G was designed to natively support a much broader and more divergent range of use cases, from enhanced mobile broadband to ultra-reliable low-latency and massive machine-type communications, while also strengthening network security through a unified authentication framework.

One of the clearest reflections of this shift is the Service-Based Architecture, which marks a departure from the monolithic architecture of 4G and previous generations. 5G adopts a service-based approach, where network functions are designed to be modular units that provide services to the network and can be independently configured and implemented. These network functions expose their capabilities and communicate via HTTP/2 using a standardized REST Application Programming Interface (API), enabling

interoperability between services.

In this paradigm, functions act as either Service Providers or Service Consumers, depending on whether they offer or use specific services. The Network Repository Function (NRF), which will be discussed in more detail further on, plays a crucial role in this architecture by maintaining a registry of available network functions and their services, enabling dynamic discovery and interaction among them.

Closely tied to the Service-Based Architecture (SBA) is Control and User Plane Separation (CUPS): in 5G systems, the separation of the control plane and user plane is one of the fundamental design principles that sets them apart from previous generations, defining them as independent functional entities that can be deployed, scaled, and managed separately, as shown in Figure 2.2.

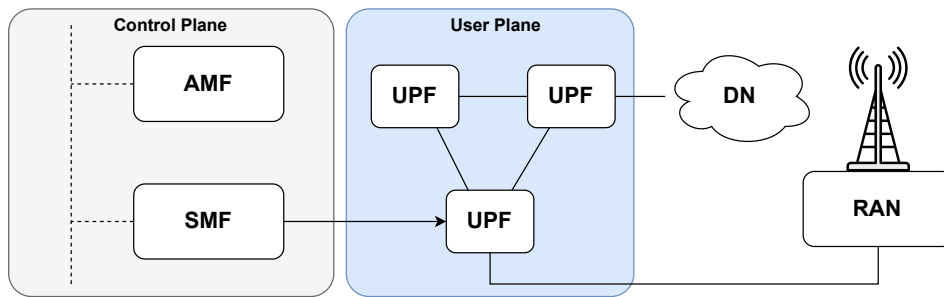


Figure 2.2: Control and User Plane Separation.

This means network operators can dimension their networks to match real traffic patterns, deploying control plane functions in centralized locations for efficient signaling, while distributing user plane functions closer to the network edge to minimize service latency and jitter, improving throughput and overall user experience.

Building on this separation, network slicing is a further key feature of the 5G architecture. The concept itself is not new, but 5G builds upon it, driven by the rising popularity of diverse use cases such as Low-Power Wide-Area (LPWA) networks, IoT devices, and Ultra-Reliable Low-Latency Communications (URLLC) applications.

Network slicing lets operators create and deploy multiple virtual networks on top of a shared physical infrastructure, each optimized for specific use cases and services. 5G network slicing also allows operators to dynamically allocate resources based on the subscription and applications running on the UE.

In contrast to 4G Evolved Packet System (EPS) slicing, a 5GS network slice consists of both RAN and Core components and allows for multiple concurrent slices to be active on a single UE, with the only drawback being that the RAN and Access and Mobility Management Function (AMF) must be shared across those network slices.

Virtualization underpins all of these features. Traditionally, mobile network components were tied to hardware appliances that ran only a specific function. This meant that software had to be pre-installed and configured for each network function, leading to inefficiencies in resource utilization and scalability. In 5G, network functions can be virtualized and run in Virtual Machines (VMs) or containers, allowing them to share physical hardware with other functions. This approach provides greater flexibility and resource optimization, since functions are no longer tied to hardware and can be dynamically allocated and decommissioned based on current network demands.

Finally, building upon virtualization, 5G network functions are designed to be cloud-native, meaning they are developed from scratch and optimized to be run in cloud environments, such as public clouds (Amazon Web Services, Google Cloud Platform, Microsoft Azure, etc.) or private operator clouds. This means network operators can run part of their 5G network in a cloud environment, leveraging its inherent resilience and scalability, as well as its global reach to deploy network functions closer to end-users and reduce network latency.

Because these functions are cloud-native, operators can also use orchestration and automation tools, such as Kubernetes and OpenStack [23], to manage the lifecycle of network functions, including deployment, scaling, and healing, further improving the efficiency of their 5G networks.

Security is another dimension along which the 5G architecture departs from its predecessors. Rather than treating authentication as a function tied to a single access technology, 5G defines a unified authentication framework that serves both 3rd Generation Partnership Project (3GPP) and non-3GPP access networks through a set of network functions, along with mechanisms to conceal the subscriber's permanent identity over the air. Subsection 2.4.1 revisits this framework when detailing the specific Core network functions responsible for enforcing it.

2.2 Standalone vs Non-Standalone 5G Deployments

The adoption of 5G technology has been gradual for many network operators, as the advent of 5G networks, while innovative, also required a significant revamp of existing infrastructure, bringing considerable costs, logistical challenges, and unavoidable service disruptions. Before evolving into fully independent 5G networks, many operators opted for integrating 5G radio capabilities into their existing 4G infrastructure. This hybrid approach is known as Non-Standalone, whereas fully independent 5G networks are known as Standalone deployments. In this section, we explore the differences between these two deployment modes, illustrated in Figure 2.3.

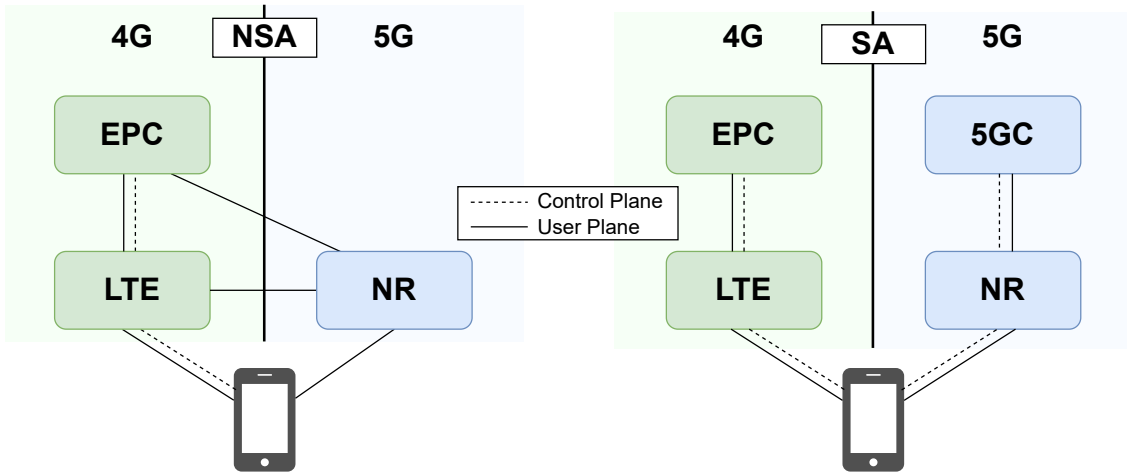


Figure 2.3: Architecture of 5G NSA Network vs SA Network deployments.

2.2.1 Non-Standalone (NSA) 5G Deployments

Non-Standalone deployments were introduced as a way to accelerate the rollout of 5G services while keeping costs minimal by leveraging existing 4G infrastructure while implementing 5G RAN components. Standardized by the 3GPP under Deployment Option 3 [5], in this architecture, the 4G EPC continues to handle control plane functions, such as mobility management and session control, but user plane data is transmitted over the 5G RAN, allowing users to benefit from the enhanced data throughput of 5G technology.

However, this approach is fundamentally constrained by its reliance on the legacy 4G core. Consequently, it cannot leverage the advanced, cloud-native capabilities of the 5G

system described in Section 2.1. Furthermore, while the operator's capital expenditure is initially kept low, Non-Standalone (NSA) deployments significantly increase network complexity. Operators must manage and synchronize both 4G and 5G nodes concurrently, increasing operational burden. Ultimately, NSA served as a necessary transitional phase, allowing operators to monetize early 5G radio investments while preparing for the complex migration to SA architectures.

2.2.2 Standalone (SA) 5G Deployments

Standalone deployments represent the true end-to-end realization of the 5G system as originally defined by 3GPP Release 15 [3] and expanded in subsequent releases. Formally known as 3GPP Deployment Option 2, this architecture is completely decoupled from legacy Long-Term Evolution (LTE) networks. Both the Radio Access Network and the Core are built entirely on 5G protocols, utilizing the full suite of cloud-native network functions. In this model, all control plane and user plane traffic is natively routed through the 5G Core, fully realizing CUPS.

Although SA deployments require a substantial upfront investment, they offer critical technical advantages over NSA architectures. Operating a native 5G environment is the only way to facilitate strict URLLC and dynamically instantiate end-to-end network slices for enterprise use cases. By eliminating the reliance on legacy 4G nodes, SA networks reduce long-term signaling overhead. They are inherently more future-proof, allowing operators to seamlessly integrate emerging 3GPP standards and dynamically scale resources to meet evolving market demands.

2.3 5G RAN

The 5G Radio Access Network constitutes the interface between UEs and the 5G Core network, providing the radio connectivity and resource management functions essential to mobile communication. It encompasses the Physical, Medium Access Control (MAC), and higher layers of the protocol stack, and enables data transmission, mobility management, and strict service differentiation. Beyond providing radio connectivity, the 5G RAN plays a central role in facilitating the diverse service categories defined for 5G networks, such as

enhanced Mobile Broadband (eMBB), URLLC and massive Machine Type Communications (mMTC). To meet these requirements, the 5G RAN has evolved into a highly flexible, programmable architecture that can operate across a wide range of frequency bands while actively adapting its numerology, scheduling, and resource allocation strategies.

Compared with its predecessors, the NG-RAN has undergone extensive softwarization and disaggregation of the RAN. This architectural shift replaces fixed hardware entities with modular components and virtualized functions that can be deployed in cloud-native environments. As a result, in this generation, the RAN acts more like a platform that interacts closely with the 5G Core network, with its functions delivering optimized connectivity and services.

This section provides a detailed overview of the 5G RAN architecture and features, highlighting its key components and functionalities.

2.3.1 NG-RAN Architecture

The NG-RAN architecture was fundamentally redesigned from the beginning to maximize deployment flexibility by disaggregating traditional base station functions into modular logical nodes. At its core, the NG-RAN is built around the Next Generation Node B (gNodeB) (Figure 2.4), the 5G equivalent of 4G's eNodeB, which is responsible for radio transmission, user-plane data forwarding, and control-plane signaling with the 5G Core network.

The gNodeB is further divided into three main units, as illustrated in Figure 2.4:

Centralized Unit

This unit handles higher-layer protocol processing, specifically the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP) layers, including mobility, security, and Quality of Service (QoS) flows, and is typically deployed in centralized locations and/or cloud environments. The Centralized Unit (CU) can manage multiple Distributed Units (DUs), coordinating resource usage across a wide area.

Distributed Unit

The DU handles lower-layer protocol processing, including Radio Link Control (RLC),

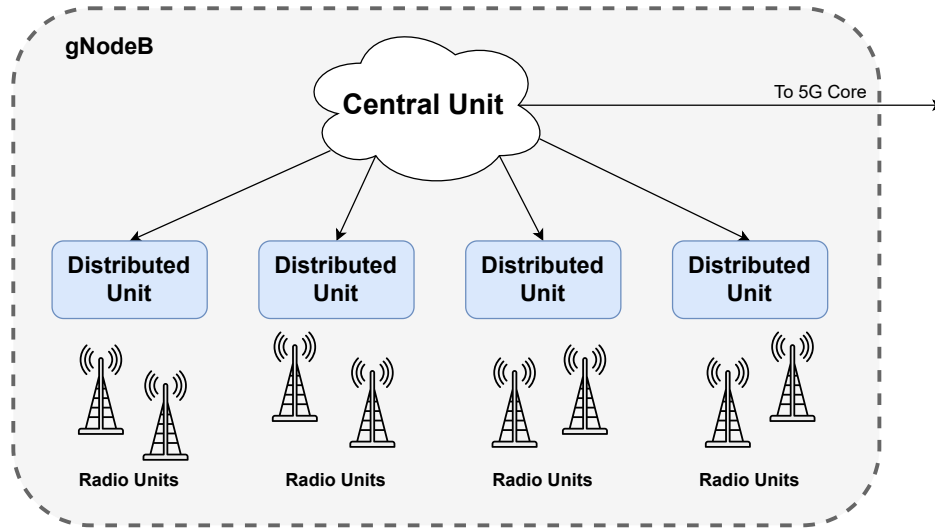


Figure 2.4: gNodeB Architecture [36].

MAC, and Physical Layer (PHY) functions. Multiple DUs can be connected to the same CU. They are strategically deployed closer to the cell sites to minimize latency and optimize radio resource management.

Radio Unit

The Radio Units (RUs) are responsible for physical radio-frequency functions, including signal transmission, reception, amplification, and antenna management. They consist of the antennas that interact with the UEs, along with the necessary back-end, and are typically located at the cell site. They are connected to the DU through front-haul links.

Figure 2.5 shows how the PHY, MAC, RLC, PDCP and RRC layers described above map onto the overall 5G RAN protocol stack.

2.3.2 RAN Interfaces

The RAN interfaces, illustrated in Figure 2.6, are communication channels that enable the exchange of signaling and user data between gNodeB components and the 5G Core network. 3GPP standardizes these interfaces to ensure interoperability between different vendors' equipment and to facilitate the deployment of multi-vendor networks.

The interfaces that make up the NG-RAN are described below:

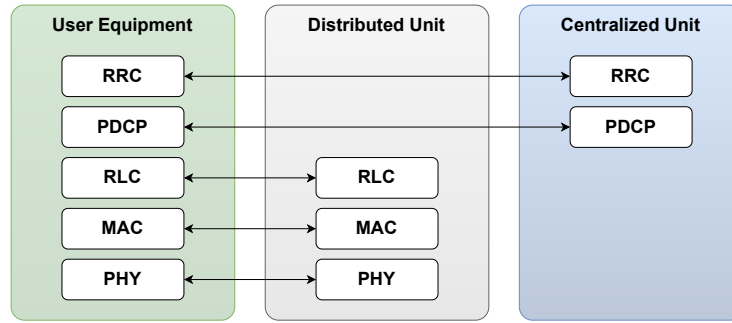


Figure 2.5: 5G RAN Protocol Stack across the three types of units [36].

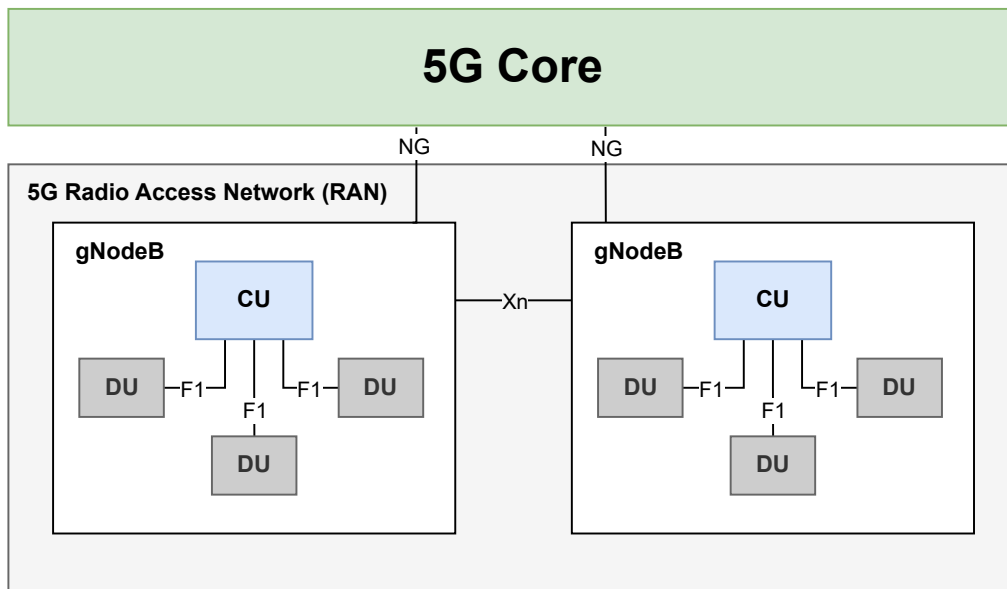


Figure 2.6: 5G RAN Interfaces between the gNodeB units and the 5G Core network [36].

NG Interface

The Next Generation (NG) interface connects the gNodeB to the 5G Core. It is structurally divided into two sub-interfaces: NG Control Plane Interface (NG-C) for control plane signaling with the AMF, and NG-U for user plane data transfer with the User Plane Function (UPF), supporting both control plane and user plane communications.

Xn Interface

The Xn Interface (Xn) interface connects gNodeBs to each other, enabling inter-cell coordination and handover management. This interface also supports dual connectivity scenarios, where a UE can be simultaneously connected to multiple gNodeBs for improved performance and reliability.

F1 Interface

The F1 Interface (F1) interface bridges the DUs to the CU within the disaggregated gNodeB. It is also divided into two sub-interfaces: F1 User Plane Interface (F1-U) for user plane data and F1 Control Plane Interface (F1-C) for control plane signaling.

2.3.3 RAN Features

5G NR builds on pre-existing technologies introduced in previous generations while adding new features to meet the ever-growing requirements of modern mobile communications. One feature that sets NR apart from its predecessors is the ability to use advanced Multiple Input Multiple Output (MIMO) technology, including Massive MIMO.

In 4G LTE, MIMO capabilities were already employed, dividing the available antennas between different spatial cells to increase spectral efficiency and throughput. However, 5G New Radio (NR) takes this concept further by significantly increasing the number of antennas at the base station, often with 100+ antennas per gNodeB (Figure 2.7), allowing for multiple User Equipments to be served simultaneously by a single node using the same time-frequency resources.

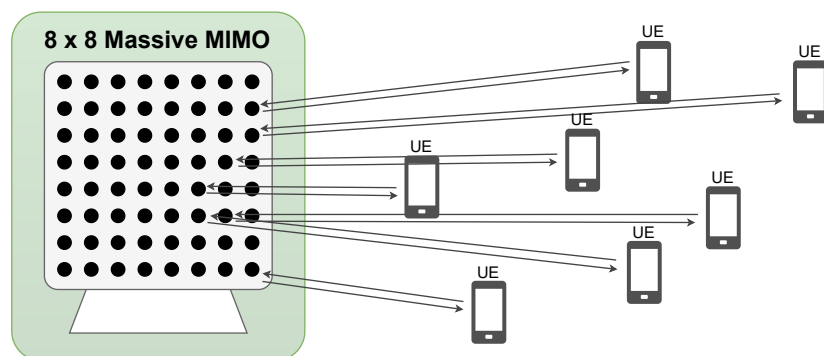


Figure 2.7: Example of massive MIMO, in which one antenna array serves several UEs on the same time-frequency resources.

Each antenna is capable of beamforming, dynamically adjusting the radio signal directly toward the UE rather than broadcasting omnidirectionally. By dynamically adjusting the phase and amplitude of the signal at each antenna, the system can create highly directional beams that target specific users or cells. This improves signal strength and efficiency, reducing wasted energy and interference with other devices and enhancing QoS and throughput.

Driven by increasing demand for higher bandwidth, 3GPP Release 17 [4] extended the operating spectrum up to 71 GHz, subdividing it into two frequency ranges, as shown in Figure 2.8.

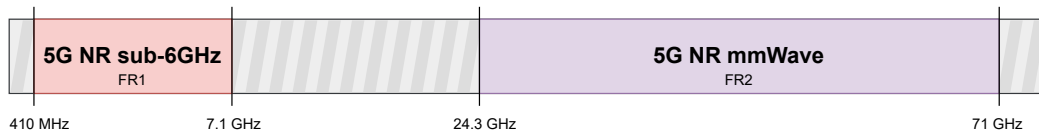


Figure 2.8: 5G Frequency Bands: FR1 (410 MHz to 7.125 GHz) and FR2 (24.25 GHz to 71 GHz).

- Frequency Range 1 (FR1): Known as the Sub-6GHz range, it encompasses frequencies from 410 MHz to 7.125 GHz.
- Frequency Range 2 (FR2): Known as mmWave, it covers frequencies from 24.25 GHz to 71 GHz. This range greatly increases available bandwidth and, therefore, throughput, but it suffers from significant path loss. It is important to note that many COTS devices are not compatible with this frequency range.

Beyond the frequency range already discussed, a RAN must also decide how uplink and downlink traffic share the available spectrum, a choice known as the duplex mode. 3GPP defines two duplex modes for NR [1]: Frequency Division Duplexing (FDD), in which uplink and downlink are transmitted simultaneously over two separate, paired frequency channels, and TDD, in which uplink and downlink share a single frequency channel, taking turns in time instead. FDD guarantees each direction a fixed, dedicated share of spectrum at all times, but requires a pair of channels separated by a duplex gap, which is not always available. TDD, by only requiring a single channel, is more spectrally flexible and, as it does not need a duplex gap, allows the split between uplink

and downlink capacity to be adjusted to match traffic demand, at the cost of requiring tight time synchronization between the gNodeB and every UE it serves, so that no UE transmits while another is meant to be receiving.

TDD's time-domain split, defined by 3GPP in TS 38.211 [1], divides each 10 ms radio frame into 10 subframes of 1 ms, a structure inherited unchanged from 4G. The two generations diverge, however, in how each subframe is further split: in 4G, a subframe is always composed of 2 slots of 0.5 ms, each carrying 7 OFDM symbols, for a fixed 14 symbols per subframe, since 4G only ever operates at a single, fixed subcarrier spacing. 5G NR instead fixes the slot itself at 14 OFDM symbols regardless of numerology, and lets the Subcarrier Spacing (SCS) in use dictate the number of slots per subframe. Both quantities are set by a single numerology index μ , defined by 3GPP in TS 38.211 [1, §4.2], through the subcarrier spacing

$$\Delta f = 2^\mu \times 15 \text{ kHz}, \quad (2.1)$$

so that the useful symbol duration $T_u = 1/\Delta f$ halves with every increment of μ , while the number of slots per subframe grows to compensate:

$$N_{\text{subframe},\mu}^{\text{slot}} = 2^\mu. \quad (2.2)$$

Concretely: at $\mu = 0$ (15 kHz SCS), 1 slot per subframe (14 symbols, 1 ms long), the same 14 symbols per subframe as 4G, but repackaged as a single slot rather than two; at $\mu = 1$ (30 kHz), 2 slots (28 symbols); and at $\mu = 2$ (60 kHz), 4 slots (56 symbols). Figure 2.9 compares this structure across both generations and these three lowest 5G numerologies.

3GPP TS 38.213 [2] defines how a TDD carrier assigns these slots and symbols to each direction, through the TDD-UL-DL-ConfigurationCommon RRC parameter. This configuration specifies a periodicity (a whole number of slots, repeated for the duration of the connection), the number of slots at the start of that period reserved entirely for downlink, and the number of slots at the end of the period reserved entirely for uplink. Between these two, 3GPP additionally allows for one further, partially-allocated slot, commonly referred to as the flexible slot, in which a configurable number of symbols at the start are assigned to downlink and a configurable number of symbols at the end are assigned to uplink; any remaining symbols in that slot, along with any slots not explicitly assigned to either direction, serve as a guard period, giving the radio time to switch between transmitting

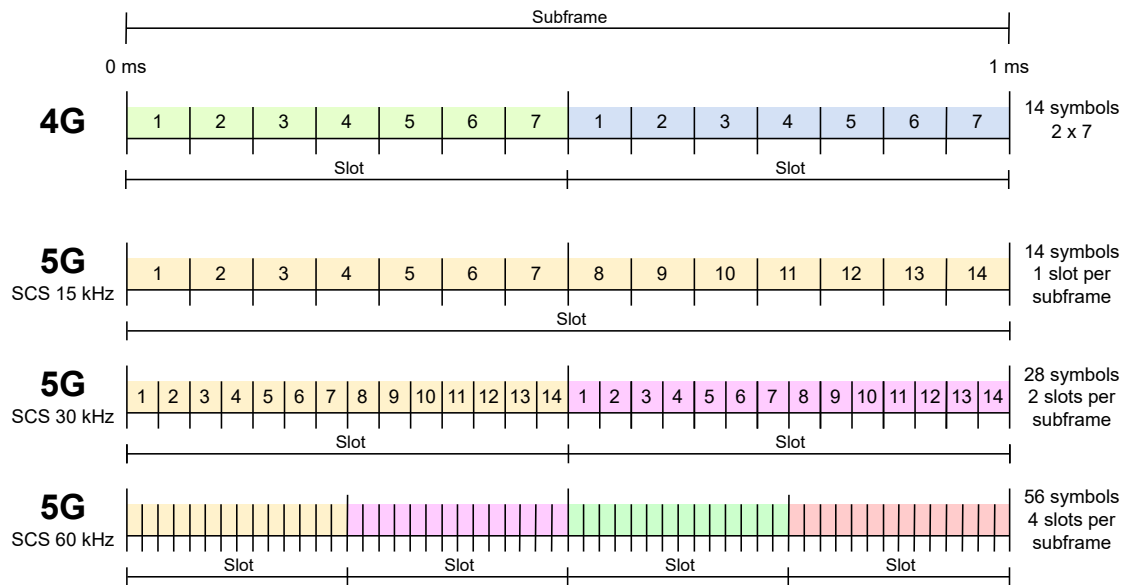


Figure 2.9: Frame, subframe, slot and symbol structure of a 1 ms subframe in 4G and in 5G NR at 15, 30 and 60 kHz SCS, per 3GPP TS 38.211 [1].

and receiving without interference between the two directions. Figure 2.10 illustrates this structure for a generic TDD period.

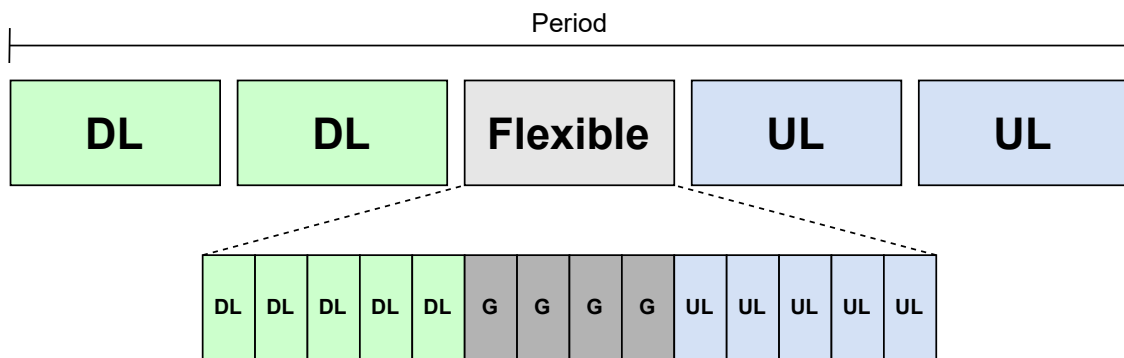


Figure 2.10: Separation of downlink, uplink and flexible slots in a generic TDD period, per 3GPP TS 38.213 [2].

2.4 5G Core

As mentioned in Section 2.1, a key difference between the 5G Core network and the EPC used in 4G networks is its service-based architecture. Network Functions are the main components of the 5G Core and are designed to be modular and communicate with

each other using standardized APIs. Figure 2.11 shows these network functions and the interfaces that connect them.

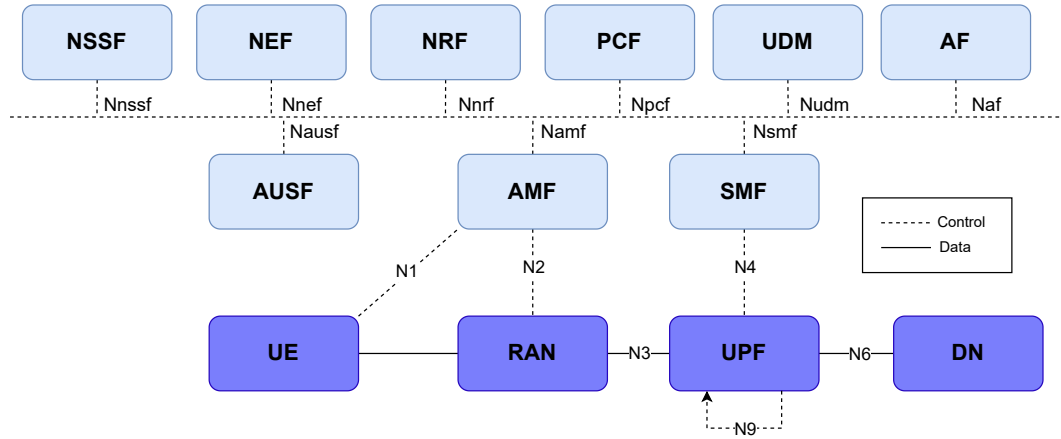


Figure 2.11: 5G Core Network Architecture [36].

2.4.1 Core Network Functions

Control and User Plane Separation defines the separation of control plane network functions and user plane network functions. As defined by Rommer et al. [32], the main control plane functions are described below:

2.4.1.1 Control Plane

Access and Mobility Management Function

The AMF is the key control plane function in the 5G Core network, the counterpart of the EPC's Mobility Management Entity (MME). It is responsible for handling connection and mobility management tasks for the UE. It manages registration, authentication, reachability, and mobility procedures, ensuring seamless connectivity as users move across different network areas. The AMF also interfaces with other network functions to facilitate session management and policy enforcement.

Session Management Function

The Session Management Function (SMF) is the function responsible for establishing, modifying, and releasing Protocol Data Unit (PDU) sessions, determining the PDU

session type (IPv4, IPv6, IPv4v6, Ethernet, or Unstructured), and selecting the UPF that will handle the user plane traffic for each session. It also manages IP address allocation for the UE and, along with the Policy Control Function (PCF), enforces session-management policies.

Policy Control Function

Equivalent to EPC's Policy and Charging Rules Function (PCRF), the PCF handles policy control and decision-making within the 5G Core network. In addition, the PCF also provides policy information to the UE, such as Access Network Discovery and Selection Policies and UE Route Selection Policies.

Unified Data Management & Unified Data Repository

The Unified Data Management (UDM), along with the Unified Data Repository (UDR), serves as the 5G Core (5GC) equivalent to EPC's Home Subscriber Server (HSS). These functions store and manage subscriber data and profiles. The UDR stores user subscription information, including authentication credentials, service entitlements, and policy data. The UDM serves as a front end to the registry, interacting with other network functions, specifically the PCF and NEF, to facilitate user authentication, authorization, and mobility management.

Network Repository Function

The Network Repository Function is a central component of the 5G Core network that serves as a service registry and discovery function. It maintains a database of available network functions and the services they offer, allowing other network functions to dynamically discover and interact with each other without needing to be configured beforehand. The NRF plays a crucial role in enabling the service-based architecture of the 5G Core, facilitating communication and coordination among various network functions.

Authentication Server Function

The Authentication Server Function (AUSF) handles authentication procedures within the 5G Core network. It works in conjunction with the UDR/UDM to authenticate UEs attempting to access the network. The AUSF also provides security

parameters in roaming contexts and ensures that authentication processes comply with 5G security standards.

Network Exposure Function

Equivalent to EPC's Service Capability Exposure Function (SCEF), the Network Exposure Function (NEF) acts as an intermediary between the 5G Core network and external application functions. It exposes network services and events, such as a UE's location, reachability, roaming status, and loss of connectivity, to authorized applications and functions, enabling them to interact with the network securely and in a controlled manner. The NEF also handles API management, security, and policy enforcement for these interactions.

Other, more specific control plane functions, such as the Network Slice Selection Function (NSSF), Network Data Analytics Function (NWDAF), Non-3GPP Interworking Function (N3IWF), Application Function (AF), Short Message Service Function (SMSF) and Location Management Function (LMF) are also defined, but not detailed here, due to their limited scope of purpose.

In terms of security advancements compared to 4G, the AUSF and UDM are the network functions that put into practice the unified authentication framework introduced in Section 2.1. Together, they let a single authentication procedure serve both 3GPP and non-3GPP access (the latter via the N3IWF), rather than requiring separate mechanisms for each access technology as in previous generations. This is paired with subscriber identity concealment: the UE's permanent identifier (Subscription Permanent Identifier (SUPI)) is encrypted into a Subscription Concealed Identifier (SUCI) before ever being sent over the air, closing a long-standing weakness of 4G, where the International Mobile Subscriber Identity (IMSI) could be exposed in the clear during initial attach.

2.4.1.2 User Plane

The user plane in the 5G Core network consists of a single main function: the User Plane Function. It handles all user data traffic between the RAN and external data networks. It performs packet routing and forwarding, QoS enforcement, traffic shaping, usage reporting, and, in some cases, lawful interception. Multiple UPFs can be deployed in series, co-existing

within the same 5G Core network, enabling distributed architectures and facilitating scalability if necessary.

2.4.2 Core Interfaces

Network functions inside the 5G Core exchange information through a common access medium using APIs. For communication with external components, it relies on a set of interfaces that define how UEs, the RAN, and the various network functions exchange information. These interfaces support the service-based architecture of the 5G Core and communicate directly with network functions. Table 2.1 summarizes the main interfaces illustrated in Figure 2.11.

Interface	Connections	Protocol
N1	UE ↔ AMF	NAS over NG-C
N2	RAN ↔ AMF	NGAP
N3	RAN ↔ UPF	GTP-U
N4	SMF ↔ UPF	PFCP
N6	UPF ↔ external DNs	IP-based Protocols
N9	UPFs ↔ UPFs	GTP-U

Table 2.1: 5G Core Network Interfaces between the multiple Network Functions in the 5G Core network [32, 36].

2.5 Existing Open-Source Software for 5G Networks

This section provides a comprehensive review of existing open-source software packages for 5G networks, as this falls within the scope of this thesis. The goal is to identify and analyze the most relevant and widely adopted open-source projects that can be used to prototype 5G networks. Overall, Open-Source Software (OSS) has become a growing trend in computer engineering, and the telecommunications sector is no exception. The adoption of OSS for 5G networks has been driven by the need for cost-effective solutions that can adapt to the rapidly evolving mobile communications landscape, moving away from traditional vendor-locked, closed-source solutions.

These tools play a crucial role in research, prototyping, and, in some cases, even production deployments of 5G networks, as they allow researchers to experiment freely with 5G architectures without relying on expensive proprietary solutions. The existing

suite of open-source projects already spans both the 5G Core and RAN, as well as tools for support, testing, performance evaluation, and validation.

2.5.1 Open-Source 5G Core Implementations

On the core side, many projects have emerged to provide an accessible way to deploy a complete 5G core network using OSS components, with varying maturity levels. Some of the most notable projects will be described below, with a brief overview in Table 2.2.

2.5.1.1 Open5GS

Open5GS [27] is an open-source implementation of a 5G Core network, written in C. Out of the box, it provides a set of both 5G Core network functions, such as control plane functions (AMF, SMF, PCF, etc.) and the user plane function (UPF), as well as 4G EPC functions for NSA implementations and backward compatibility. It is lightweight and easy to deploy, with extensive documentation that guides developers from installation to configuration and, finally, operation. Each network function in Open5GS is defined in a separate configuration file, making it straightforward to modify individual components without affecting the entire system. Open5GS is one of the most widely adopted open-source 5G Core implementations, as reported in [9, 19, 23, 38], making it a popular choice for 5G network research and prototyping.

2.5.1.2 Free5GC

Free5GC [13] is another open-source project that implements the 5G Core in Golang and integrates well with web services. It provides a full suite of 5G Core network functions, as according to 3GPP Release 15 [3]. Although less common than other deployments, Free5GC has been used in academia, as reported in [19], and has an active community that contributes to its development and maintenance.

2.5.1.3 Magma

Initially developed by Facebook, Magma [12] provides 2G, 3G, 4G, and 5G Core network implementations and was made open source in 2019. Despite being relatively new, Magma

has seen a spike in contributors and users because it was designed from scratch for implementation in poor and rural areas, where connectivity is scarce. Within their project, they offer a full 5G SA Core network implementation, written mainly in C++, Go, and Python.

Although not as widely adopted as other solutions because of its novelty and limited documentation, Magma has been used in research, as described in [18].

2.5.1.4 OpenAirInterface Core Network

The OpenAirInterface Software Alliance, composed of major global players (such as Nokia, Vodafone, or Ericsson), provides a 5G Core [28] implementation, mainly maintained by its community. It is written in C++ and provides a complete set of 5G Core network functions, based on containers, making it easy to deploy and manage. The project is well documented, with many tutorials and examples, and has an active community. It is widely used in academia and research, as mentioned in [9, 19, 40].

Project	Programming Language	Supported Generations	Maturity level
Open5GS	C	4G/5G	Most widely adopted open-source 5G Core implementation, with extensive documentation and community support. Used in multiple research projects.
Free5GC	Go	5G	Well-maintained project with a focus on web-services integration. Less popular than Open5GS but still used in academia.
Magma	C++, Go, Python	2G/3G/4G/5G	Newer project with a focus on rural connectivity. Growing community and adoption, but less mature than other solutions.
OAI Core	C++	4G/5G	Backed by major industry players, well-documented and widely used in academia. Container-based deployment for ease of management.

Table 2.2: Overview of different open-source 5G Core implementations.

2.5.2 Open-Source 5G RAN Implementations

On the RAN side, many open-source projects have also emerged to provide 5G RAN implementations. Among them, two key players stand out: srsRAN and OpenAirInterface RAN, due to their maturity, extensive documentation, and active communities. These projects are described below, along with UERANSIM, a simulator that stands in for a RAN when no radio hardware is available, followed by a brief comparison in Table 2.3.

2.5.2.1 srsRAN

srsRAN [25] is an open-source software suite that provides a complete implementation of the 5G RAN, as defined by 3GPP. It is written in C++ and is designed to be modular, allowing the developer to customize both the CU and DU components of the gNodeB separately.

It supports all FR1 bands, FDD/TDD modes, all bandwidths up to 100 MHz TDD and 50 MHz FDD, 15/30 kHz subcarrier spacing, 256-QAM modulation, Network Slicing, among other features. srsRAN integrates well with Open5GS and is widely adopted in both academia and industry, as reported in [19, 23, 38, 40].

The srsRAN project also provides 4G LTE support through its srsRAN 4G project [37], allowing for backward compatibility with existing 4G networks.

In June 2026, midway through this dissertation’s development, the project was renamed OCUDU and migrated to a new repository under Linux Foundation governance [25]. For consistency with the literature reviewed in this dissertation, and with most of the project’s testbed development, which predates the rename, this dissertation refers to the 5G RAN implementation as srsRAN throughout.

2.5.2.2 OpenAirInterface

Along with a Core Network, the OpenAirInterface Software Alliance also provides a 5G RAN implementation [29] that implements full gNodeB and UE functionality. Also written in C and C++, it is the most complete 5G open-source RAN stack, supporting both NSA and SA deployments, monolithic gNodeB as well as disaggregated CU/DU architectures and FR2 frequency band support. Like srsRAN, it is widely used in academia, as noted in [9, 19].

2.5.2.3 UERANSIM

UERANSIM [16] is not a real RAN implementation, but rather a 5G UE and RAN simulator, written in C++. It can be used to test 5G Core networks without the need for SDRs. It supports SA deployments but not NSA deployments, and can simulate multiple UEs simultaneously.

Feature	srsRAN	OpenAirInterface RAN
Frequency Bands Supported	All FR1 (sub-6 GHz) bands	All FR1 and FR2 bands
Deployments supported	NSA and SA	NSA and SA
Duplex Mode	FDD/TDD	FDD/TDD
Sub-Carrier Spacing	15/30 kHz	15 and 30 kHz (FR1), 120 kHz (FR2)
Bandwidth (BW)	10, 20, 40, 80, 100 MHz	10, 20, 40, 80, 100 MHz
Programming Language	C & C++	C & C++
Antenna Configuration	SISO	SISO and MIMO
Documentation & Support	Extensive	Fair
Deployment Complexity	Easier and faster to deploy	Complex to deploy

Table 2.3: Comparison of the open-source 5G RAN projects, as described by Mamushiane et al. [23].

2.6 Summary

The material covered above maps out both the architectural rationale and the practical toolset behind the testbed developed in this dissertation. Sections 2.1 and 2.2 established why 5G looks so different from previous generations at a structural level: the SBA replaces monolithic network elements with independently deployable services, CUPS decouples control and user plane functions so each can be scaled and placed according to its own latency and throughput needs, the advanced network slicing and cloud-native virtualization let a single physical infrastructure host multiple logical networks tailored to distinct use cases, and a unified, SUCI-based authentication framework closes the IMSI-exposure weakness inherited from 4G. Building on those principles, the contrast between NSA and SA deployments motivated this dissertation’s choice of a full SA architecture, free from any dependence on legacy 4G core infrastructure.

Sections 2.3 and 2.4 then worked through the two halves of the architecture in detail. On the RAN side, the gNodeB is composed of CU, DU, and RU components and the interfaces that connect them (F1, NG, Xn), alongside the radio-level features, Massive MIMO, beamforming, and the FR1/FR2 frequency split, that set 5G NR apart from LTE. On the Core side, the control plane functions (AMF, SMF, PCF, UDM/UDR, NRF, AUSF and NEF) and the single user plane function (UPF) were examined individually, along with the interfaces that tie them together.

Finally, Section 2.5 surveyed the open-source ecosystem available for implementing all

of the above, comparing Open5GS, Free5GC, Magma and OAI Core on the Core side, and srsRAN, OAI RAN and UERANSIM on the RAN/simulation side.

With this foundation in place, Chapter 3 reviews related work that has similarly explored open-source 5G deployments and positions this dissertation's contributions relative to the existing body of research.

STATE OF THE ART

Deploying 5G networks using OSS has become a research area, with many researchers exploring the feasibility of deploying 5G networks with open-source components. Researchers have taken very different approaches, from small-scale testbeds to complex environments integrated with commercial equipment. This chapter groups the relevant works surveyed by the kind of question they set out to answer. Section 3.1 covers works that build and report on a single, self-contained 5G SA testbed, with a particular focus on UE integration. Section 3.2 covers works that instead compare multiple open-source implementations or deployment approaches directly against one another. Section 3.3 covers works concerned with automation and with scale, from continuously tested lab deployments to real-world and production networks. Section 3.4 closes the chapter with a comparative overview of the reviewed testbeds, summarized in Table 3.1. Most of these works were sourced from srsRAN’s website, under the Research tab, complemented by a small number of additional papers found through a broader literature search.

3.1 Single-Testbed Deployments and UE Integration

Tío et al. [38] implemented a complete 5G SA testbed using Open5GS for the Core and srsRAN for the gNodeB, deployed on an Intel i7-12700 host with two SDRs, a high-end Universal Software Radio Peripheral (USRP) X310 and a mid-range USRP B200, operating on band n3 (FR1). The Core network was deployed in containers, using Docker Compose. Their main contribution lies in comparing three fundamentally different UE implementations side by side: srsUE, srsRAN’s own 5G UE software-defined implementation; a COTS

Samsung Galaxy A33 5G handset, provisioned with a programmable SIM card from the Open Cells project; and a Sixfab 5G HAT (built around a Quectel RM502Q-AE modem) mounted on a Raspberry Pi 4, an intermediate option offering more configurability than a COTS device without the fully virtualized paradigm of srsUE.

Across the four bandwidth configurations tested, the COTS handset consistently delivered the highest downlink throughput, peaking at 68.6 Mbps at 20 MHz, with the Sixfab HAT close behind at 63.0 Mbps and srsUE trailing at 20.8 Mbps. Combining multiple UEs simultaneously exposed connection problems: pairing the COTS device with the Sixfab HAT roughly halved each device's individual throughput, a predictable outcome of sharing the radio medium, but adding srsUE to the mix depressed the other two devices' throughput by over 75%, an effect the authors attribute to srsRAN's resource allocation behavior and the high Radio Frequency (RF) gain srsUE's USRP B200 link required. Internet-facing Speedtest measurements (via Ookla) were more consistent for the COTS and Sixfab devices, reaching over 20.1 Mbps. In contrast, srsUE showed an unexplained throughput peak at 10 MHz followed by a drop at higher bandwidths.

Qualitative real-world tests, such as package downloads through apt, video streaming on YouTube, and video calls over Zoom, further differentiated the three UEs: the COTS handset handled every test cleanly; the Sixfab HAT stayed connected throughout but could not render video calls reliably, a limitation of the Raspberry Pi's browser rather than the radio link; and srsUE, while functional, suffered frequent YouTube buffering problems and could not sustain a stable video call. The authors conclude that COTS devices remain the strongest option for raw performance, but that low-cost, low-power alternatives such as the Sixfab HAT are viable for IoT and portable deployment scenarios.

In a similar vein, Vázquez et al. [40] also developed a complete 5G SA testbed, named "MAQ5G". Their approach relies solely on the OpenAirInterface project, deploying both their 5G Core and RAN software, with some modifications to the gNodeB scheduler's code. Their RAN is deployed on USRP X310 and B210 SDRs.

Although similar, the evaluation in this work places greater emphasis on radio performance, measuring throughput across different transmission methods and RAN configurations using two COTS devices fitted with programmable SIM cards. The authors also evaluated the RAN host system's performance under different traffic loads, measuring

CPU usage.

Results showed that their testbed could provide connectivity to the UEs and achieve a maximum downlink throughput of 149 Mbps. However, they also observed that, under high traffic loads, the host system's behavior became unreliable, leading to loss of connection for the UEs, with the only remedy being a full UE reboot.

Mamushiane et al. [23] took a markedly more practical angle, positioning their work as a deployment guideline rather than a performance study. Their testbed paired srsRAN, chosen over OAI for its stronger documentation and community support, with Open5GS, deployed on a workstation (Intel Core i9-9900, 64 GB RAM) connected to a National Instruments NI-2944R USRP over PCIe, with the Core network itself running inside Docker containers on an OpenStack virtual machine. The paper's main contribution is a detailed account of the practical obstacles involved in attaching consumer-grade UEs to a 5G SA network: out of four COTS handsets tested (a Xiaomi Redmi Note 11 Pro, a Huawei P40 Pro, a Samsung Galaxy S21, and a Huawei 5G Router CPE), only the Huawei P40 Pro and a Samsung Galaxy S21 attached successfully, with connection requiring a GPSDO external clock reference for gNodeB/UE synchronization, a 30 kHz SCS for TDD compatibility, and custom-programmed SIM cards (Sysmocom and Open-Cells) carrying authentication parameters matched to the Core's subscriber database. The authors further worked through the mechanics of network slice selection from the UE's perspective, distinguishing the Configured, Subscribed, Requested, and Allowed slice-selection states, and provided a two-part troubleshooting guide at the end, covering RAN and Core errors they encountered along the way.

While not the central focus of their work, the authors also captured a gNodeB packet trace under minimal RF optimization (30 kHz SCS, 100 resource blocks), which showed almost perfect link conditions, with at most one to two packets dropped per second on the downlink and negligible loss on the uplink.

3.2 Comparative Evaluations of Open-Source Implementations

Closer to methodological concerns, Rouili et al. [33] asked a different question altogether: how much does the choice of RAN deployment approach, between real hardware, emulation and pure simulation, actually matter for the realism of the results? Using a common OAI 5G Core, the authors built and benchmarked three RAN scenarios against one another: a full SDR deployment (an OAI gNodeB driving a USRP X310, connected to a COTS Google Pixel 7 Pro over the air, on band n78 with 40 MHz bandwidth); OAI's RFsimulator, which emulates the radio interface between a simulated UE and the gNodeB while still exercising the full protocol stack; and UERANSIM, which removes the PHY layer entirely and simulates a UDP-based radio link.

Measured against the 3GPP-derived theoretical throughput ceiling for their configuration (122 Mbps), the SDR deployment reached 95% of that ceiling, RFsimulator only around 60%, while UERANSIM, unconstrained by any PHY-layer bottleneck, achieved the full 1 Gbps saturated link. This unrealistically high result reflects the absence of any radio-layer resource allocation rather than true RAN performance. Idle round-trip latency measured 15 ms for the SDR deployment and 7 ms for RFsimulator, both influenced by frame alignment, HARQ retransmissions, and channel conditions, compared with under 1 ms for UERANSIM. The gap was just as pronounced under load: under sustained traffic, latency rose to roughly 220 ms and 250 ms respectively for the SDR and RFsimulator scenarios, while UERANSIM again showed negligible change. Resource consumption followed a different, almost inverted pattern: the PHY-layer processing of the SDR deployment consumed up to 94% of a CPU core, and RFsimulator's software-based signal generation peaked even higher at around 140% (spread across more than one core), whereas UERANSIM never exceeded roughly 30%. The authors conclude that RFsimulator and, especially, UERANSIM are useful for functional and protocol-level testing, but systematically misrepresent the throughput, latency, and resource costs a real radio deployment incurs, a finding that reinforces the value of the SDR-based, real hardware approach this dissertation's own testbed relies on.

Shifting focus towards containerization and orchestration, and the complexity of managing heterogeneous software stacks, Horstmann et al. [19] developed open5Gcube, a

modular framework that unifies the deployment and management of multiple open-source mobile network project combinations, rather than committing to a single Core/RAN pairing as the previous works do, and can build 2G, 4G and 5G SA/NSA networks from the same infrastructure.

Their framework wraps each project's services (Open5GS, Free5GC and OAI Core for the Core network, srsRAN and OAI RAN for the base station, and OsmoBTS for legacy 2G) in Docker containers organized into use-case-specific "stacks", configured through shared Docker Compose templates and driven by a script-based orchestrator built specifically to avoid the operational overhead the authors found unjustified in Kubernetes or Docker Swarm for a laboratory-scale deployment. The hardware backbone is a portable 19-inch rack housing two USRP X310 SDRs and space for USRP B210s, synchronized via a Leo Bodnar GPSDO reference clock, the same device family used in this dissertation's project testbed.

The evaluation is the most extensive UE compatibility study among the related works surveyed here, covering a Quectel RM500Q-GL modem and six COTS smartphones: a Samsung Galaxy S25, a OnePlus 13, a Motorola Edge 50 PRO, an iPhone 13 Mini, an Asus ROG Phone 6, and a Google Pixel 9, tested against every combination of Core and RAN implementation. All six devices connected reliably for 2G and basic 4G data; results diverged sharply once VoLTE and 5G SA were tested. Under 4G VoLTE (srsRAN plus Open5GS, with Kamailio providing the IMS), most devices registered successfully, but one handset repeatedly dropped its connection seconds after attaching due to a missing IMS configuration. Under 5G SA, the Samsung Galaxy S25, OnePlus 13, Asus ROG Phone 6 and the Quectel modem connected and sustained a two-minute flood-ping without any connection loss across every Core/RAN combination tested; the Google Pixel 9, by contrast, could not see the cell at all when paired with srsRAN, and connected to the RAN but failed to establish a PDU session when paired with OAI RAN and Free5GC; and the iPhone required manual SUCI concealment configuration on its SIM and would not attach at all to OAI Core, which lacks SUCI-based authentication support. The authors conclude that, while basic connectivity is broadly achievable across the open-source ecosystem, compatibility with advanced features remains handset- and implementation-specific, reinforcing the value of testing against the specific UE models a deployment is meant to serve.

Focusing specifically on the Core network rather than the RAN, Barbosa et al. [8] conducted the most direct head-to-head comparison among the surveyed works, evaluating Open5GS, Free5GC and OAI Core (a fourth platform, SD-Core, was excluded from the practical evaluation as it only supports Kubernetes-based deployment) against a shared RAN: an OAI gNodeB driving an Ettus B210 SDR on band n78, connected to a COTS handset over the air. Each Core implementation ran containerized on the same server, isolating the Core software itself as the only variable between test runs.

For control-plane procedures, Open5GS was consistently the fastest, registering UEs in 0.47 s and establishing PDU sessions in 0.28 s, against 0.53 s/0.31 s for Free5GC and 0.66 s/0.33 s for OAI Core, roughly 13–40% slower depending on the procedure. The result is reversed on the data plane: OAI Core delivered the highest throughput, with 100.1 Mbps downlink and 4.8 Mbps uplink, with the lowest latency (12.7 ms) of all three, around 50% higher throughput than Free5GC, which came last on every data-plane metric (66.5 Mbps downlink, 3.11 Mbps uplink, 14.14 ms latency), with Open5GS in between. Together with the resource-consumption results reported alongside these measurements, where Free5GC drew the least required computing power, the authors conclude that no single open-source Core platform dominates across all axes: the right choice depends on whether a deployment prioritizes signaling responsiveness, data-plane throughput, or resource efficiency, a trade-off directly relevant to platform selection for any new open-source 5G testbed.

3.3 Automation, Scale, and Real-World Deployments

Integrating CI/CD and automation tools, Bonati et al. [9] presented “5G-CT”, a framework focused on automated deployment and Over-The-Air (OTA) testing. Their network consisted of an Open5GS core, an A5G Networks core, and OpenAirInterface RAN, deployed at a Northeastern University lab in Massachusetts.

Their work stands out for the tools they used: OpenShift, a Kubernetes cluster where the Core and RAN components were deployed, and GitOps workflows, the automation tool used to orchestrate network deployment. By using these tools, they automated provisioning of their 5G networks, streamlining setup and ensuring a consistent, reproducible

deployment process.

To demonstrate the framework’s capabilities, they ran a series of tests over roughly nine months. During this period, the framework executed an automated cycle every six hours (four times a day), instantiating the gNodeB and Core, connecting a Sierra Wireless EM9191 5G modem as UE, and exchanging downlink UDP traffic at one of three target data rates (10, 20, and 30 Mbps). Finally, the framework reported results to dedicated data-collector pods running on OpenShift, where they were analyzed and stored before the network was torn down.

Results over the nine-month campaign showed that the 10 and 20 Mbps configurations sustained steady throughput on almost every run, while the 30 Mbps configuration was less stable, showing intermittent throughput dips and the associated packet loss the authors report (visualized only for this worst-case configuration, for clarity). The authors traced this instability to a combination of factors, bugs in the OAI code, gNB-to-SDR communication issues, interference, and Linux kernel scheduling behavior, which was substantially mitigated by an update to the host kernel configuration, midway through the testing campaign, after which even the 30 Mbps runs stabilized. Beyond raw throughput, 5G-CT also compares each new test statistically against a confidence interval built from the full history of prior runs at the same configuration, and each test is only flagged as successful if it falls within that historical envelope, letting the framework detect performance regressions dynamically, rather than relying on a fixed threshold. The framework also kept the gNodeB software itself continuously up to date, checking weekly for new OAI releases and rebuilding its container image whenever one appeared.

While these works focus on OSS implementations using SDRs, Zu et al. [42] bridged the gap between OSS and commercial applications, focusing on the realism and the scale of real-world deployments. Unlike the testbeds discussed elsewhere in this chapter, built from scratch around open-source RAN software, their contribution is a real-world application. The ARA Wireless Living Lab, a real-scale wireless lab spanning Iowa State University’s campus, the City of Ames, and surrounding research farms and rural communities across a 30 km radius, ran for years on Ericsson’s native 5G core (E-5GC). The authors report on transitioning the already deployed, commercial-grade RAN to Open5GS and the resulting performance differences between the two cores.

The RAN itself is composed of four Ericsson gNodeBs, each split into three sectors, with every sector carrying both a mid-band AIR 6419 radio and a millimeter-wave AIR 5322 radio, connecting to more than 30 UE sites equipped with Quectel RG530 modems, scattered across residential areas, university research farms and commercial buildings. The original E-5GC deployment split its control and user planes geographically, where the control plane ran out of an Ericsson lab in California, while the user plane sat locally at the ARA data center in Iowa, with the two connected over a nationwide 100 Gbps fiber backbone. Migrating to Open5GS brought both planes onto a single Dell R750 server (Intel Xeon Gold 6342, 395 GB RAM, 100 Gbps NICs) co-located with the gNodeBs. The authors credit this architectural change, more than the core software change itself, for most of the latency improvement they measured.

Performance-wise, the transition came at a measurable but acceptable cost. Open5GS reduced average registration and service-request latency by 19.1 ms and 63.0 ms, respectively, which the authors credit to the now-local control/user-plane location. In comparison, PDU session establishment latency improved only marginally (1.7 ms), which the authors attribute to inefficiencies in Open5GS's own network functions. On the user plane, Open5GS reached 445.6 Mbps in a short-range farm field test, around 20% below the roughly 557 Mbps the commercial E-5GC achieved under the same conditions (but still several times over the ~100 Mbps eMBB requirement it needs to meet). Across a broader set of field sites, downlink throughput averaged closer to 300 Mbps, with over 90% of measurements exceeding 200 Mbps, demonstrating that an open-source 5G Core can reliably sustain carrier-grade performance alongside a commercial RAN in real-world conditions.

The authors also ran a parallel comparison against UERANSIM, the same RAN simulator benchmarked by Rouili et al. [33] in Section 3.2: swapping the field UE for a simulated one connected to the same Open5GS core roughly tripled measured throughput (978.6 Mbps versus 299.1 Mbps) and cut registration latency by an order of magnitude (14.1 ms versus 142.0 ms), since the simulator bypasses real channel impairments entirely. This independent confirmation of the same simulation/reality gap reinforces the case for the real, SDR-based approach this dissertation's own testbed takes.

Stepping outside the lab entirely and differing on the Core, Hasan et al. [18] reported on Magma not as an experimental testbed but as a production system, describing lessons

learned from real-world deployments that diverge sharply from the campus- and lab-scale setups of the other works surveyed here. Magma’s core design decision, terminating radio-specific protocols at an access gateway placed close to the radio rather than deep in the Core, lets a single control plane and management platform serve heterogeneous, geographically dispersed sites regardless of the underlying radio technology (5G, LTE, Wi-Fi, or others). The authors report deployments spanning more than 800 eNodeBs across 45 US states at the time of writing, alongside smaller-scale examples such as a solar-powered rural LTE site built with Magma for an independent ISP in Peru, and quantify cost savings of around 43% in one deployment compared to a traditional carrier core architecture.

While Magma’s production focus and its emphasis on Wi-Fi/LTE heterogeneity rather than 5G SA set it apart from the rest of this chapter (and, being a production case study rather than a fixed lab configuration, it does not map onto the comparison in Table 3.1), it demonstrates that the open-source approaches surveyed here are not confined to research prototypes and can scale to genuine commercial deployments.

3.4 Overview of Reviewed Testbeds

Table 3.1 summarizes the network implementations described in the preceding sections, covering each work’s primary focus, the Core and RAN software used, the UE types tested against, the hardware each testbed was built on, and the throughput each work reported.

Paper	Primary Focus	Core	RAN	UE	SDR	Throughput
[8]	Core platform comparison	Free5GC, Open5GS, OAI	OAI	COTS	USRP B210	66.5–100.1 Mbps DL
[9] (5G-CT)	CI/CD automation & OTA testing	Open5GS, A5G	OAI	Sierra Wireless EM9191 5G	8x USRP X310/X410	10–30 Mbps (targeted rates)
[19] (open5Gcube)	Multi-stack framework & UE compatibility	Open5GS, Free5GC, OAI	OAI, srsRAN	COTS, Modem	USRP X310	Not reported (flood-ping stability)
[23]	UE integration & deployment guide	Open5GS	srsRAN	COTS, Emulator	NI-2944R	Not reported ($\leq 1-2$ pkt/s loss)
[33]	RAN deployment realism (SDR vs. simulation)	OAI	OAI (SDR, RFsimulator, UERANSIM)	COTS, Emulator, Simulator	USRP X310	116 Mbps (95% of theoretical, SDR)
[38]	UE implementation comparison	Open5GS	srsRAN	COTS, 5G HAT, Emulator	USRP X310/B200	68.6 Mbps DL (COTS)
[40] (MAQ5G)	RAN/radio performance under load	OAI	OAI	COTS	USRP X310/B210	149 Mbps DL (max)
[42] (ARA)	Commercial-RAN core migration at scale	Open5GS	Ericsson	Quectel modem	Ericsson proprietary	~300 Mbps DL (field avg.)

Table 3.1: Overview of network implementations in related work.

IMPLEMENTATION

This chapter documents how the testbed was built: the physical hardware, the network topology connecting it, and the software stack for both the Core and the two Radio Access Networks. Section 4.1 gives an overview of the overall system and how its four machines are laid out and connected. Section 4.2 details the physical hardware assets, software-defined radios, and clock synchronization equipment. Section 4.3 describes the Core Network implementation, Section 4.4 outlines the structure of the Radio Access Network for both generations, and Section 4.5 describes the user equipment used to connect to the network. Section 4.6 lists the logs, packet captures, and test outputs the testbed records, from which every result in this dissertation is drawn, and Section 4.7 brings the chapter together in a single testbed diagram.

Appendix A provides a step-by-step guide to building both networks.

4.1 System Architecture Overview

The testbed is built around four machines: one running the 5G RAN, another running the 4G RAN, a third running the Core network (shared by both generations), and a fourth reserved for a 5G NSA deployment that was ultimately not implemented. Each working RAN runs on its own dedicated machine rather than sharing a single host between radio access technologies, so the 4G and 5G networks can run independently rather than being intermingled on the same machine. This choice simplifies each network's management and follows directly from one of the dissertation's goals: to leave a working network testbed in place, rather than a single disposable experiment, for future projects to build

on and explore. It makes the testbed easier to navigate for the students and researchers who continue this work. The software, configuration files, and attached radio for one generation's RAN are all found on that generation's machine, and the Core network for both generations is found on the Core machine. Someone interested in only one generation can therefore start from a single machine, without separating its components from those of the other.

The machines connect through an Ethernet switch, with static IP addresses assigned to each one, as shown in Figure 4.1.

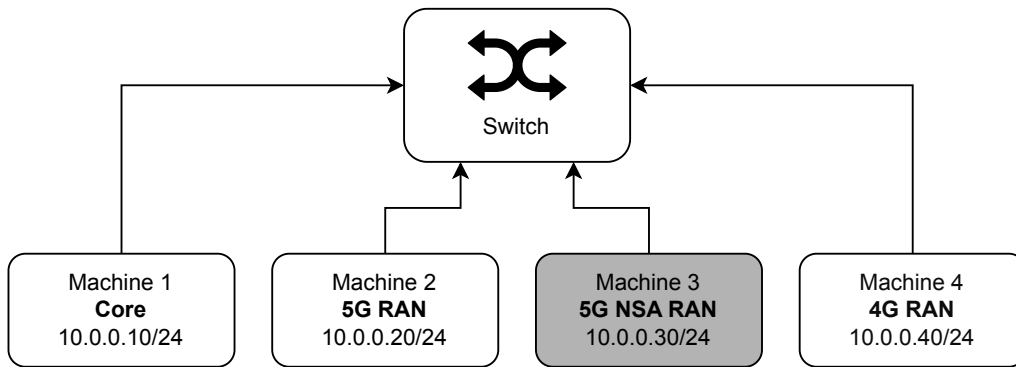


Figure 4.1: Network topology of the four machines, showing each machine's role and static IP address. The machine in gray was reserved for a 5G NSA deployment but was ultimately not used in the actual testbed.

Figure 4.2 shows the laboratory bench with all four machines.

4.2 Hardware Components

This section describes the testbed hardware components. All four machines chosen for this project are Dell OptiPlex Micro 7020 machines, with the specifications listed in Table 4.1.

Component	Specification
CPU	Intel Core i7-14700T
RAM	16 GB DDR5
Storage	512 GB SSD

Table 4.1: Specifications of the Dell OptiPlex Micro 7020 machines, identical across all four testbed machines.

All machines run Ubuntu 22.04 LTS. Table 4.2 lists the software running on each

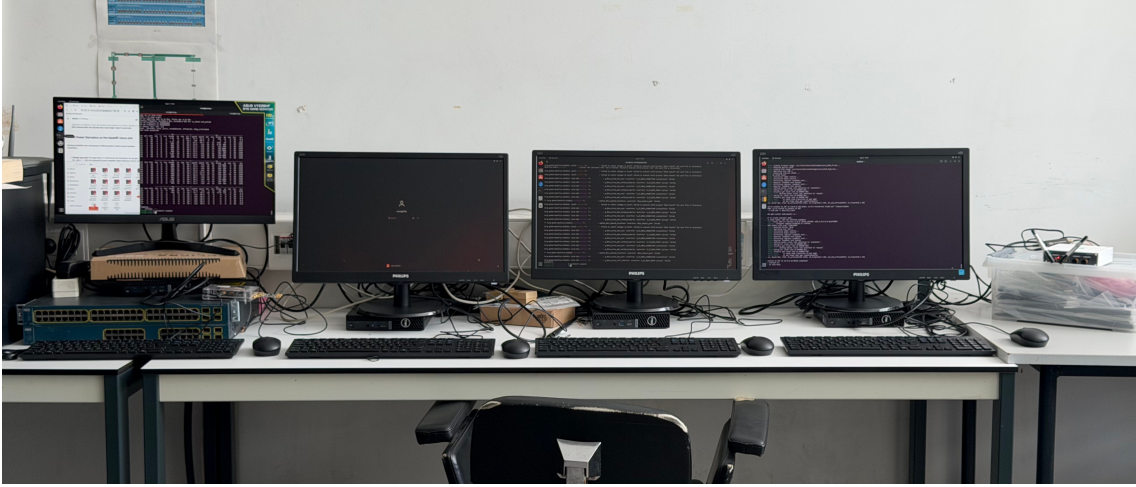


Figure 4.2: The bench, showing all four machines. From left to right: 4G RAN, 5G NSA (unused), Core, and 5G RAN. Connected to the 4G RAN machine is the bladeRF xA4, and connected to the 5G RAN machine is the NI B2901.

machine and the version used throughout the test campaigns so that the testbed can be rebuilt to the same state.

Software	Version	Machine
Ubuntu	22.04 LTS (Jammy Jellyfish)	All
Linux kernel	6.8.0-136-generic	All
Open5GS	2.8.0	Core
MongoDB	8.0	Core
srsRAN Project (OCUDU)	commit 29dc1aede7	5G RAN
UHD	4.10.0.0	5G RAN
srsRAN 4G	commit 6bcbd9e5b	4G RAN
bladeRF host tools (bladeRF-cli)	1.8.0	4G RAN

Table 4.2: Software versions used on each machine throughout the test campaigns.

For the radio, two SDRs were chosen:

- National Instruments B2901 (B210): This is a mid-level SDR that uses USRP Hardware Driver (UHD), the gold standard for SDR development. These boards are widely used in research projects and testbeds, and they are natively compatible with most SDR software.
- Nuand bladeRF xA4: This is a cheaper mid-level SDR that uses its own vendor library, libbladeRF, to communicate with the host machine. It is less common than the USRP B210 in research projects because it is not directly compatible with most

software. Through the use of a wrapper library, it is possible to use this SDR with most software that does not natively support it.

Both of these boards use an Analog Devices AD9361 transceiver, which makes their RF capabilities theoretically similar. The main difference between them is their Field-Programmable Gate Arrays (FPGAs): the B210 uses a Xilinx Spartan-6, while the bladeRF xA4 uses an Intel Altera Cyclone V.

Along with the SDRs, a Leo Bodnar GPSDO was chosen to provide a stable clock source for the NI B2901, which is crucial for the proper operation of the 5G network, given the strict timing requirements of the 5G standard, compounded by the TDD operation. The GPSDO provides a stable 10 MHz reference clock and a 1 PPS signal, which synchronize the SDR and ensure accurate timing for signal transmission and reception.

Connected to the 4G RAN machine is the Nuand bladeRF xA4 (Figure 4.3), which was used for the 4G deployment. Unlike the B2901, this board was powered entirely over its USB connection to the RAN machine, with no separate power supply.

Connected to the 5G RAN machine is an NI B2901 (Figure 4.4), the National Instruments designation for the USRP B210 described above. In addition to its USB connection to the RAN machine, the B2901 was connected to a wall socket through its own power supply, which was required to sustain the board at the gain settings used during testing. Along with the board, the Leo Bodnar GPSDO was connected to provide both a stable clock source and a 1 PPS signal for synchronization. The GPSDO's GPS lock was verified using `1bgpsdo` [17], an open-source command-line utility for configuring and querying Leo Bodnar GPSDO devices.

Both radios used the same antennas. The 4G rig used a BT-200 Low Noise Amplifier (LNA) on the RX port and a BT-100 power amplifier on the TX port, both Nuand components, as seen in Figure 4.5. The 5G rig had no additional RF front-end components.

4.3 Core

The core network is implemented using Open5GS [27], an open-source project that provides both a complete 5G core network (5GC) and a 4G EPC implementation, following the 3GPP Release 17 specifications. This dual support enables 4G/5G comparison on shared



Figure 4.3: The Nuand bladeRF xA4, connected and cabled on the bench. This board had no external synchronization source, although an SMA cable is present in the figure.



Figure 4.4: The B2901 and the Leo Bodnar GPSDO, as connected on the bench. Output 1 of the GPSDO provides a 1 PPS signal to the B2901's PPS IN port, while Output 2 provides a 10 MHz reference clock to the REF IN port.

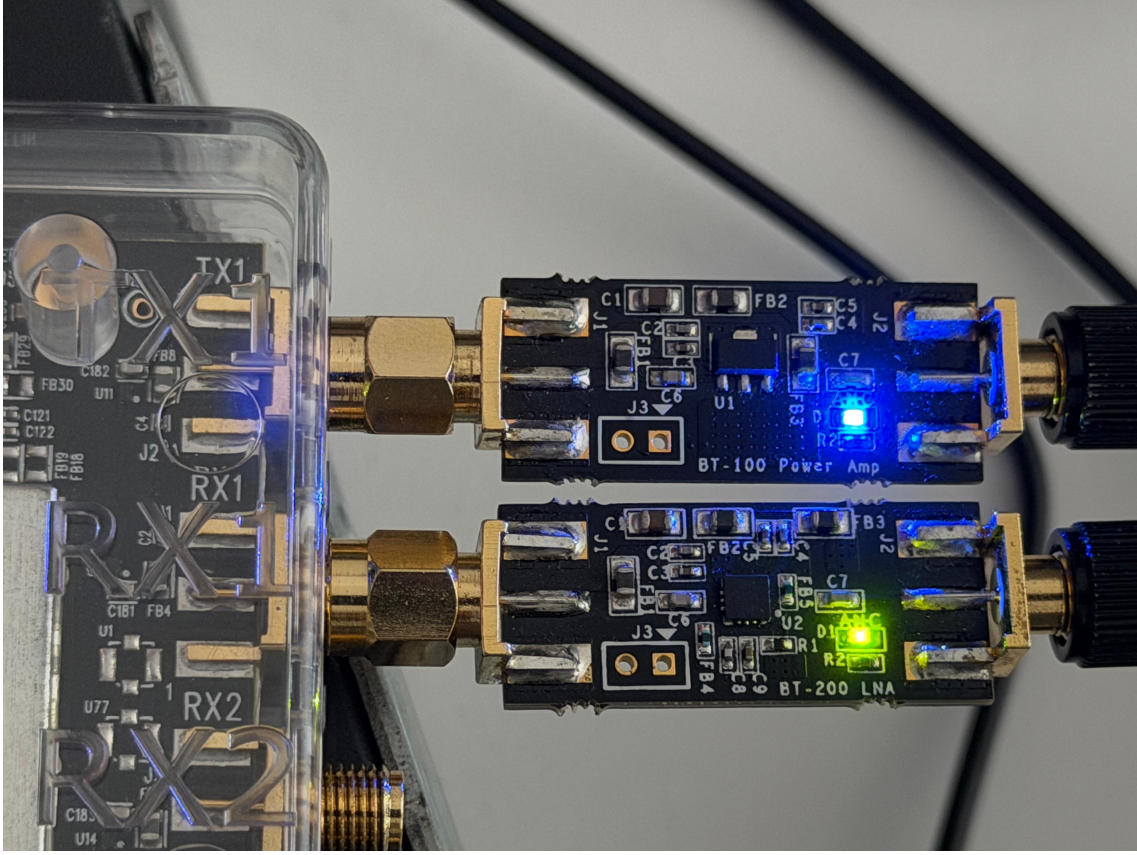


Figure 4.5: The BT-200 LNA and BT-100 power amplifier bias tees, mounted on the bladeRF xA4's RX1 and TX1 ports respectively.

hardware, as the same Open5GS deployment serves as the core for both generations; the 4G baseline and the 5G network under test differ physically only in their RAN. Open5GS is designed to be modular, with each network function implemented as a separate process, each with its own configuration file on the machine. Open5GS stores subscriber data for both generations in a single MongoDB database, which it uses as its backend. The core network is deployed on its own dedicated machine.

The Open5GS [27] core ran at version 2.8.0, with the main configuration values listed in Table 4.3. Harking back to the reusability objective stated in the Introduction, all configuration files for the core were made available in a GitHub repository [31], and a script `deploy_open5gs_configs.sh` to deploy them directly onto the core machine was created and made public, allowing anyone to redeploy the same core configuration without having to reconstruct it by hand. This approach ensures the testbed configuration can be easily replicated in the future, supporting the goal of reusability and facilitating further

research or experimentation based on this work.

Both generations are served by the same identity and addressing: a single Public Land Mobile Network (PLMN), tracking area, and data network, with every UE, whether attached over 4G or 5G, drawing its IP address from the same pool. This is possible because Open5GS's SMF and UPF also act as the control and user plane halves of the EPC's Packet Data Network Gateway (PGW) (PGW-C and PGW-U), so the user plane of both generations terminates at the same function on the core machine. All interfaces facing the RANs are bound to the core machine's address, 10.0.0.10: the NG Application Protocol (NGAP) (N2) and GPRS Tunneling Protocol - User Plane (GTP-U) (N3) endpoints for the 5G RAN, and the MME (S1-MME) and Serving Gateway (SGW) user plane (SGW-U) endpoints for the 4G RAN.

Parameter	Value
PLMN (MCC/MNC)	001/01
Tracking Area Code	7
Network slice (S-NSSAI)	SST 1
Data network (DNN/APN)	internet
UE IPv4 pool	10.45.0.0/16 (gateway 10.45.0.1)
MTU	1400 bytes
DNS servers	1.1.1.1, 1.0.0.1
RAN-facing address (N2, N3, S1-MME, S1-U)	10.0.0.10

Table 4.3: Main configuration values of the Open5GS core, shared by the 4G and 5G networks.

4.4 RAN

The RAN is implemented using srsRAN [25] (renamed OCUDU partway through this project; see Section 2.5.2.1), an open-source software suite that provides a complete 5G NR RAN, following the 3GPP Release 17 specifications. Written in C++, it is modular by design, allowing developers to customize the CU and DU components of the gNodeB separately. Each generation's RAN, deployed on its dedicated machine, connects to its SDR via USB 3.0.

4.4.1 5G RAN

On the software side, the 5G srsRAN RAN [25] ran at commit 29dc1aede7. Table 4.4 lists its configuration side by side with the 4G RAN's.

The parameter `rx_gain` and the TDD structure were changed between testing rounds and are discussed further in Chapter 5.

4.4.2 4G RAN

The 4G srsRAN RAN [37] ran at commit 6bcbd9e5b, with no configuration changes across testing rounds. Table 4.4 compares its configuration with that of the 5G RAN. Both cells broadcast the PLMN and tracking area configured in the core (Table 4.3). Almost everything else differs between the two, from the band and duplexing to the radio and its front-end. These differences bound every comparison drawn between the two generations in Chapter 6.

Parameter	4G RAN	5G RAN
Band	7	n78
Duplex mode	FDD	TDD
Channel number	EARFCN 3350	NR-ARFCN 627340
Downlink center frequency	2680.0 MHz	3410.1 MHz
Uplink center frequency	2560.0 MHz	3410.1 MHz
Channel bandwidth	10 MHz	20 MHz
Subcarrier spacing	15 kHz	30 kHz
Resource blocks	50	51
MIMO layers	1 (SISO)	1 (SISO)
TDD pattern	–	3:1 (Round 1), 2:2 (Rounds 2–3)
PCI	1	1
SDR	Nuand bladeRF xA4	NI B2901 (USRP B210)
RF front-end	BT-200 LNA (RX), BT-100 PA (TX)	None
Clock reference	Internal	GPSDO (10 MHz + 1 PPS)
<code>tx_gain</code>	66	89.75
<code>rx_gain</code>	60	70 (Rounds 1–2), 60 (Round 3)

Table 4.4: Configuration of the 4G and 5G RANs. The 4G configuration did not change across test rounds; the 5G rows that did are detailed in Chapter 5.

4.5 UE

The proposed UE for the testbed was a Motorola Moto G56 5G, a mid-range smartphone chosen for its 5G and eSIM support. This did not match what was actually used: the Moto G56 struggled to connect to the network during testing for a reason that was never identified. A Samsung Galaxy Z Flip5 (personal device) was brought in instead and became the primary UE for the actual test campaigns. Both devices connect to the network using an eSIM profile, provisioned through a remote SIM provisioning service. Table 4.5 compares the two candidate devices.

Device	Chipset	RAM / Storage
Motorola Moto G56 5G [14]	MediaTek Dimensity 7060	8 GB / 512 GB
Samsung Galaxy Z Flip5 [15]	Qualcomm Snapdragon 8 Gen 2	12 GB / 512 GB

Table 4.5: Comparison of the two candidate UEs. Both support NR band n78, the band used by this testbed.

The Galaxy Z Flip5 supports both LTE band 7 and NR band n78, so the same device was used against both networks throughout every test round. The UE is therefore not a variable in the comparison between generations. The eSIM profile it used was created and installed as described in Appendix B, and the same procedure applies to any eSIM-capable device a future project wishes to connect.

4.6 Metrics Collection

Every result presented in Chapter 6 is drawn from records the testbed itself produces while running, rather than from external measurement equipment. Table 4.6 lists these sources.

The 5G RAN is the more heavily measured of the two, because it was the network under investigation. In Round 1, its metrics were available only from the console, and its log also carried hex dumps of every message. From Round 2 onward, the gNodeB was configured to print its metrics into `gnb.log` with timestamps, which allows them to be aligned with the rest of the log, and hex dumps were turned off. In Round 1, the hex dumps accounted for 57 to 77% of the lines in each log, or 41 to 60% of its size, and the gNodeB wrote an average of 1.7 MB/s during testing, about 6.1 GB per hour, with peaks of up to 12.9 MB/s during saturated downlink tests. From Round 2 onward, without hex

dumps but with the timestamped metrics and heavier traffic, the average fell to between 1.1 and 1.3 MB/s, about 4 to 4.7 GB per hour, and the peaks to 4.4 MB/s. Anyone running the testbed continuously should plan for this, or lower the log levels.

Recorded on	Source	Content
5G RAN	gnb.log	All layers at <code>info</code> level; per-second metrics from Round 2
5G RAN	MAC and NGAP captures	.pcap files readable in Wireshark
5G RAN	Console metrics	Per-second link metrics printed by the gNodeB
4G RAN	enb.log	All layers at <code>info</code> level
4G RAN	Console metrics	Per-second link metrics printed by the Evolved Node B (eNodeB)
Core	Open5GS logs	One log per network function, rotated daily
Core	iperf3 server output	Per-interval throughput of every test
UE	CellularLab exports	One JSON file per test with the client-side iperf3 results, from Round 2

Table 4.6: Data sources recorded by the testbed during the test campaigns.

The scripts used to parse these records and produce the figures in Chapter 6 are kept in the same repository as the configuration files [31], so that the analysis can be re-run on new data.

4.7 Summary

Figure 4.6 maps the elements described in this chapter together into a single view of the testbed as it was used in the test campaigns, and how it is connected. Both RAN machines reach the core over Ethernet, so the 4G and 5G networks share everything behind the RAN and differ in what lies in front of it: band, duplexing, bandwidth, SDR, and RF front-end, as listed in Table 4.4.

This setup is the starting point for the evaluation that follows. Chapter 5 describes how it was put to the test: where the UE was placed relative to the radios, the traffic used to load each network, and the three test rounds across which the 5G RAN configuration was adjusted, namely its TDD structure and `rx_gain`, while the 4G network was left untouched as the baseline.

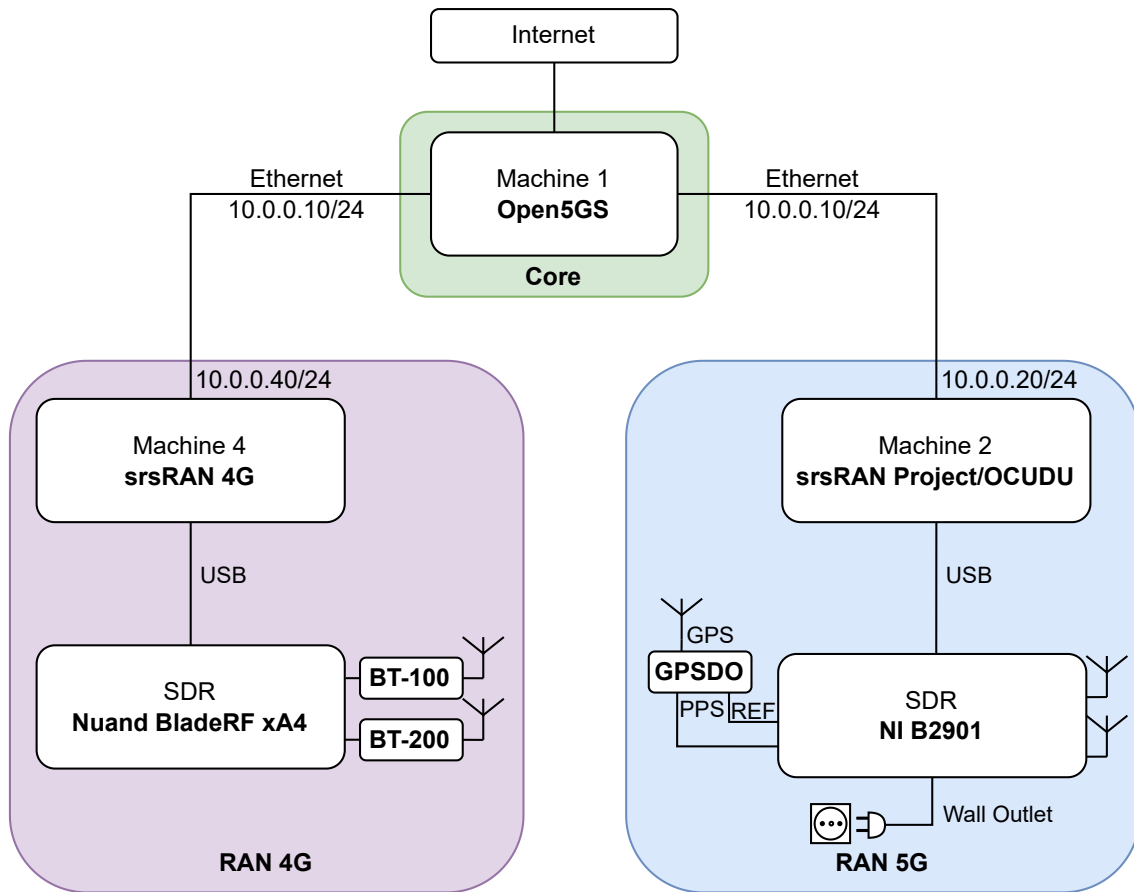


Figure 4.6: The testbed as deployed: the Open5GS core on Machine 1, the 5G RAN on Machine 2 with the NI B2901 and the GPSDO, and the 4G RAN on Machine 4 with the bladeRF xA4 and its BT-100/BT-200 front-end. The unused 5G NSA machine, the Ethernet switch and the UE are not shown.

METHODOLOGY

This chapter describes the methodology used to evaluate the testbed described in Chapter 4. Section 5.1 describes the three test locations, their distances from the radio, and the obstructions on each path. Section 5.2 describes the traffic conditions used at each location, and the TDD structures the 5G RAN was configured with. Section 5.3 describes the three test rounds and the configuration changes made between them. The results of these tests will follow in Chapter 6.

5.1 Experimental Testbed

The network's performance was evaluated at three fixed UE positions in the same room as the RAN, chosen to combine increasing distance from the radio with different obstruction conditions at each point. Figure 5.1 shows the top plan view of the room, and Figure 5.2 shows the elevation, giving the height of the radio and of the UE at each location above the floor.

Material and thickness are recorded for each obstruction, as they, and not distance alone, also account for the difference between the three locations: signal attenuation through a wall depends strongly on its material and thickness, particularly at the 3.4 GHz center frequency used by the 5G RAN, so the three locations should be read as distance-plus-obstruction conditions rather than a simple distance variation. Figure 5.3 shows the tape-measure readings taken for both obstructions.

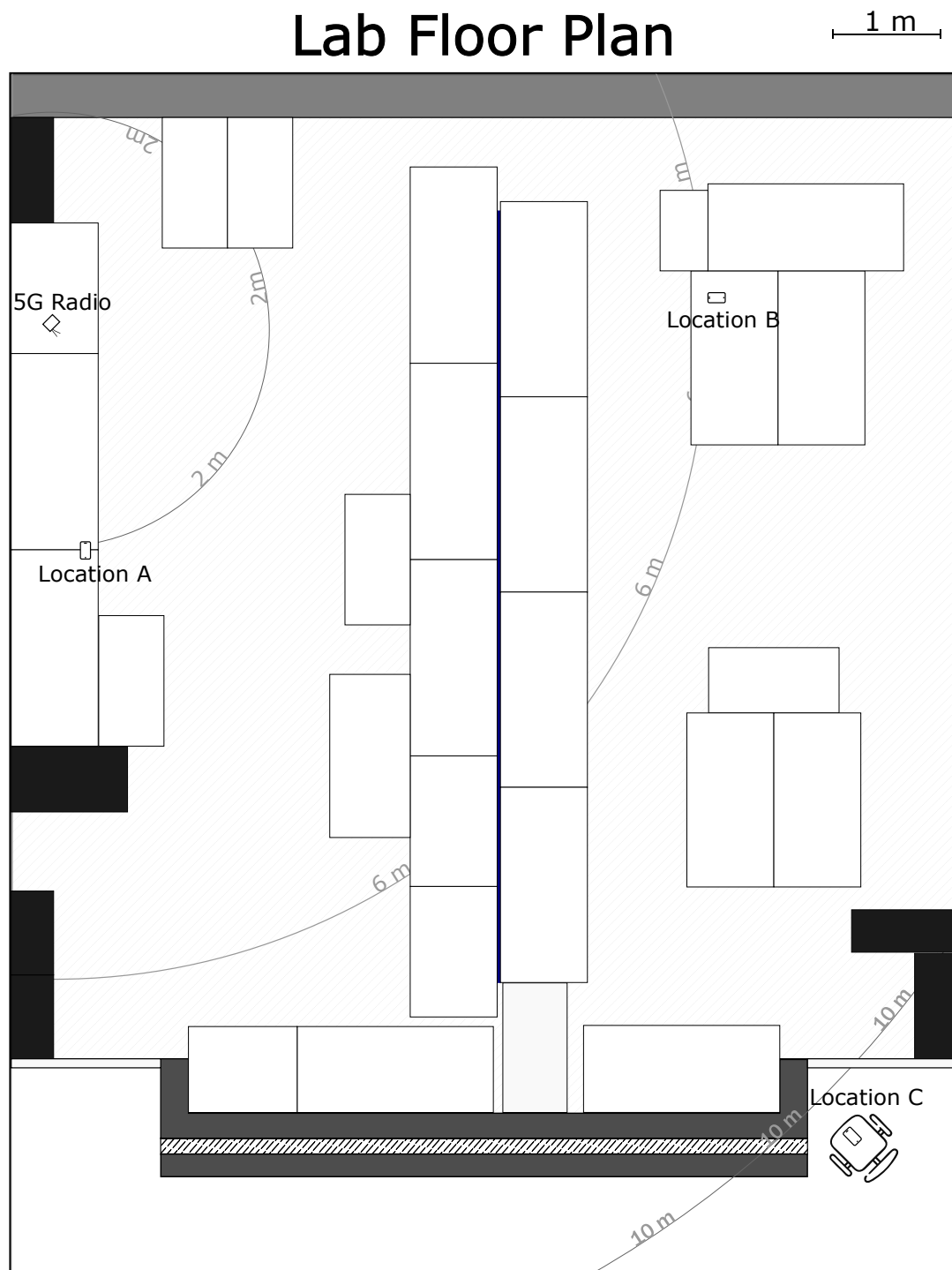


Figure 5.1: Plan view of the room, showing the radio position and the three UE test locations, drawn to scale.

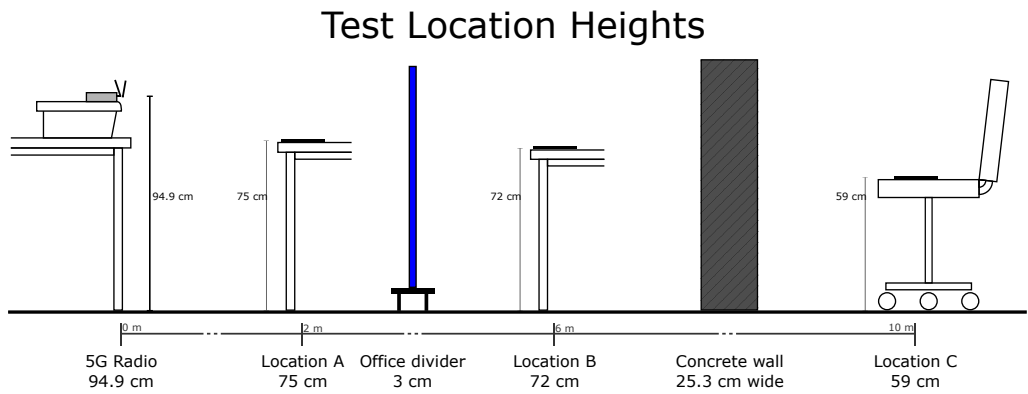


Figure 5.2: Elevation view showing the mounting height of the radio and the UE height at each test location, along with the obstruction on the path to locations B and C.



Figure 5.3: Tape-measured thickness readings for the two obstructions on the direct path: the concrete wall at Location C (25.3 cm) and the office divider at Location B (3 cm).

Besides obstructions, the objects present in the room may account for signal attenuation as well. In Figure 5.1, the objects shown in dark gray are metal furniture and equipment present in the room. None of these lie directly on the signal path between the RAN and any of the three test locations. However, their metal construction means they may still have contributed to reflections and multipath propagation, and should be kept in mind when interpreting the results. Other objects present in the room, such as the tables and doors shown on the diagram, present minimal attenuation and are not expected to have a significant effect on the results. A summary of all measurements and signal obstruction conditions for each of the three test locations is presented in Table 5.1.

Location	Distance	Height	Obstruction	Material	Thickness
A	2 m	75 cm	Clear line of sight	—	—
B	6 m	72 cm	Office divider	Plastic and felt	3 cm
C	10 m	59 cm	Concrete wall	Concrete	25.3 cm

Table 5.1: The three test locations: distance from the radio, UE height above the floor, the obstruction material and thickness on the direct path at each point. All distances, thicknesses, and heights were measured directly with a tape measure.

5.2 Experimental Methodology

At each location, tests were run for each generation of network, for both TCP and UDP, and for both uplink and downlink, giving four traffic conditions per location, per generation. For the 4G network, each of these conditions was tested once ($n = 1$); for the 5G network, each was repeated three times ($n = 3$). This difference was not a deliberate methodological choice about sample size per generation: the 4G network was only tested during the first round of testing, in which every combination, both in 4G and 5G, was only tested once. Subsequent rounds of testing focused on the 5G network, following configuration corrections described later in this chapter, and used three repetitions per condition from that point onward. The 4G tests were meant to be repeated under the same procedure, which would have made the results of the two generations more directly comparable, but this could not be carried out, as the Nuand bladeRF used by the 4G RAN was no longer available at the time.

Traffic was generated with `iperf3`, with the server running on the core machine and the client running on the UE through the CellularLab app [6], which provides an `iperf3` client for Android devices. Each test consisted of a single 20-second `iperf3` run, over either TCP or UDP, in either the uplink or the downlink direction. To saturate the link, the client opened eight parallel streams. TCP tests used the default `iperf3` parameters, while UDP tests set a target rate of 100 Mbps per stream (`-b 100M`), for 800 Mbps in total. This is well above the capacity of either network, so the measured UDP throughput reflects what the link can carry at its maximum, rather than the rate that was offered. The results of each run were recorded both on the UE and on the server.

Figure 5.4 shows the three test locations as they were used for testing, with the UE in place.

Initially, the 5G network was configured with a 3:1 TDD structure. As specified by TS 38.213 [2] and described in Section 2.3.3, the TDD structure is defined by the TDD-UL-DL-ConfigurationCommon parameter, which is the `tdd_ul_dl_cfg` block of the `srsRAN` gNodeB configuration file, shown in Listing 5.1.



Figure 5.4: The three test locations, with the UE in place: Location A, 2 m, clear line of sight; Location B, 6 m, behind the office divider; Location C, 10 m, behind the concrete wall.

Listing 5.1: `gnb.yaml` TDD configuration for the 3:1 pattern used in Round 1.

```
tdd_ul_dl_cfg:
  dl_ul_tx_period: 5
  nof_dl_slots: 3
  nof_dl_symbols: 6
  nof_ul_slots: 1
  nof_ul_symbols: 2
```

This configuration dictates that for every 5-slot period, 3 slots are allocated for downlink and 1 slot is allocated for uplink, with the remaining flexible slot composed as follows: 6 symbols for downlink, 2 symbols for uplink, and the remainder (6 symbols) are flexible. In total, given that a slot has 14 symbols, the total number of symbols per period is 70, with 48 symbols (3×14 symbols + 6 symbols) allocated for downlink and 16 symbols (1×14 symbols + 2 symbols) allocated for uplink. That gives 68.6% of airtime for downlink and 22.9% for uplink per period of transmission, with the remaining airtime being the guard period.

Figure 5.5 shows the slot and symbol structure of the 5-slot period of the 3:1 TDD pattern, which was used in Round 1.

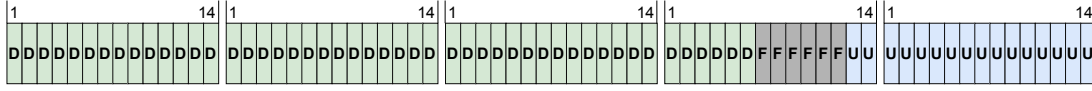


Figure 5.5: Slot and symbol structure of the 3:1 TDD pattern used in Round 1.

This structure was later found to impose a hard structural cap on uplink airtime, independent of signal quality. It was rebalanced to a symmetrical 2:2 structure from Round 2 onward, which meant that each transmission period was composed of two full downlink slots and two full uplink slots, along with a symmetrical flexible slot. Listing 5.2 shows the corresponding `tdd_ul_dl_cfg` block.

Listing 5.2: `gnb.yaml` TDD configuration for the rebalanced 2:2 pattern used from Round 2 onward.

```
tdd_ul_dl_cfg:
  dl_ul_tx_period: 5
  nof_dl_slots: 2
  nof_dl_symbols: 5
  nof_ul_slots: 2
  nof_ul_symbols: 5
```

With this configuration, out of the 70 total symbols per period, 33 symbols (2×14 symbols + 5 symbols) are equally assigned to both downlink and uplink, giving 47.1% of airtime for each direction, with the remaining airtime being the guard period.

Figure 5.6 shows the resulting slot and symbol structure.

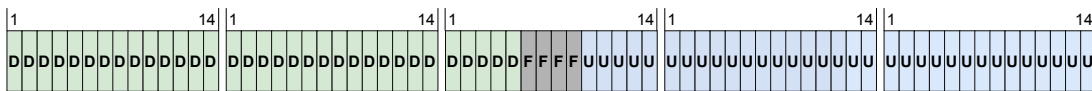


Figure 5.6: Slot and symbol structure of the rebalanced 5-slot period of the 2:2 TDD pattern used from Round 2 onward.

All together, the rebalance is expected to roughly double the uplink airtime available to the scheduler ($22.9\% \rightarrow 47.1\% = \times 2.06$) while reducing the downlink share to little

more than two-thirds of what it was under the 3:1 pattern ($68.6\% \rightarrow 47.1\% = \times 0.69$), trading some downlink capacity for a substantial gain in uplink capacity. Whether this expected trade-off actually materialized in the measured throughput, and how closely the scheduler's grant rates tracked it, is examined in Section 6.4 of Chapter 6. Table 5.2 summarizes the slot, symbol, and airtime allocation for both patterns side by side.

Parameter	3:1 (Round 1)	2:2 (Round 2 onward)
DL slots	3	2
UL slots	1	2
DL symbols	48	33
UL symbols	16	33
DL airtime	68.6%	47.1%
UL airtime	22.9%	47.1%
Guard	8.6%	5.7%

Table 5.2: Slot and symbol allocation per 5-slot, 70-symbol transmission period, for the two TDD patterns used across testing.

5.3 Experiments

5.3.1 Test Round 1

This test round was the first to be run, on the 25th and 28th of July 2026. It comprised 24 tests, 12 for each generation, differing in location, protocol, and direction, each with a single repetition. The 5G network was run with the 3:1 TDD structure, a remnant of the original srsRAN configuration used as a basis for the network configuration, which was later found to cap the maximum uplink airtime structurally and was rebalanced to a symmetrical 2:2 structure for Round 2. Running a single test for each configuration was also found to produce insufficient analytical data, thus not capturing the real behavior of the 5G network, so the following rounds were run with three repetitions per configuration. Results from this round are presented in Chapter 6.

5.3.2 Test Round 2

This test round was run on the 2nd of August. The 5G network was reconfigured with the balanced 2:2 TDD structure devised after Round 1, and testing proceeded with three repetitions per configuration at each location; the 4G network was not re-tested. The TDD

rebalance alone resolved the uplink problem from Round 1, with 5G uplink throughput matching the 4G baseline at 2 m and 10 m, though the downlink regressed as a trade-off for the additional uplink airtime. Location B, at 6 m, produced unusable data this round, with the UE re-attaching to the network 18 times during the session, and was excluded from analysis.

A likely, though unconfirmed, explanation was only identified afterward: a full metal stand (Figure 5.7), belonging to an unrelated thesis project sharing the lab, was standing directly in the signal path between the UE and RAN at this location and was unnoticed at the time. A metal obstruction of this size may have been enough to block the link given the radio's otherwise limited transmit power. Keeping `rx_gain` at 70 for this round was later found to overload the receiver at 2 m, and measures were taken to address this issue on Round 3. Results from this round are also presented in Chapter 6.



Figure 5.7: The metal stand at Location B.

5.3.3 Test Round 3

Before this round, the metal stand identified as a likely obstruction at Location B was moved out of the signal path, and a gain sweep was run at Location A to determine the optimal `rx_gain` setting for the 5G network, resulting in a locked-in value of 60. This test round was run on the 3rd of August, with the 5G network reconfigured to `rx_gain`: 60, keeping the 2:2 TDD structure from Round 2, and with three repetitions per configuration; once again, the 4G network was not re-tested. This setting resolved the receiver overload found in Round 2, roughly doubling 5G uplink throughput over the 4G baseline at 2 m, but severely limited uplink performance at 10 m, where throughput dropped to roughly a sixth of what `rx_gain` 70 had achieved at the same location in Round 2. This showed that

a fixed, manually set receiver gain cannot serve every point in the cell. Results from this round are presented in Chapter 6.

Table 5.3 summarizes the 5G configuration and data validity across the three rounds for reference.

Round	Date	TDD	rx_gain	Reps	4G tested	Total reps	Locations valid
1	25–28 Jul	3:1	70	1	Yes	24	A, B, C
2	2 Aug	2:2	70	3	No	36	A, C
3	3 Aug	2:2	60	3	No	36	A, B, C

Table 5.3: Summary of the 5G network configuration and data validity across the three test rounds. Round 1 covers both generations; Rounds 2 and 3 cover the 5G network only, as the 4G network was not re-tested.

Together, these three rounds make up the full test campaign analyzed in Chapter 6.

RESULTS

This chapter presents the results of the performance evaluation of the 4G and 5G networks implemented in the testbed described in Chapter 4.

6.1 General comparison between 4G and 5G performance

This section presents a comparison of the 4G baseline against the 5G network across the three test rounds described in Section 5.3, focusing on the evolution of performance metrics. Because the 5G configuration changed between rounds, namely the TDD structure and the `rx_gain` setting, these results are presented as a progression rather than a single snapshot, tracking how each change affected throughput at each location and in each direction. As will become clear over the following sections, no single round can be presented as the definitive 5G result: the configuration that performs best at short range is not the same as the one that performs best at long range, a trade-off examined in more detail later in this chapter.

In Figures 6.1 through 6.4, the mean throughput results for TCP and UDP in both downlink and uplink directions are presented for each test location, with black lines representing the range between Min/Max throughput values between each repetition for that combination. These figures mainly illustrate the performance differences between 4G and 5G across the three rounds of testing, with the results showing that the 5G network can outperform the 4G baseline in certain configurations and locations, but also underperform in others.

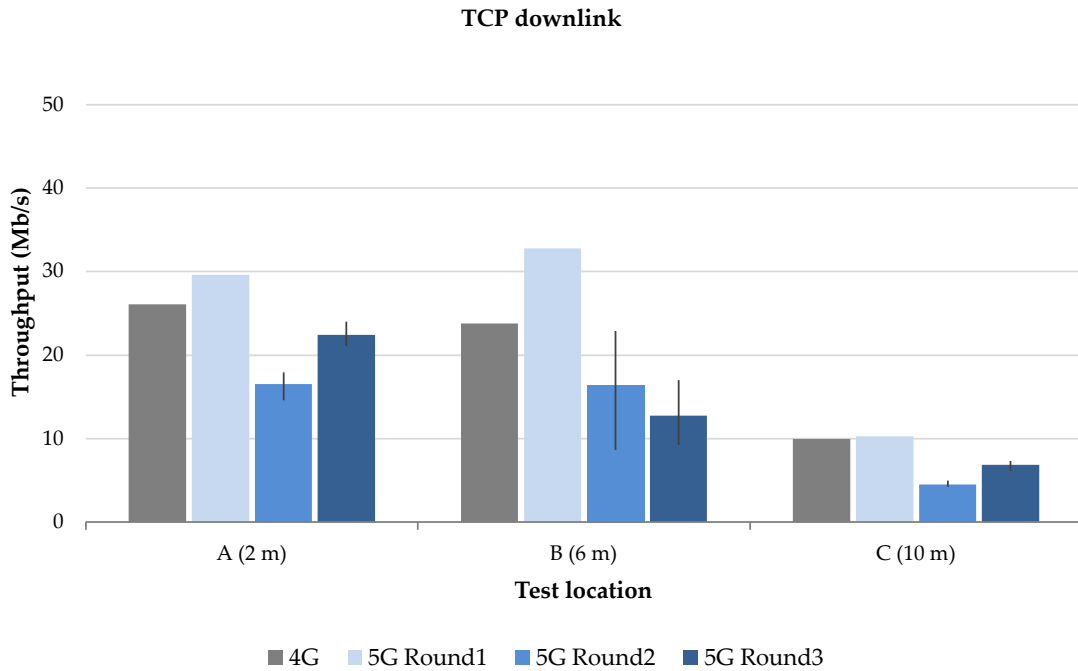


Figure 6.1: TCP downlink throughput at each test location, throughout the three rounds of testing. The black lines represent the range between Min/Max throughput values between each repetition for that combination.

Table 6.1 lists the mean throughput of every condition measured, and the same values are plotted in Figures 6.1 to 6.4.

In Round 1 the comparison divides cleanly by direction. Over UDP, the 5G downlink was ahead of the 4G baseline at every location, by roughly a factor of two, close to what the twofold difference in channel bandwidth alone would predict; over TCP, the two generations were nearly level. The uplink went the other way, and by a far wider margin, falling to 0.6 Mbps over TCP and 0.5 Mbps over UDP at Location C, 21 and 28 times below the 4G baseline. Part of that deficit was structural, as Section 6.4 shows.

Round 2 reversed the uplink result. With the rebalanced 2:2 TDD pattern, the 5G uplink matched the 4G baseline at both of the locations measured, and the downlink lost between 22 and 56% of its Round 1 throughput in exchange.

Round 3 separated the two ends of the room. Lowering `rx_gain` to 60 roughly doubled the Location A uplink again, to about twice the 4G baseline at that location. At Location C, the same change worked in reverse, leaving every measurement there, in both directions

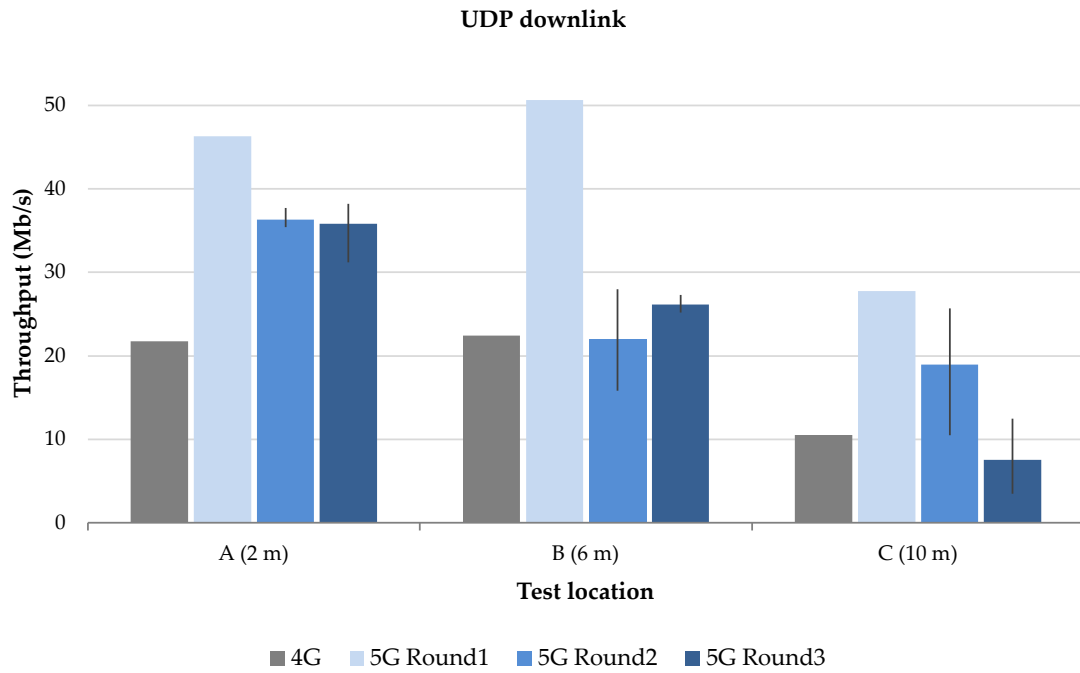


Figure 6.2: UDP downlink throughput at each test location, throughout the three rounds of testing. The black lines represent the range between Min/Max throughput values between each repetition for that combination.

Location	Protocol	Direction	4G	Round 1 (3:1, gain 70)	Round 2 (2:2, gain 70)	Round 3 (2:2, gain 60)
A (2 m)	TCP	DL	26.0	29.6	16.5	22.4
	TCP	UL	13.1	5.1	12.6	25.2
	UDP	DL	21.8	46.3	36.3	35.8
	UDP	UL	17.2	4.6	15.3	31.8
B (6 m)	TCP	DL	23.8	32.8	—	12.8
	TCP	UL	16.0	4.4	—	4.0
	UDP	DL	22.4	50.6	—	26.2
	UDP	UL	15.8	0.8	—	10.3
C (10 m)	TCP	DL	10.0	10.3	4.5	6.9
	TCP	UL	12.6	0.6	11.6	2.8
	UDP	DL	10.5	27.7	19.0	7.5
	UDP	UL	14.1	0.5	14.2	1.7

Table 6.1: Mean throughput, in Mbps, of every combination of location, protocol and direction, for the 4G baseline and the three 5G rounds. The 4G and Round 1 figures are single measurements ($n = 1$); Rounds 2 and 3 are means over three repetitions ($n = 3$).

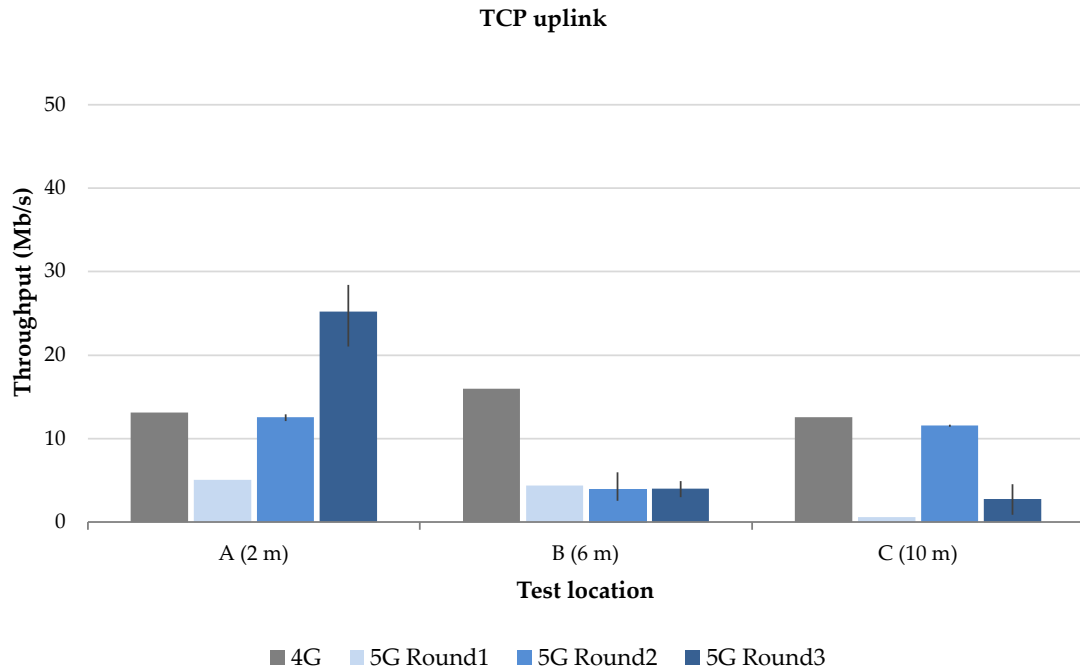


Figure 6.3: TCP uplink throughput at each test location, throughout the three rounds of testing. The black lines represent the range between Min/Max throughput values between each repetition for that combination.

and over both protocols, below the 4G baseline. This is the trade-off referred to above, and Sections 6.4 and 6.5 examine its two causes in turn.

6.2 Unloaded latency

The throughput results describe how much data the network can carry, but not how quickly it responds when it is idle. To measure this, a ping command was used to send 30 ICMP echo requests at one-second intervals from the core machine to the UE, at each of the three test locations, with no other traffic on the network. These measurements were taken during Round 3 only, with the 2:2 TDD pattern and rx_gain 60, and were not repeated for the 4G network, so the values in this section describe the 5G testbed latency only. Table 6.2 presents the results.

Two results stand out. The first is that no packets were lost at any location, including through the concrete wall at Location C. The second is that the minimum round-trip time is around 15.7 ms at all three locations. This is expected, as ten meters of air adds only 33 ns in

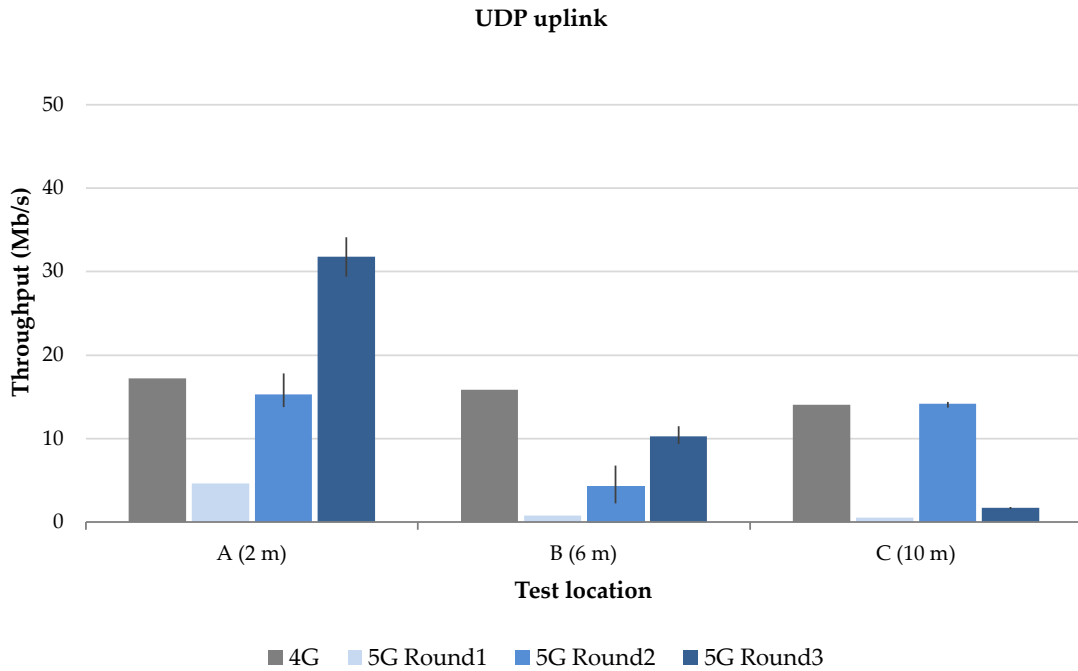


Figure 6.4: UDP uplink throughput at each test location, throughout the three rounds of testing. The black lines represent the range between Min/Max throughput values between each repetition for that combination.

Location	Loss	Min	Mean	Median	p95	Max
A (2 m, line of sight)	0%	16.0 ms	25.0 ms	24.6 ms	33.7 ms	34.9 ms
B (6 m, divider)	0%	15.9 ms	28.1 ms	26.7 ms	41.6 ms	48.5 ms
C (10 m, concrete wall)	0%	15.7 ms	26.2 ms	25.1 ms	35.3 ms	48.4 ms

Table 6.2: Round 3 round-trip time from the core machine to the UE, 30 ICMP echoes per location, with no other traffic on the network.

each direction, which is far below the measured floor, so propagation delay is irrelevant at these distances. The floor instead measures the fixed processing and scheduling overhead the packet passes through: the UE modem, the gNodeB PHY and MAC layers, the GTP-U tunnel through the UPF, and the same path back. This is the figure that best characterizes the latency of the testbed as deployed.

The means, at 25 to 28 ms, sit well above that floor, and the spread between the minimum and the maximum across the three locations is wide. Plotting the round-trip time of each echo in sequence, shown in Figure 6.5, reveals a regular sawtooth in all locations, with the round-trip time falling steadily by 1 to 2 ms from one echo to the next, then jumping back up, repeatedly throughout the ping test.

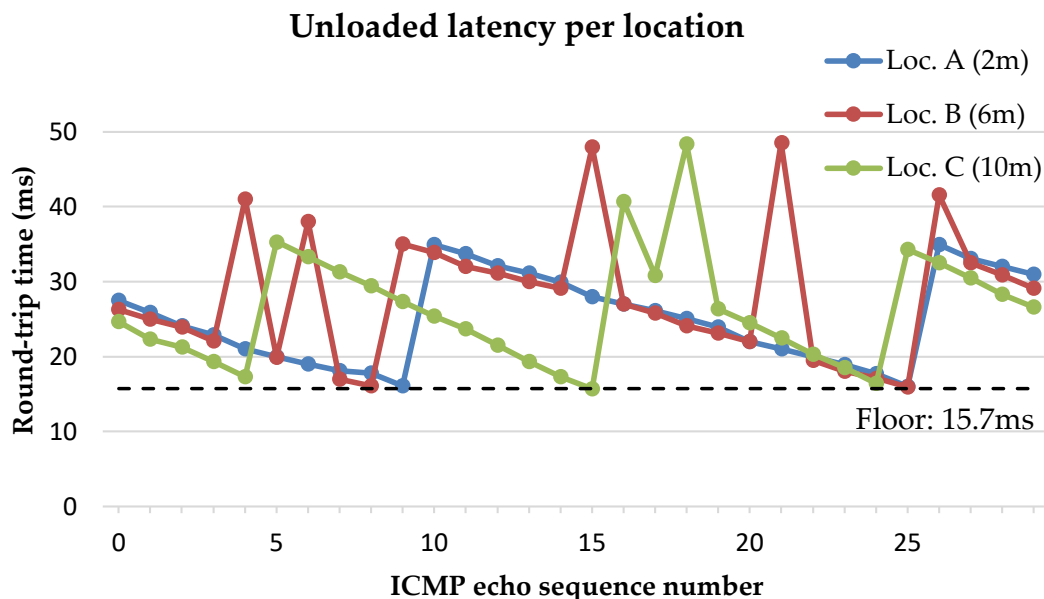


Figure 6.5: Round-trip time of each of the 30 ICMP echoes at the three test locations, with the 15.7 ms floor marked.

The rate of descent is similar at Locations A and B, around 1.2 ms per echo, and steeper at Location C, around 2.0 ms per echo, a difference that lies in the ping tool rather than in the network. The locations also differ in how often the ramp restarts: Location A completes two long ramps over the 30 echoes, while Locations B and C restart more frequently and peak higher, reaching 48 ms against Location A's 35 ms. Those higher peaks exceed the 20 ms period, so at the weaker locations an echo occasionally waits more than one cycle, which is consistent with the weaker uplink signal at those locations under Round 3's `rx_gain` of 60, examined later in this chapter.

The sawtooth is caused by a drift between the ping interval and the uplink scheduling cycle. Although ping was set to send one echo every 1000 ms, the total interval duration reported in ping's own summary, divided by the 29 intervals between the 30 echoes, gives 1001.24 ms at Location A, 1001.28 ms at Location B, and 1001.97 ms at Location C, most likely because the software timer on the core machine overshoots each interval by a small amount.

The uplink scheduling cycle comes from the way a UE gains access to the uplink. In the downlink, the gNodeB decides when to transmit, but a UE with data to send and no uplink

grant must first ask for one by sending a Scheduling Request (SR), and it can only do so at periodic opportunities configured by the gNodeB. Only once the request is received does the gNodeB schedule the uplink transmission itself, on the Physical Uplink Shared Channel (PUSCH). In srsRAN, the period of these opportunities is set by the `sr_period_ms` parameter of the `pucch` block, which defaults to 20 ms and was not changed in this testbed's `gnb.yaml`, so the cell offered a scheduling request opportunity every 20 ms throughout the campaign. As this cycle is driven by the gNodeB's radio, whose clock is locked to the GPSDO, it repeats exactly every 20 ms, and each echo reply generated by the UE has to wait for the next opportunity before it can be sent. Because the ping interval is not an exact multiple of 20 ms, each echo arrives later in the cycle than the one before it, by the amount the interval exceeds the one-second mark. The time each reply waits for the next uplink opportunity therefore shrinks by that amount from one echo to the next, until an echo arrives just after an opportunity has passed and has to wait almost a full cycle, producing the jump. Table 6.3 shows that the slope of the ramps in Figure 6.5 matches the excess of the ping interval over one second at every location, including the steeper ramp at Location C.

Location	Ping interval	Excess over 1000 ms	Fitted ramp
A (2 m)	1001.24 ms	1.24 ms	1.23 ms/echo
B (6 m)	1001.28 ms	1.28 ms	1.17 ms/echo
C (10 m)	1001.97 ms	1.97 ms	2.03 ms/echo

Table 6.3: Actual ping interval at each location, from the total duration in `ping`'s summary divided by 29 intervals, against the slope of the round-trip time ramp fitted from the 30 echoes.

This shows that, despite `ping`'s time drift, the network maintains the 20 ms scheduling cycle with precision. With a perfectly timed one-second interval, every echo would have arrived at the same point of the cycle, and the round-trip time would have been constant. By sweeping the echoes across the whole cycle, the drift shows that a packet the UE has to send at an arbitrary moment waits anywhere between 0 and 20 ms, about 10 ms on average, for its next scheduling request opportunity. This is why the mean round-trip times, at 25 to 28 ms, sit around 10 ms above the 15.7 ms floor.

Figure 6.6 illustrates this mechanism with four consecutive echoes from the Location A ping series, echoes 23 to 26. Each bar is the time the echo's reply waits for the next

scheduling request opportunity, taken as its measured round-trip time minus the 15.7 ms floor. From one echo to the next, the reply becomes ready roughly 1.2 ms later in the 20 ms cycle, the amount by which Location A's ping interval exceeds one second, so the wait shrinks from 3.2 ms to 2.0 ms and then to 0.3 ms. Echo 26 becomes ready just after the opportunity at 20 ms has passed, and has to wait 19.2 ms for the following one, which produces the jump from 16.0 ms to 34.9 ms seen in Figure 6.5.

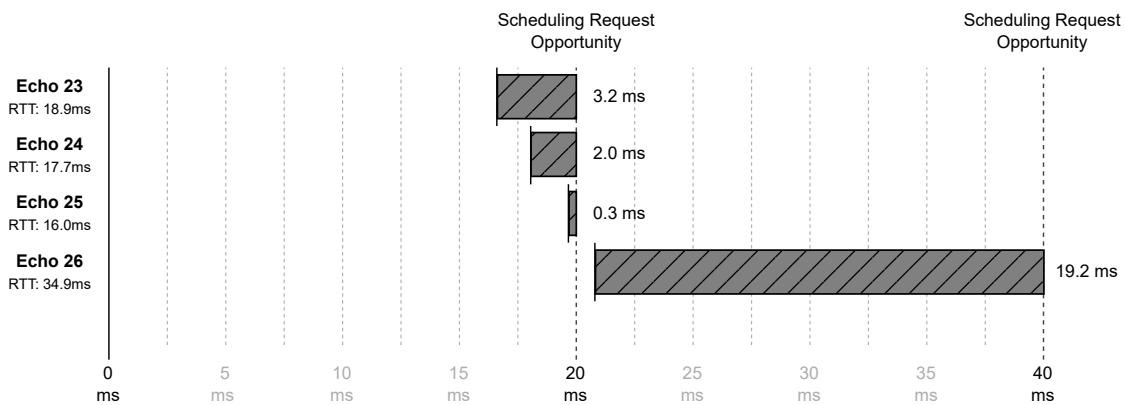


Figure 6.6: Wait for the next scheduling request opportunity of four consecutive echo replies (echoes 23 to 26 of the Location A series), relative to the 20 ms scheduling request cycle. Each wait is the echo's measured round-trip time minus the 15.7 ms floor.

The waiting time explains the spread above the 15.7 ms floor, but the scheduler's metrics also show where part of the floor itself goes. The gNodeB reports the delay between receiving a scheduling request and the PUSCH transmission it grants in its per-second scheduler metrics, an excerpt of which is shown in Listing 6.1. Across all 630 one-second reports containing at least one scheduling request in the three Round 3 logs, this delay has a median of 7.5 ms at every location and never exceeds 8 ms.

Listing 6.1: Scheduling request fields from three consecutive gnb.log scheduler-metrics entries at Location A in Round 3.

```
17:39:53 sr_count=12 avg_sr_to_pusch_delay=7.79ms
        max_sr_to_pusch_delay=8ms
17:39:54 sr_count=3  avg_sr_to_pusch_delay=7.5ms
        max_sr_to_pusch_delay=7.5ms
17:39:55 sr_count=13 avg_sr_to_pusch_delay=7.5ms
        max_sr_to_pusch_delay=7.5ms
```

Roughly half of the floor is therefore dedicated to the uplink access procedure itself, with 7.5 ms between the scheduling request and the uplink transmission, and the remaining 8.2 ms covering the downlink leg, the UE's processing of the echo request, and the return path through the gNodeB and the UPF to the core machine. Putting the section together, the idle round-trip time of the testbed is made up of this fixed 15.7 ms, plus between 0 and 20 ms spent waiting for the next scheduling request opportunity, determined by when the packet happens to arrive in the cycle rather than by how far it travels.

One qualification applies to all of the figures in this section: they describe an idle network. Once a transfer is running, the round-trip time rises by more than an order of magnitude, as the next section shows.

6.3 Latency under load

The round-trip times in Section 6.2 describe the network with no other traffic on it. The throughput tests, however, give a different picture. In a downlink TCP test, the core machine is the sender, so the `iperf3` server records the round-trip time that the TCP stack measures for each stream, along with the number of retransmissions. Table 6.4 summarizes these statistics for the Round 3 TCP downlink tests, which were run in the same session as the idle measurements.

Location	Throughput	Mean RTT (range)	Max RTT	Retransmissions	Idle mean RTT
A (2 m)	21.1–24.0 Mbps	981 ms (529–1213)	1.3–2.2 s	242–2240	25.0 ms
B (6 m)	9.2–17.0 Mbps	1272 ms (230–2011)	0.7–4.2 s	108–767	28.1 ms
C (10 m)	6.1–7.3 Mbps	2133 ms (1995–2255)	3.5–5.3 s	0–54	26.2 ms

Table 6.4: Round 3 TCP downlink tests at each location: the mean round-trip time reported by the `iperf3` server, averaged over the three repetitions, with the lowest and highest repetition in brackets; and the range across the three repetitions of the throughput, the maximum round-trip time, and the number of retransmissions, with the idle mean round-trip time from Section 6.2 for comparison.

As soon as a download starts, the mean round-trip time rises from 25–28 ms to between 0.2 and 2.3 s, with individual samples exceeding 5 s at Location C. The radio link is the same

one measured in the previous section, so this added delay is not caused by propagation or processing, but by the time the packets spend waiting in a queue.

That queue is inside the gNodeB. Each data radio bearer has a downlink RLC transmit queue, sized when the bearer is set up. Listing 6.2 shows the configuration the gNodeB printed when the UE attached.

Listing 6.2: Downlink data radio bearer configuration from gnb.log at Location A in Round 3.

```
[PDCP] DRB1 DL: PDCP configured. rlc_mode=AM discard_timer=infinity
[RLC ] DRB1 DL: RLC AM configured. queue_size=16384 queue_size_bytes
      =6172672
```

In the radio configuration, the queue holds up to 6.17 MB of downlink data, and with the PDCP discard timer set to infinity, nothing is removed from it for being too old. At the 22 Mbps reached at Location A, 6.17 MB takes about 2.2 s to transmit, and at Location C's 7 Mbps, about 7 s. TCP increases its sending rate until it detects packet loss, and as long as the queue has room, packets keep accumulating and none are lost, so TCP keeps sending more data into it. The gNodeB's own metrics confirm this: the downlink buffer it reports during these tests averages between 1.7 and 4.5 MB at Location A and between 2.3 and 3.3 MB at Location C.

At Location A, in two of the three repetitions, the round-trip time climbs steadily for around 14 seconds, then levels off at approximately 2 s, the time needed to empty a full queue at that throughput. At the same moment, retransmissions appear for the first time, in bursts of several hundred per second as the queue gets full; new packets are dropped, and only then does TCP slow down. The gNodeB's log confirms this directly: it records packets being dropped with the cause "RLC SDU queue is full" starting from 12 seconds in both of these tests, within a second of the retransmissions beginning. Listing 6.3 shows the moment the first drop occurs in the second repetition: the last packet accepted brings the queue to 6.08 MB, 98.6% of its limit, and the next packets are discarded.

Listing 6.3: First queue-full drop in gnb.log during the second Location A TCP downlink test in Round 3, 13.6 seconds into the test, shortened to the relevant fields.

```

17:40:35.600971 [RLC ] DRB1 DL: TX SDU. queued_sdus=4337 queued_bytes
    =6084811
17:40:35.601196 [PDCP] DRB1 DL: Dropping SDU. Cause: RLC SDU queue is
    full
17:40:35.603092 [PDCP] DRB1 DL: Drop burst finished. nof_sdus=15

```

At Location C, the link is slow enough that the queue only fills in the last seconds of the test, with the first drops logged 18 to 21 seconds in (the gNodeB keeps draining its queue after `iperf3` stops sending, so a test's traffic can outlast its nominal 20 seconds), too late for TCP to detect the losses before the test ends. The round-trip time climbs for the whole test, from 391 ms to 4.9 s in the second repetition, without a single retransmission, so TCP never receives the signal that would make it slow down.

This result affects how the downlink throughput figures in Section 6.1 should be read. The gNodeB used the same 6.17 MB queue in Round 2, so the 5G TCP downlink figures from both Rounds 2 and 3 were obtained with seconds of queueing delay. The throughput is real, but any interactive application, such as a video call or streaming, sharing the radio link with a download would experience such delay, in the form of buffering or stuttering.

This is not a limitation of the 5G radio, but of the gNodeB's queue size, which, when not set in its configuration file, defaults to the value reported above. A smaller queue could keep latency low under load at little cost to throughput. Admittedly, this change was identified after the final test round and was not tested, so it is left as future work.

6.4 TDD rebalance effects on uplink airtime and throughput

Returning to the throughput results of Section 6.1, the remaining sections examine the configuration factors behind them. One of the problems identified in the first round of testing was that the 5G uplink throughput was capped by the TDD structure configuration implemented in the testbed, which, as mentioned in Section 5.2, allocated TDD timeslots disproportionately in favor of the downlink. The figures in that section show that with this configuration, the 5G downlink received 68.6% of the total airtime, compared with 22.9% for the uplink.

We can verify this asymmetry by examining the raw metrics from the gNodeB trace

files, checking the scheduler's actual uplink grant rate against the theoretical maximum allowed by the TDD configuration.

In the radio configuration file, shown in Listing 6.4, the `cell_cfg` block sets the SCS to 30 kHz. As explained in Chapter 2, at this SCS, a slot is 0.5 ms long, and the 3:1 TDD pattern sets a 5-slot transmission period with one uplink slot, giving a total period of 2.5 ms. This means that the maximum number of uplink slots that can be scheduled per second is $1000 \text{ ms} / 2.5 \text{ ms} = 400$ uplink slots, regardless of the radio link quality.

Listing 6.4: `gnb.yaml` `cell_cfg` block used in Round 1.

```
cell_cfg:
  dl_arfcn: 627340
  band: 78
  channel_bandwidth_MHz: 20
  common_scs: 30
  plmn: "00101"
  tac: 7
  pci: 1
```

Listing 6.5 shows an excerpt of the raw uplink metrics from one of the Round 1 gNodeB trace files at Location A, taken while a TCP uplink transfer was saturating the link. The columns of interest here are `ok` and `nok`: the number of successful and failed uplink transmissions the scheduler reports for that one-second period. The remaining columns (Signal-to-Interference-plus-Noise Ratio (SINR), Reference Signal Received Power (RSRP), and Modulation and Coding Scheme (MCS)) speak to link quality rather than to the TDD structure itself.

Listing 6.5: Console trace from the Location A (2 m) TCP uplink test in Round 1.

-----UL-----											
	pusch	rsrp	ri	mcs	brate	ok	nok	(%)	bsr	ta	phr
	18.3	-17.0	1	14	7.77M	382	18	4%	700k	240n	3
	19.1	-16.2	1	15	7.68M	376	24	6%	700k	227n	0
	19.7	-15.6	1	16	6.25M	308	92	23%	700k	234n	-3

The `ok` and `nok` columns sum to $382 + 18 = 400$ transmissions for the first second, as do the following seconds. Iterating over the full Round 1 run and discarding the initial

TCP slow-start period (considering only seconds where the grant count reaches at least 90% of the structural ceiling), this averages to 400.0 grants/s at Location A and 399.7 grants/s at Location C, matching the same maximum established before. The 4G baseline, for comparison, measured the same way from the eNodeB traces, grants 1000.0 and 992.5 subframes/s at the same two locations, respectively. This equates to a $2.5\times$ reduction in uplink transmission opportunities ($1000.0/400.0 = 2.5$ at Location A, $992.5/399.7 = 2.48$ at Location C) between 4G and 5G performance, which no amount of signal quality could recover, since it is imposed before a single symbol is transmitted.

In the subsequent rounds of testing, the TDD pattern was rebalanced to a symmetrical 2:2 pattern, expected to improve uplink throughput at the cost of downlink throughput, making their performance almost equal, as shown in Figure 6.7. Only one configuration variable changes between each pair of rounds in the figure (the TDD pattern between Rounds 1 and 2, and `rx_gain` between Rounds 2 and 3), so the two steps should be read as evidence for two separate effects rather than a single continuous trend.

The pattern used from Round 2 onward, shown in Listing 5.2 of Chapter 5, doubles `nof_ul_slots` to 2 within the same 5-slot, 2.5 ms period, raising the structural ceiling to $2 \times 400 = 800$ uplink transmissions per second. Round 2 metrics data was extracted directly from srsRAN's native timestamped scheduler log rather than the periodic console dump used in Round 1, so the raw evidence takes a different shape: Listing 6.6 shows an excerpt of the relevant uplink fields from three consecutive one-second `gnb.log` entries for the Location A TCP uplink test (the full lines also report downlink and per-HARQ statistics, omitted here for space).

Listing 6.6: Uplink fields extracted from three consecutive `gnb.log` scheduler-metrics entries for the Location A (2 m) TCP uplink test in Round 2 (2:2 pattern).

15:14:46	<code>ul_mcs=14</code>	<code>ul_brake=14.1Mbps</code>	<code>ul_nof_ok=692</code>	<code>ul_nof_nok=108</code>
15:14:47	<code>ul_mcs=17</code>	<code>ul_brake=16.3Mbps</code>	<code>ul_nof_ok=647</code>	<code>ul_nof_nok=153</code>
15:14:48	<code>ul_mcs=19</code>	<code>ul_brake=18.1Mbps</code>	<code>ul_nof_ok=638</code>	<code>ul_nof_nok=162</code>

The `ul_nof_ok` and `ul_nof_nok` fields sum to exactly 800 in every row shown, and the same holds throughout the saturated portion of the run at both Location A and Location C, confirming the scheduler reaches the full doubled ceiling regardless of distance. This is

a direct consequence of the adjustment in the TDD pattern, which is what the grant rate alone predicts.

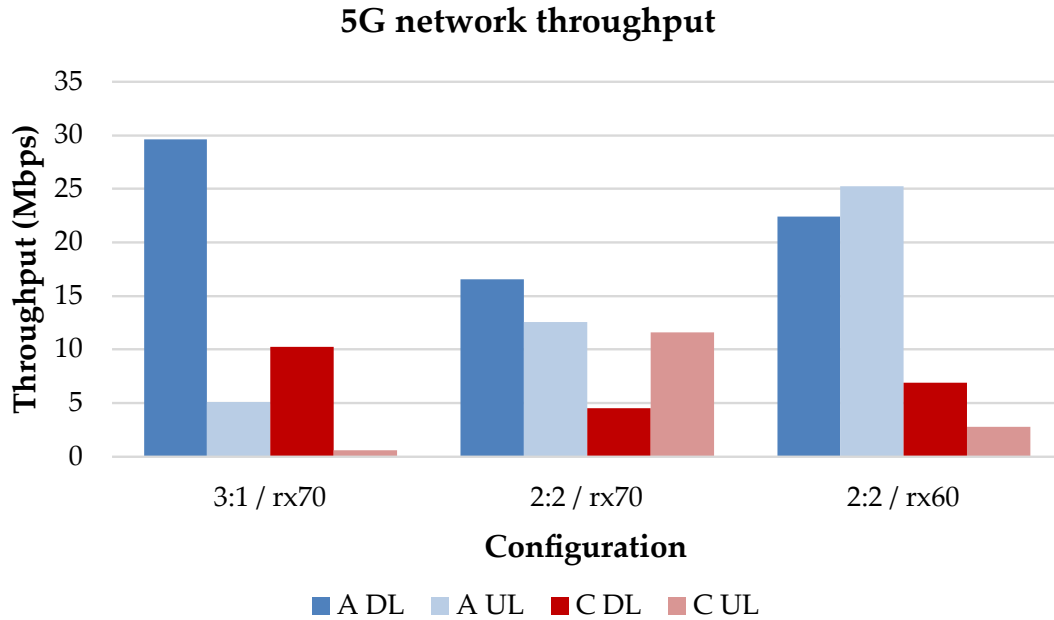


Figure 6.7: TCP throughput at Locations A (2 m) and C (10 m) across the three 5G rounds, labeled by the configuration tested in each location.

We see in the figure that initially, as expected, when using the 3:1 TDD pattern, the uplink throughput in both Location A and C was significantly lower than the downlink throughput. In Round 2, after rebalancing the TDD pattern to 2:2, the uplink performance rose to nearly match the downlink in Location A, surpassing the downlink throughput at Location C. We also note that the downlink throughput decreased in both locations, as expected.

The same airtime accounting also predicts the downlink share should fall to $\times 0.69$ of its Round 1 value; the observed downlink throughput fell further still, to $\times 0.56$ at Location A and $\times 0.44$ at Location C, so some factor beyond airtime alone also affected the downlink.

Finally, in Round 3, after adjusting the rx_gain to 60: at Location A (2 m), the overall network throughput increased significantly, as this change removed the receiver overload reported at that location, thus stabilizing the network’s performance. However, at Location C, further away from the radio, the uplink throughput decreased significantly as well, due

to the lower `rx_gain` desensitizing the receiver and reducing the link budget. This change was foreseeable as a trade-off between the two locations, and it highlights the challenge of finding a single configuration that optimizes performance across all distances.

We also see that in this final configuration, the throughput between both directions is closest to being equal at short range: at Location A the two directions converge to within a narrow margin (22.4 vs. 25.2 Mbps), the best balance observed anywhere in the dataset, while at Location C the `rx_gain` reduction gives back most of the uplink gain Round 2 had recovered. This Location C behavior is the trade-off associated with the `rx_gain` change.

6.5 Radio link quality

The previous section showed that the TDD pattern limits how often the 5G uplink can transmit. How much data each transmission carries depends on the quality of the radio link, which this section examines through two metrics the RAN reports for every uplink transmission: the SINR measured by the base station, and the power headroom reported by the UE. The power headroom is the difference between the UE's maximum transmit power and the power it is currently using, so a positive value means the UE still has power in reserve. A negative value means it is being asked for more power than it can deliver.

6.5.1 Round 1: 4G and 5G

Table 6.5 compares both metrics for the two generations in Round 1, averaged over the uplink tests at each location.

Location	4G SINR	5G SINR	4G headroom	5G headroom
A (2 m)	13.6 dB	14.6 dB	17.1 dB	-1.7 dB
B (6 m)	14.3 dB	8.0 dB	14.0 dB	-13.1 dB
C (10 m)	12.0 dB	-3.8 dB	9.8 dB	-13.8 dB

Table 6.5: Round 1 mean uplink SINR and UE power headroom at each location, averaged over the TCP and UDP uplink tests ($n = 1$).

At Location A, both networks see a similar uplink SINR. With increasing distance, however, the two behave very differently: the 4G SINR stays between 12 and 14.3 dB at every location, while the 5G SINR falls by more than 18 dB between Locations A and C. The power headroom explains the difference. The 4G UE keeps between 10 and 17 dB

of transmit power in reserve at every location, so as the path loss grows, the eNodeB's power control can ask it to transmit harder and hold the SINR roughly constant. The 5G UE, on the other hand, has no headroom left even at Location A, and is well beyond its maximum at Locations B and C, so every additional decibel of path loss is lost directly from the received SINR.

Part of this difference is expected, as the 5G RAN operates at a higher frequency, 3.4 GHz against 2.6 GHz, and without the low noise amplifier used on the 4G receive chain. However, why the 5G UE runs out of headroom this early was not established.

6.5.2 Rounds 2 and 3: the receiver gain trade-off

Between Round 2 and Round 3, the 5G uplink was also affected by the receive gain of the radio, set by the `rx_gain` parameter. Table 6.6 compares the uplink at each location with `rx_gain` set to 70 in Round 2 and to 60 in Round 3, with the same 2:2 TDD pattern in both. Location B is only shown for Round 3, as its Round 2 data was excluded (Section 5.3.2).

Location	rx_gain	Throughput	SINR	MCS	BLER	Headroom
A (2 m)	70	13.9 Mbps	20.5 dB	18.5	22.4%	-1.9 dB
	60	28.5 Mbps	25.2 dB	24.8	6.6%	-12.9 dB
B (6 m)	60	7.1 Mbps	11.0 dB	6.9	1.7%	-11.3 dB
C (10 m)	70	12.9 Mbps	18.0 dB	12.1	1.3%	-16.1 dB
	60	2.2 Mbps	0.7 dB	1.0	5.7%	-22.6 dB

Table 6.6: 5G uplink at `rx_gain` 70 (Round 2) and 60 (Round 3), averaged over the TCP and UDP uplink tests at each location ($n = 3$): throughput, SINR, mean MCS index, BLER and UE power headroom.

At `rx_gain` 70, the two ends of the room show an inconsistency. The gNodeB reports a similar SINR at Locations A and C, 20.5 and 18.0 dB, yet at Location A, the closest to the radio, almost a quarter of the uplink transmissions fail, against 1.3% at Location C. A SINR reading that high, specifically paired with such high BLER, is the signature of an overloaded receiver, as the distortion produced by the saturated front end is not recognized as noise, so the reported SINR stays high, making the scheduler pick an MCS that the distorted signal cannot support.

The radio's own `ov1` flag, which srsRAN reports whenever the receiver is saturated, supports this reading. The gNodeB writes one metrics line per second for as long as the

UE is attached, so they cover the whole session at each location, the twelve tests and the intervals between them. At `rx_gain 70`, the flag was raised 28 times at Location A and 12 times at Location B, and never at Location C. Of the 28 at Location A, six were raised while the uplink was carrying between 6 and 10 Mbps and four during downlink transfers, several of them with uplink error rates between 25 and 100%.

Lowering `rx_gain` to 60 in Round 3 removed the overload at Location A, where the flag was raised only three times over a session of comparable length. The uplink BLER fell from 22.4 to 6.6%, and the throughput doubled to 28.5 Mbps, the highest uplink throughput measured in the testbed. At Location C, the same change had the opposite effect: the SINR fell by 17 dB, to just above the noise floor, and the throughput fell to less than a fifth of its Round 2 value.

The power headroom rules out one possible explanation for why the same 10 dB reduction in receive gain helped one location and harmed the other. At `rx_gain 70`, the UE was already at or beyond its maximum transmit power at both locations, with a headroom of -1.9 dB at Location A and -16.1 dB at Location C, so in neither case could it transmit harder to make up for the gain removed from the receiver. The further drop in headroom in Round 3, by 11 dB at Location A and 6.5 dB at Location C, reflects the power the gNodeB's power control asked for, not the power the UE was able to deliver.

What separates the two locations is instead what limited the link at `rx_gain 70`. At Location A, the signal was strong enough to saturate the receiver, and the link was limited by the resulting distortion rather than by noise. Removing 10 dB of gain brought the signal back into the receiver's linear range; the distortion disappeared, and the SINR rose from 20.5 to 25.2 dB, even though less signal reached the digital stage. At Location C, the receiver was never overloaded, and the link was already limited by noise. With no low noise amplifier ahead of the radio, lowering its gain raises the noise contributed by the rest of the receive chain relative to the signal. With the UE unable to compensate, the SINR fell from 18.0 to 0.7 dB.

With a single, manually set receive gain and no automatic gain control, the gNodeB cannot serve the whole room. A gain of 70 suits Location C and overloads the receiver at Location A, while a gain of 60 suits Location A and leaves Location C barely above the noise floor. This is a consequence of running a software RAN on an SDR whose receive

gain is fixed at start-up.

6.6 Core network performance

The core network is involved in the data path only through the UPF, whose contribution to the round-trip time is part of the 15.7 ms floor measured in Section 6.2. Its main role is in the control plane: registering the UE, authenticating it, and setting up the session through which it reaches the data network. The time this takes can be measured directly from the Open5GS logs, which record every registration request received by the AMF, and every attach request received by the MME, together with the moment the procedure completes. Table 6.7 summarizes these times over the three test sessions.

The same intervals can also be measured from the outside, in the NGAP captures taken at the gNodeB, which record the procedure from the radio side rather than from within the core. These files can be opened through Wireshark and reveal that 23 registrations appear in these captures. For each one of them, the time from the `InitialUEMessage` carrying the registration request to the message with which the AMF closes the procedure matches the interval in the Core’s AMF log to within 2 ms, despite being recorded on a different machine and a different clock.

Procedure	Count	Median	Range
5G registration, all	39	186 ms	142–383 ms
with network function discovery	6	320 ms	289–383 ms
without network function discovery	33	186 ms	142–350 ms
4G attach	8	342 ms	322–386 ms

Table 6.7: Time from the UE’s registration or attach request to the completion of the procedure, from the AMF (5G) and MME (4G) logs of the three test sessions.

A 5G registration took a median of 186 ms. The six registrations that required network function discovery were among the slowest and share two properties, both visible in the AMF logs. The first is that the AMF had to set up its connections to the AUSF, UDM and PCF during the registration rather than reuse existing ones: in the SBA, a network function does not know in advance where the others are, so it first asks the NRF for them, and then connects to each one. Timing this directly, from the AMF’s request for each network function to the endpoint being established, gives between 12 and 16 ms for all

three together.

The second property accounts for most of the difference: in these six registrations, the AMF, three held no security context for the UE, so the procedure ran a full authentication, exchanging an identity request, an authentication request, and a security mode command with the UE, each one a round trip over the air. One of the six, the slowest registration of the campaign, was also captured on the NGAP interface at the gNodeB, and Figure 6.8 places it next to a registration that reused a context, so that the two can be followed frame by frame.

No.	Time	Protocol	Length	Info
1	0.000	NGAP	66	NGSetupRequest
2	0.009	NGAP	66	NGSetupResponse
3	17.181	NGAP/NAS-SGS	141	InitialUEMessage, Registration request [RRCEstablishmentCause=mo-Signalling]
4	17.182	NGAP/NAS-SGS	40	DownlinkNASTransport, Identity request
5	17.208	NGAP/NAS-SGS	80	UplinkNASTransport, Identity response
6	17.212	NGAP/NAS-SGS	78	DownlinkNASTransport, Authentication request
7	17.308	NGAP/NAS-SGS	83	UplinkNASTransport, Authentication response
8	17.313	NGAP/NAS-SGS	55	DownlinkNASTransport, Security mode command
9	17.348	NGAP/NAS-SGS	125	UplinkNASTransport
10	17.358	NGAP/NAS-SGS	161	InitialContextSetupRequest
11	17.438	NGAP	31	InitialContextSetupResponse
12	17.438	NGAP	904	UERadioCapabilityInfoIndication
13	17.468	NGAP/NAS-SGS	65	UplinkNASTransport
14	17.564	NGAP/NAS-SGS	87	DownlinkNASTransport
15	17.588	NGAP/NAS-SGS	136	UplinkNASTransport
16	17.598	NGAP/NAS-SGS	218	PDUSessionResourceSetupRequest
17	17.648	NGAP	52	PDUSessionResourceSetupResponse

No.	Time	Protocol	Length	Info
1	0.000	NGAP	66	NGSetupRequest
2	0.007	NGAP	66	NGSetupResponse
3	6.141	NGAP/NAS-SGS	141	InitialUEMessage, Registration request [RRCEstablishmentCause=mo-Signalling]
4	6.142	NGAP/NAS-SGS	161	InitialContextSetupRequest
5	6.191	NGAP	31	InitialContextSetupResponse
6	6.191	NGAP	904	UERadioCapabilityInfoIndication
7	6.229	NGAP/NAS-SGS	65	UplinkNASTransport
8	6.316	NGAP/NAS-SGS	87	DownlinkNASTransport
9	6.349	NGAP/NAS-SGS	136	UplinkNASTransport
10	6.361	NGAP/NAS-SGS	218	PDUSessionResourceSetupRequest
11	6.409	NGAP	52	PDUSessionResourceSetupResponse

Figure 6.8: Two 5G registrations as recorded in the NGAP captures written by the gNodeB. Top: the slowest registration of the campaign, at Location A in Round 1, in which the AMF held no security context for the UE. Bottom: a registration at Location B in the same round, in which it did. In both, the registration request is frame 3.

In the upper capture, the registration request arrives in frame 3, and the AMF accepts it in frame 10 with the `InitialContextSetupRequest` that carries the registration accept. The six frames in between are the authentication: an identity request and its response, an authentication request and its response, and a security mode command and its completion. Together they take 178 of the registration's 383 ms, and each is a round trip to the UE over the radio, which is why they dominate the total. In the lower capture, those six frames are

absent, and the accept follows the request as frame 4, a single millisecond later. The cost specific to the SBA is therefore around 13 ms, paid only on first use, while the rest of the spread in Table 6.7 is the ordinary cost of authenticating a UE, which 4G pays as well.

The one registration without discovery that took longer, at 350 ms, occurred at Location C in Round 3, where the uplink SINR was 0.7 dB. The AMF completed its own part within 9 ms, and most of the remaining time was spent on the radio side, where the initial context setup with the UE took 217 ms, compared with around 50 ms at the other locations.

CONCLUSIONS

This dissertation set out to build a private 5G SA network from open-source software and COTS hardware, and to evaluate it against a 4G network built with the same tools and sharing the same core. This chapter summarizes the work carried out and its main findings in Section 7.1, discusses the limitations of the evaluation in Section 7.2, and outlines the work left open for the future in Section 7.3.

7.1 Summary and Main Findings

Both networks were deployed and remain operational. The testbed consists of four machines connected through a switch: a core machine running Open5GS, which serves as the 5G core for the gNodeB and as the EPC for the eNodeB, a 5G RAN machine running the srsRAN Project gNodeB with a USRP B210 disciplined by a GPSDO, a 4G RAN machine running srsRAN 4G with a bladeRF xA4, and a fourth machine reserved for a future NSA deployment. Every step of the setup is documented in Appendix A, organized by machine, so that each network can be rebuilt, or extended, without repeating the exploration carried out in this work.

The two networks were evaluated at three locations in the same room, with increasing distance from the radio and different obstructions, over three test rounds. The results of these rounds lead to the following main findings.

- In Round 1, the 5G uplink throughput was more than 20 times lower than that of the 4G baseline at the farthest location. Part of this deficit came from the 3:1 TDD pattern inherited from a reference configuration, which allowed only 400 uplink

transmissions per second against the 1000 subframes per second of the 4G FDD uplink. Rebalancing the pattern to 2:2 doubled the uplink transmission opportunities to 800 per second, as measured in the scheduler's own metrics, and brought the 5G uplink to the level of the 4G baseline, at the cost of part of the downlink throughput.

- With `rx_gain` at 70, the receiver was overloaded close to the radio, where almost a quarter of the uplink transmissions failed. Lowering it to 60 removed the overload and doubled the uplink throughput at 2 m, to 28.5 Mbps, the best uplink result of the testbed and around twice the 4G baseline at the same location. The same change, however, left the 10 m location barely above the noise floor, as the UE there had no transmit power left to compensate. With a receive gain fixed at start-up and no automatic gain control, the gNodeB can be tuned for short or long range, but not for both.
- The 5G downlink, with a 20 MHz channel against the 10 MHz of the 4G network, reached up to 50.6 Mbps over UDP in Round 1, more than twice the 4G baseline at every location. However, this margin narrowed once the TDD pattern gave part of the airtime to the uplink, and at the farthest location was reversed in Round 3 by the lower receive gain.
- With no other traffic on the network, the 5G round-trip time has a floor of around 16 ms at every location, of which around 7.5 ms is the delay between a scheduling request and the uplink transmission it is granted. On top of this floor, every uplink packet waits between 0 and 20 ms for the next scheduling request opportunity, set by the default 20 ms scheduling request period, which places the mean round-trip time between 25 and 28 ms.
- During a TCP download, the mean round-trip time rose to between 0.2 and 2.3 s, as the default 6.17 MB downlink RLC queue filled with data that TCP had no reason to hold back. The downlink throughput measured in the test rounds is real, but it came with seconds of delay that would degrade the performance of any interactive application sharing the link.

- A 5G registration took a median of 186 ms, and a 4G attach took 342 ms median. The only cost specific to the 5G SBA was the discovery of the other network functions through the NRF, which added around 13 ms to the first registrations after the core was started, and did not recur once the connections were established.

Taken together, these findings answer the question posed in Chapter 1: a 5G SA network built entirely from open-source software and COTS hardware can match or exceed a 4G network built with the same tools, but only after its configuration has been adapted to the hardware and environment in which it runs. The 4G network reached the performance reported here with its first configuration and was never tuned again. In contrast, the 5G network needed a rebalanced TDD pattern and a receive gain sweep before it matched (and then doubled) at short range, which still serves only part of the room.

What makes these parameters easy to miss is that none of them prevent the network from working: the cell starts, the UE attaches, and traffic flows in both directions with all of them at the wrong value, and only a measurement reveals what they cost. Most of the problems found in this work were therefore not limitations of 5G or of the open-source stack, but defaults and inherited values that had never been revisited, and the diagnosis of each one is documented so that others deploying a similar stack can avoid repeating it.

7.2 Limitations

The results of this dissertation should be read with the following limitations in mind.

- The 4G network was only tested in Round 1, with a single repetition per condition, and its throughput was taken from the RAN's own metrics rather than from `iperf3`. Rounds 2 and 3 carried three repetitions, with throughput measured by `iperf3` itself. A repetition of the 4G tests under the later procedure was planned but could not be carried out, as the Nuand bladeRF used by the 4G RAN was no longer available in the laboratory. The comparisons between the 4G baseline and the later 5G rounds are therefore indicative rather than exact.
- The 4G and 5G networks use different frequency bands, bandwidths, duplexing schemes and radios, and only the 4G receive chain includes a low noise amplifier.

Each result therefore compares this 4G deployment with this 5G deployment, rather than the two generations in general.

- ping latency measurements were only carried out for the 5G network, in Round 3.
- The reason why the 5G UE had no transmit power in reserve even at the closest location was not established, just as the reason why the Motorola UE could not connect to the 5G network was not determined.

7.3 Future Work

The testbed remains in place, and the findings of this work suggest several ideas for future work to go with.

- Reducing the downlink RLC queue of the gNodeB, and enabling a PDCP discard timer, should keep the round-trip time low during a download at little cost to throughput. This change was identified after the last test round and is the most valuable one left to test.
- A shorter scheduling request period than the default 20 ms would reduce the time an uplink packet waits for its next opportunity, and with it the mean round-trip time, at the cost of more uplink control resources.
- The 4G RAN uses a power amplifier on its transmit chain and an LNA on its receive chain, while the 5G RAN has neither. A power amplifier was proposed during the project, but none was available in time. Adding an amplifier on transmit would extend the downlink range, while an LNA on the receive chain, combined with a fixed attenuator, would raise the uplink sensitivity at the farther locations without saturating the receiver close to the radio, which is the trade-off the `rx_gain` sweep could not resolve on its own.
- Calibrating the cell's power parameters against the radio hardware would address the lack of UE power headroom found in Round 1.
- Repeating the 4G tests with the procedure used in Rounds 2 and 3, including latency measurements, would make the comparison between the two generations direct.

- The fourth machine was reserved for a 5G NSA deployment, which would allow the two 5G deployment models discussed in Chapter 2 to be compared on the same hardware. Tests with several UEs, and over longer periods, would also show how the network behaves under the conditions of its intended use in research and coursework.

BIBLIOGRAPHY

- [1] 3rd Generation Partnership Project (3GPP). *NR; Physical channels and modulation*. Tech. rep. TS 38.211. 3GPP, 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.211 (visited on 2026-01-27) (cit. on pp. 15–17).
- [2] 3rd Generation Partnership Project (3GPP). *NR; Physical layer procedures for control*. Tech. rep. TS 38.213. 3GPP, 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.213 (visited on 2026-01-27) (cit. on pp. 16, 17, 52).
- [3] 3rd Generation Partnership Project (3GPP). *Release 15*. 2018. URL: <https://www.3gpp.org/specifications-technologies/releases/release-15> (visited on 2026-01-27) (cit. on pp. 10, 22).
- [4] 3rd Generation Partnership Project (3GPP). *Release 17*. 2022. URL: <https://www.3gpp.org/specifications-technologies/releases/release-17> (visited on 2026-01-27) (cit. on p. 15).
- [5] 3rd Generation Partnership Project (3GPP). *Study on New Radio Access Technology: Radio Access Architecture and Interfaces*. Tech. rep. TR 38.801. 3GPP, 2017. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.801 (visited on 2026-01-27) (cit. on p. 9).
- [6] Abhi5h3k. *CellularLab: iPerf3 Client for Android*. URL: <https://github.com/Abhi5h3k/CellularLab> (cit. on pp. 51, 116).
- [7] J. Ansari, C. Andersson, P. de Bruin, J. Farkas, L. Grosjean, J. Sachs, J. Torsner, B. Varga, D. Harutyunyan, N. König, and R. H. Schmitt. “Performance of 5G Trials

- for Industrial Automation”. In: *Electronics* 11.3 (2022). ISSN: 2079-9292. DOI: [10.3390/electronics11030412](https://doi.org/10.3390/electronics11030412) (cit. on p. 1).
- [8] M. Barbosa, M. Silva, E. Cavalcanti, and K. Dias. “Open-Source 5G Core Platforms: A Low-Cost Solution and Performance Evaluation”. In: *2025 International Conference on Information Networking (ICOIN)*. 2025. DOI: [10.1109/ICOIN63865.2025.10992769](https://doi.org/10.1109/ICOIN63865.2025.10992769) (cit. on pp. 2, 32, 36).
- [9] L. Bonati, M. Polese, S. D’Oro, P. Brach del Prever, and T. Melodia. “5G-CT: Automated Deployment and Over-the-Air Testing of End-to-End Open Radio Access Networks”. In: *IEEE Communications Magazine* 63.1 (2025), pp. 155–162. DOI: [10.1109/MCOM.001.2300675](https://doi.org/10.1109/MCOM.001.2300675) (cit. on pp. 22–24, 32, 36).
- [10] G. Brown. *Ultra-Reliable Low-Latency 5G for Industrial Automation. A Heavy Reading White Paper Produced for Qualcomm Inc.* White Paper. Heavy Reading, 2018. URL: <https://www.qualcomm.com> (visited on 2026-01-20) (cit. on p. 1).
- [11] W. Ejaz, A. Anpalagan, M. A. Imran, M. Jo, M. Naeem, S. B. Qaisar, and W. Wang. “Internet of Things (IoT) in 5G Wireless Communications”. In: *IEEE Access* 4 (2016), pp. 10310–10314. DOI: [10.1109/ACCESS.2016.2646120](https://doi.org/10.1109/ACCESS.2016.2646120) (cit. on p. 1).
- [12] Facebook. *Magma: Open Source Software Platform for Building Carrier-Grade Networks*. URL: <https://github.com/magma/magma> (cit. on p. 22).
- [13] free5GC Project. *free5GC: Open Source 5G Core Network Based on 3GPP Release 15*. URL: <https://github.com/free5gc/free5gc> (cit. on p. 22).
- [14] GSMArena. *Motorola Moto G56 5G – Full Phone Specifications*. URL: https://www.gsmarena.com/motorola_moto_g56_5g-13841.php (cit. on p. 45).
- [15] GSMArena. *Samsung Galaxy Z Flip5 – Full Phone Specifications*. URL: https://www.gsmarena.com/samsung_galaxy_z_flip5-12252.php (cit. on p. 45).
- [16] A. Güngör. *UERANSIM: Open Source 5G UE and RAN (gNodeB) Simulator*. URL: <https://github.com/aligungr/UERANSIM> (cit. on p. 24).
- [17] hamarituc. *lbgpsdo: Configuration Utility for Leo Bodnar GPSDO*. URL: <https://github.com/hamarituc/lbgpsdo> (cit. on pp. 40, 109).

- [18] S. Hasan, A. Padmanabhan, B. Davie, J. Rexford, U. Kozat, H. Gatewood, S. Sanadhya, N. Yurchenko, T. Al-Khasib, O. Batalla, M. Bremner, A. Lee, E. Makeev, S. Moeller, A. Rodriguez, P. Shelar, K. Subraveti, S. Kandi, A. Xoconostle, P. Kumar Ramakrishnan, and X. Tian. “Building Flexible, Low-Cost Wireless Access Networks with Magma”. In: *GetMobile: Mobile Comp. and Comm.* 27.3 (2023-11), pp. 40–47. ISSN: 2375-0529. DOI: [10.1145/3631588.3631599](https://doi.org/10.1145/3631588.3631599) (cit. on pp. 2, 23, 34).
- [19] T. Horstmann, D. Brunke, T. Kremeyer, M. Wilmes, G. Schneider, J. Sturm, H. König, and M. Rademacher. *open5Gcube: A Modular and Usable Framework for Mobile Network Laboratories*. 2025. arXiv: [2505.14501](https://arxiv.org/abs/2505.14501) [cs.NI] (cit. on pp. 22–24, 30, 36).
- [20] K. Khujamatov, D. Khasanov, E. Reypnazarov, and N. Akhmedov. “Existing Technologies and Solutions in 5G-Enabled IoT for Industrial Automation”. In: *Blockchain for 5G-Enabled IoT: The new wave for Industrial Automation*. Ed. by S. Tanwar. Cham: Springer International Publishing, 2021, pp. 181–221. ISBN: 978-3-030-67490-8. DOI: [10.1007/978-3-030-67490-8_8](https://doi.org/10.1007/978-3-030-67490-8_8) (cit. on p. 1).
- [21] J. M. Lourenço. *novathesis: A LaTeX Template for Academic Theses and Dissertations*. Version 8.3.0. NOVA University Lisbon. 2026. DOI: [10.5281/zenodo.21938603](https://doi.org/10.5281/zenodo.21938603) (cit. on p. 124).
- [22] Z. Lv, J. Lloret, and H. Song. “Internet of Things and augmented reality in the age of 5G”. In: *Computer Communications* 164 (2020), pp. 158–161. ISSN: 0140-3664. DOI: [10.1016/j.comcom.2020.08.019](https://doi.org/10.1016/j.comcom.2020.08.019) (cit. on p. 1).
- [23] L. Mamushiane, A. Lysko, H. Kobo, and J. Mwangama. “Deploying a Stable 5G SA Testbed Using srsRAN and Open5GS: UE Integration and Troubleshooting Towards Network Slicing”. In: *2023 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD)*. 2023, pp. 1–10. DOI: [10.1109/icABCD59051.2023.10220512](https://doi.org/10.1109/icABCD59051.2023.10220512) (cit. on pp. 2, 8, 22, 24, 25, 29, 36).
- [24] MongoDB, Inc. *MongoDB Community Edition: Installation Guide for Linux*. URL: <https://www.mongodb.com/docs/manual/administration/install-community/?operating-system=linux&linux-distribution=ubuntu&linux-package=default&search-linux=with-search-linux> (cit. on p. 93).

-
- [25] OCUDU Project. *OCUDU: Open Source O-RAN 5G CU/DU Solution (formerly srsRAN Project)*. Governed under the Linux Foundation; the srsRAN_Project GitHub repository was archived in June 2026 in favour of this repository. URL: <https://gitlab.com/ocudu/ocudu> (cit. on pp. 2, 24, 43, 44, 110).
- [26] Open5GS. *Open5GS Guide: Quick Start*. URL: <https://open5gs.org/open5gs/docs/guide/01-quickstart/> (cit. on p. 92).
- [27] Open5GS. *Open5GS: Open Source 5G Core Project*. AGPL-3.0 License. URL: <https://github.com/open5gs/open5gs> (cit. on pp. 2, 6, 22, 40, 42).
- [28] OpenAirInterface Software Alliance. *oai-cn5g-fed: Federation of the OpenAir CN 5G Repositories*. URL: <https://gitlab.eurecom.fr/oai/cn5g/oai-cn5g-fed> (cit. on p. 23).
- [29] OpenAirInterface Software Alliance. *OpenAirInterface5G: Open Source 4G/5G RAN and Core Network*. URL: <https://gitlab.eurecom.fr/oai/openairinterface5g> (cit. on p. 24).
- [30] M. N. Patwary, S. J. Nawaz, M. A. Rahman, S. K. Sharma, M. M. Rashid, and S. J. Barnes. “The Potential Short- and Long-Term Disruptions and Transformative Impacts of 5G and Beyond Wireless Networks: Lessons Learnt From the Development of a 5G Testbed Environment”. In: *IEEE Access* 8 (2020), pp. 11352–11379. doi: [10.1109/ACCESS.2020.2964673](https://doi.org/10.1109/ACCESS.2020.2964673) (cit. on p. 1).
- [31] M. Romão. *Open-Source 5G Testbed – Thesis Repository*. URL: <https://github.com/mvromao/thesis> (cit. on pp. 42, 46, 94, 105, 111).
- [32] S. Rommer, P. Hedman, M. Olsson, L. Frid, S. Sultana, and C. Mulligan. *5G Core Networks: Powering Digitalization*. Academic Press, 2020 (cit. on pp. 18, 21).
- [33] M. Rouili, N. Saha, M. Golkarifard, M. Zangoeei, R. Boutaba, E. Onur, and A. Saleh. “Evaluating Open-Source 5G SA Testbeds: Unveiling Performance Disparities in RAN Scenarios”. In: *2024 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. 2024, pp. 1–6. doi: [10.1109/NOMS59830.2024.10575687](https://doi.org/10.1109/NOMS59830.2024.10575687) (cit. on pp. 2, 30, 34, 36).

- [34] Simlessly. *Simlessly: eSIM Management and Remote SIM Provisioning Platform*. URL: <https://simlessly.com/> (cit. on p. 120).
- [35] Y. Siriwardhana, P. Porambage, M. Liyanage, and M. Ylianttila. “A Survey on Mobile Augmented Reality With 5G Mobile Edge Computing: Architectures, Applications, and Technical Aspects”. In: *IEEE Communications Surveys & Tutorials* 23.2 (2021), pp. 1160–1192. DOI: [10.1109/COMST.2021.3061981](https://doi.org/10.1109/COMST.2021.3061981) (cit. on p. 1).
- [36] S. Sirotkin. *5G Radio Access Network Architecture: The Dark Side of 5G*. Wiley-IEEE Press, 2021 (cit. on pp. 12, 13, 18, 21).
- [37] Software Radio Systems. *srsRAN 4G: Open Source 4G Software Radio Suite*. URL: https://github.com/srsran/srsran_4g (cit. on pp. 2, 24, 44).
- [38] B. Tío, F. Pedreira, L. Barboni, and C. Rattaro. *Exploring 5G with Open Source Solutions: Functional Testbed and User Equipment Evaluation*. Tech. rep. Facultad de Ingeniería, Universidad de la República, 2025. URL: <https://case.ar/wp-content/uploads/2025/07/Exploring-5G-with-Open-Source-Solutions.pdf> (cit. on pp. 2, 22, 24, 27, 36).
- [39] M. Torres Vega, C. Liaskos, S. Abadal, E. Papapetrou, A. Jain, B. Mouhouche, G. Kalem, S. Ergüt, M. Mach, T. Sabol, A. Cabellos-Aparicio, C. Grimm, F. De Turck, and J. Famaey. “Immersive Interconnected Virtual and Augmented Reality: A 5G and IoT Perspective”. In: *Journal of Network and Systems Management* 28.4 (2020), pp. 796–826. ISSN: 1573-7705. DOI: [10.1007/s10922-020-09545-w](https://doi.org/10.1007/s10922-020-09545-w) (cit. on p. 1).
- [40] P. Vázquez, W. Peña, W. Piastrì, L. Inglés, and C. Rattaro. “MAQ5G: Deployment of a Complete 5G Standalone Network Testbed for Testing and Development”. In: *2024 XVI Congreso de Tecnología, Aprendizaje y Enseñanza de la Electrónica (TAEE)*. 2024, pp. 1–7. DOI: [10.1109/TAEE59541.2024.10604930](https://doi.org/10.1109/TAEE59541.2024.10604930) (cit. on pp. 23, 24, 28, 36).
- [41] D. Wang, D. Chen, B. Song, N. Guizani, X. Yu, and X. Du. “From IoT to 5G I-IoT: The Next Generation IoT-Based Intelligent Algorithms and 5G Technologies”. In: *IEEE Communications Magazine* 56.10 (2018), pp. 114–120. DOI: [10.1109/MCOM.2018.1701310](https://doi.org/10.1109/MCOM.2018.1701310) (cit. on p. 1).

- [42] G. Zu, J. Boateng, V. Advani, T. U. Islam, V. Lee, S. Babu, M. Nadim, D. Qiao, M. Selim, and H. Zhang. “Real-World Integration and Evaluation of Open-Source 5G Core with Commercial RAN”. In: *2025 IEEE Military Communications Conference (MILCOM)*. 2025-10, pp. 1296–1301. doi: [10.1109/MILCOM64451.2025.11309851](https://doi.org/10.1109/MILCOM64451.2025.11309851) (cit. on pp. [33](#), [36](#)).

TESTBED SETUP

This appendix is a step-by-step guide to rebuilding the testbed described in Chapter 4. It is organized by machine, following the layout in Figure 4.6: the steps in each section are run on the machine that section names, and a reader interested in only one generation can skip the section for the other.

- **4G network:** Sections A.1, A.2, A.3 and A.5.
- **5G network:** Sections A.1, A.2, A.4 and A.5.

All machines run Ubuntu 22.04 LTS, and the software versions used in this dissertation are listed in Table 4.2. The UE additionally needs an eSIM profile, whose provisioning is described in Appendix B.

A.1 Network Connection (All Machines)

The machines are connected through an Ethernet switch, each with a static IP address, as shown in Figure 4.1 and listed below. These connections carry the control and user plane traffic between the core and both RANs.

Machine	Role	IP address
1	Core	10.0.0.10/24
2	5G RAN	10.0.0.20/24
3	5G NSA RAN (unused)	10.0.0.30/24
4	4G RAN	10.0.0.40/24

Table A.1: Static IP addresses of the testbed machines.

The steps are the same on every machine, differing only in the IP address assigned. Connect the machine to the switch and check the available network interfaces using the following command:

```
ip a
```

The output shows the available network interfaces, including the Ethernet interface (in this case `enp0s31f6`), as shown in Figure A.1.

```
core@core:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s31f6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether e8:cf:83:38:6e:22 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.10/24 brd 10.0.0.255 scope global noprefixroute enp0s31f6
        valid_lft forever preferred_lft forever
3: ogstun: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 500
    link/none
    inet 10.45.0.1/16 brd 10.45.255.255 scope global ogstun
        valid_lft forever preferred_lft forever
    inet6 2001:db8:cafe::1/48 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::bcbf:ee2d:1f27:9129/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
4: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 28:95:29:3e:b5:f2 brd ff:ff:ff:ff:ff:ff
    inet 10.22.124.34/22 brd 10.22.127.255 scope global dynamic noprefixroute wlp0s20f3
        valid_lft 3389sec preferred_lft 3389sec
    inet6 fe80::6285:5995:98b1:d032/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
core@core:~$
```

Figure A.1: Output of the `ip a` command on the core machine once set up, showing the Ethernet interface (`enp0s31f6`) at `10.0.0.10/24`, and the `ogstun` interface created by Open5GS for the UE address pool (`10.45.0.1/16`).

Enable the Ethernet interface (if disabled) and assign a static IP address to it. On the core machine:

```
sudo ip link set enp0s31f6 up
sudo ip addr add 10.0.0.10/24 dev enp0s31f6
```

To make the connection persistent across reboots, the network configuration needs to be added to the Netplan configuration file. This file may have a different name from the one in this example. Open the Netplan configuration file at `/etc/netplan/01-netcfg.yaml`

using a text editor and add the following configuration to assign a static IP address to the Ethernet interface, replacing the address with the one for the machine being configured:

```
# Let NetworkManager manage all devices on this system

network:

  version: 2

  renderer: NetworkManager

  ethernets:

    enp0s31f6:

      dhcp4: false

      addresses:

        - 10.0.0.10/24
```

After saving the changes, apply the new network configuration using the following command:

```
sudo netplan apply
```

Finally, verify that the machines can reach each other by pinging each RAN machine from the core machine:

```
ping 10.0.0.20
ping 10.0.0.40
```

The output should show successful ping responses, as shown in [Figure A.2](#).

```

core@core:~$ ping 10.0.0.20
PING 10.0.0.20 (10.0.0.20) 56(84) bytes of data.
64 bytes from 10.0.0.20: icmp_seq=1 ttl=64 time=0.625 ms
64 bytes from 10.0.0.20: icmp_seq=2 ttl=64 time=0.716 ms
64 bytes from 10.0.0.20: icmp_seq=3 ttl=64 time=0.673 ms
64 bytes from 10.0.0.20: icmp_seq=4 ttl=64 time=0.712 ms
64 bytes from 10.0.0.20: icmp_seq=5 ttl=64 time=0.914 ms
64 bytes from 10.0.0.20: icmp_seq=6 ttl=64 time=0.677 ms
64 bytes from 10.0.0.20: icmp_seq=7 ttl=64 time=0.925 ms
64 bytes from 10.0.0.20: icmp_seq=8 ttl=64 time=1.12 ms
64 bytes from 10.0.0.20: icmp_seq=9 ttl=64 time=1.10 ms
64 bytes from 10.0.0.20: icmp_seq=10 ttl=64 time=1.18 ms
^C
--- 10.0.0.20 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9185ms
rtt min/avg/max/mdev = 0.625/0.863/1.179/0.199 ms
core@core:~$ ping 10.0.0.40
PING 10.0.0.40 (10.0.0.40) 56(84) bytes of data.
64 bytes from 10.0.0.40: icmp_seq=1 ttl=64 time=1.75 ms
64 bytes from 10.0.0.40: icmp_seq=2 ttl=64 time=0.632 ms
64 bytes from 10.0.0.40: icmp_seq=3 ttl=64 time=1.08 ms
64 bytes from 10.0.0.40: icmp_seq=4 ttl=64 time=1.09 ms
64 bytes from 10.0.0.40: icmp_seq=5 ttl=64 time=0.674 ms
64 bytes from 10.0.0.40: icmp_seq=6 ttl=64 time=1.08 ms
64 bytes from 10.0.0.40: icmp_seq=7 ttl=64 time=1.10 ms
64 bytes from 10.0.0.40: icmp_seq=8 ttl=64 time=1.07 ms
64 bytes from 10.0.0.40: icmp_seq=9 ttl=64 time=1.07 ms
64 bytes from 10.0.0.40: icmp_seq=10 ttl=64 time=0.682 ms
^C
--- 10.0.0.40 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9075ms
rtt min/avg/max/mdev = 0.632/1.023/1.748/0.306 ms
core@core:~$

```

Figure A.2: Pings from the core machine to the 5G RAN machine (10.0.0.20) and the 4G RAN machine (10.0.0.40), with no packet loss and round-trip times of around 1 ms.

A.2 Core Machine (Machine 1)

This section installs and configures Open5GS, which serves as the core for both generations.

The installation process consists of the following steps:

1. Installing MongoDB.
2. Installing Open5GS.
3. Installing the WebUI for Open5GS.
4. Configuring the core network functions for the 4G and 5G networks.
5. Routing the UE traffic.
6. Adding the subscribers.

More information about the project and the installation steps can be found in the official Open5GS documentation [26].

A.2.1 Installing MongoDB

Open5GS depends on MongoDB as its database backend, where it stores subscriber information, network configurations, and operational data. Therefore, the first step in setting up Open5GS is to install MongoDB:

```
sudo apt update
sudo apt install gnupg curl -y
curl -fsSL https://pgp.mongodb.com/server-8.0.asc | sudo gpg -o
    /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

echo "deb [ arch=amd64,arm64
    signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]
    https://repo.mongodb.com/apt/ubuntu jammy/mongodb-org/8.0 multiverse" |
    sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list

sudo apt update
sudo apt install -y mongodb-org
```

After executing the above commands, MongoDB will be installed on the machine. We can then start and verify the installation of the MongoDB daemon:

```
sudo systemctl start mongod
sudo systemctl status mongod
```

The output, shown in Figure A.3, should indicate that the MongoDB service is active and running:

To ensure MongoDB starts automatically on system boot, we can enable the service with the following command:

```
core@core:~$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-04-21 17:41:12 WEST; 1 week 3 days ago
     Docs: https://docs.mongodb.org/manual
```

Figure A.3: Output of the `systemctl status mongod` command, showing that the service is active and running.

```
sudo systemctl enable mongod
```

For more detailed instructions and troubleshooting guidance, refer to the official MongoDB installation documentation [24].

A.2.2 Installing Open5GS

After installing MongoDB, the next step is to add the Open5GS repository to the system and install Open5GS. This can be done by executing the following commands:

```
sudo add-apt-repository ppa:open5gs/latest
sudo apt update
sudo apt install open5gs -y
```

This installs the latest Open5GS release available at the time; the testbed described in this dissertation ran version 2.8.0. After the installation is complete, we can verify that the Open5GS services are running correctly by checking their status:

```
sudo systemctl status open5gs-*
```

A.2.3 Installing the WebUI for Open5GS

Open5GS provides a WebUI for easier management of subscriber information, network configurations, and monitoring. The interface depends on Node.js, which needs to be installed first (on most Ubuntu builds, Node.js comes pre-installed). To install Node.js, we can use the following commands:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key |
sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg

NODE_MAJOR=20
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
      https://deb.nodesource.com/node_$NODE_MAJOR.x nodistro main" |
sudo tee /etc/apt/sources.list.d/nodesource.list

sudo apt update
sudo apt install nodejs -y
```

After installing Node.js, we can proceed to install the Open5GS WebUI by running the following command:

```
curl -fsSL https://open5gs.org/open5gs/assets/webui/install |
sudo -E bash -
```

The WebUI can be accessed by navigating to `http://localhost:9999` in a web browser. The default login credentials are:

- Username: admin
- Password: 1423

A.2.4 Configuring the Core Network Functions

In Open5GS, the configuration files for the network functions are located in the `/etc/open5gs/` directory, with each network function having its own file. The configuration files used in this dissertation are kept in the project's GitHub repository [31]. The repository also holds a script, `deploy_open5gs_configs.sh`, which copies the files into that directory, keeping a backup of the existing ones, and restarts the Open5GS services:

```
sudo ./deploy_open5gs_configs.sh
```

The following subsections describe the changes these files make to the Open5GS defaults, for each generation. Unless issued a PLMN ID by the national regulator, use the international test PLMN ID (001/01), which is the one used in this configuration.

A.2.4.1 4G: MME and SGW-U

Modify the MME configuration file, on `/etc/open5gs/mme.yaml`, to set the PLMN ID, TAC and the IP address the MME listens on for the eNodeB.

```
--- a/configs/open5gs/mme.yaml.in
+++ b/configs/open5gs/mme.yaml.in
@@ -10,7 +10,7 @@ mme:
    freeDiameter: @sysconfdir@/freeDiameter/mme.conf
    slap:
      server:
-       - address: 127.0.0.2
+       - address: 10.0.0.10
    gtpc:
      server:
-       - address: 127.0.0.2
@@ -25,14 +25,14 @@ mme:
    port: 9090
    gummei:
      plmn_id:
-       mcc: 999
-       mnc: 70
+       mcc: 001
+       mnc: 01
    mme_gid: 2
    mme_code: 1
```

```
    tai:
      plmn_id:
-       mcc: 999
-       mnc: 70
+       mcc: 001
+       mnc: 01
-       tac: 1
+       tac: 7
    security:
      integrity_order : [ EIA2, EIA1, EIA0 ]
```

Modify the SGW-U configuration file, on `/etc/open5gs/sgwu.yaml`, to set the GTP-U IP address.

```
--- a/configs/open5gs/sgwu.yaml.in
+++ b/configs/open5gs/sgwu.yaml.in
@@ -15,7 +15,7 @@ sgwu:
#       - address: 127.0.0.3
    gtpu:
      server:
-       - address: 127.0.0.6
+       - address: 10.0.0.10
```

After modifying the configuration files, restart the Open5GS daemons to apply all changes:

```
sudo systemctl restart open5gs-mmed
sudo systemctl restart open5gs-sgwud
```

A.2.4.2 5G: AMF, SMF, UPF and NRF

Four 5G network functions need changes. The other network functions of the 5G core (AUSF, UDM, UDR, PCF, NSSF, BSF and SCP) keep their default configuration, since they only talk to each other over the loopback interface of the core machine.

Modify the AMF configuration file, on `/etc/open5gs/amf.yaml`, to set the IP address the AMF listens on for the gNodeB, and the PLMN ID and TAC, which must match the ones configured in the gNodeB:

```
--- a/configs/open5gs/amf.yaml.in
+++ b/configs/open5gs/amf.yaml.in
@@ amf: @@
    ngap:
      server:
-        - address: 127.0.0.5
+        - address: 10.0.0.10
@@ amf: @@
    guami:
      - plmn_id:
-        mcc: 999
-        mnc: 70
+        mcc: 001
+        mnc: 01
      amf_id:
        region: 2
        set: 1
    tai:
      - plmn_id:
-        mcc: 999
-        mnc: 70
-        tac: 1
+        mcc: 001
+        mnc: 01
+        tac: 7
    plmn_support:
      - plmn_id:
-        mcc: 999
```

```
-         mnc: 70
+         mcc: 001
+         mnc: 01
s_nssai:
-   sst: 1
```

Modify the UPF configuration file, on `/etc/open5gs/upf.yaml`, to set the GTP-U address the gNodeB sends user plane traffic to (the N3 interface):

```
--- a/configs/open5gs/upf.yaml.in
+++ b/configs/open5gs/upf.yaml.in
@@ upf: @@
    gtpu:
      server:
-         - address: 127.0.0.7
+         - address: 10.0.0.10
```

Modify the SMF configuration file, on `/etc/open5gs/smf.yaml`, to name the data network of the UE address pool and to set the DNS servers handed to the UEs. Because the SMF and UPF also act as the PGW-C and PGW-U of the 4G network, these settings apply to both generations:

```
--- a/configs/open5gs/smf.yaml.in
+++ b/configs/open5gs/smf.yaml.in
@@ smf: @@
    session:
-         - subnet: 10.45.0.0/16
+         - dnn: internet
+         subnet: 10.45.0.0/16
      gateway: 10.45.0.1
-         subnet: 2001:db8:cafe::/48
      gateway: 2001:db8:cafe::1
    dns:
-         - 8.8.8.8
```

```

-      - 8.8.4.4
+      - 1.1.1.1
+      - 1.0.0.1
      - 2001:4860:4860::8888
      - 2001:4860:4860::8844

```

Finally, modify the NRF configuration file, on `/etc/open5gs/nrf.yaml`, to set the PLMN it serves:

```

--- a/configs/open5gs/nrf.yaml.in
+++ b/configs/open5gs/nrf.yaml.in
@@ nrf: @@
    serving:
      - plmn_id:
-         mcc: 999
-         mnc: 70
+         mcc: 001
+         mnc: 01

```

After modifying the configuration files, restart the Open5GS daemons to apply all changes:

```
sudo systemctl restart open5gs-*
```

A.2.5 Routing the UE Traffic

For the UEs to reach networks beyond the core machine, the core must forward their traffic and translate their addresses. Both generations assign UE addresses from the same pool, so this is done once:

```
### Enable IPv4/IPv6 Forwarding
$ sudo sysctl -w net.ipv4.ip_forward=1
$ sudo sysctl -w net.ipv6.conf.all.forwarding=1

### Add NAT Rule
$ sudo iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j
    MASQUERADE
$ sudo ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun -j
    MASQUERADE
```

A.2.6 Adding the Subscribers

Before a UE can attach, its SIM must be registered in the core as a subscriber. Subscribers are added through the WebUI installed in Section A.2.3: open `http://localhost:9999` on the core machine, log in, and select Subscriber, which lists the subscribers already registered, as shown in Figure A.4.

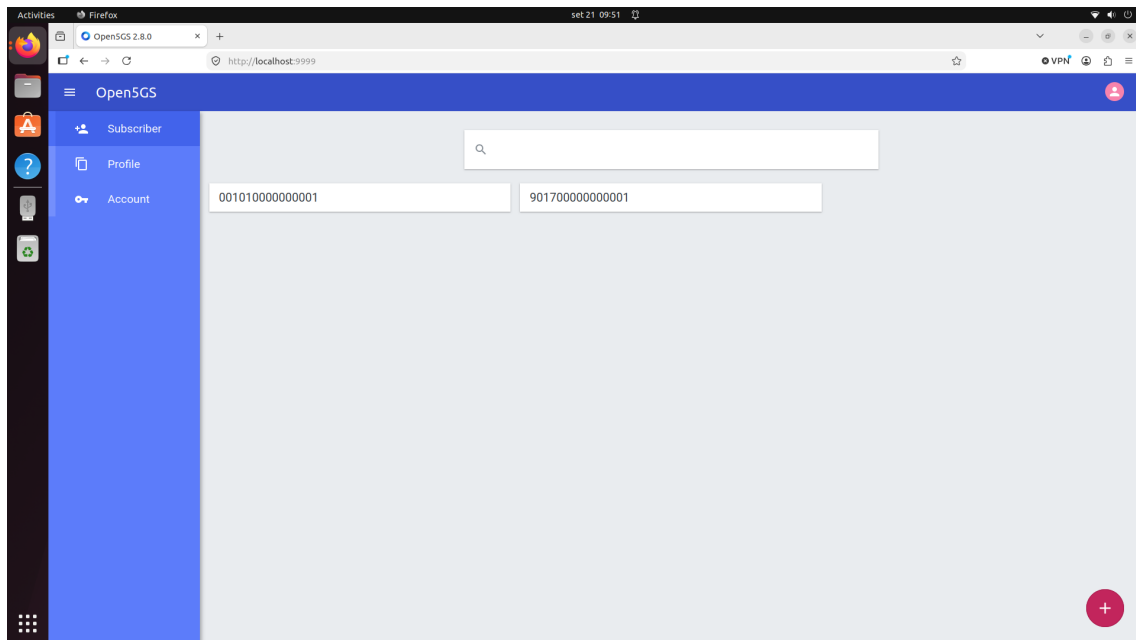


Figure A.4: The Open5GS WebUI's subscriber list, each subscriber identified by its IMSI.

The plus button at the bottom right opens the subscriber form, shown in Figure A.5.

The fields must match the values programmed into the UE's SIM, which for the eSIM used in this project are given in Appendix B:

- **IMSI:** the subscriber's identity, whose first five digits (00101) are the PLMN of the testbed.
- **Subscriber Key (K)** and **Operator Key (OPc):** the secret keys used to authenticate the SIM, with the USIM type set to OPc.
- **Authentication Management Field (AMF):** left at its default, 8000.

The remaining fields, which set the slice and data network of the subscriber, are left at the WebUI's defaults (SST 1 and internet), which match the core configuration of Table 4.3. A single subscriber entry serves both generations: the same SIM authenticates against the 4G EPC and the 5G core, since both read their subscriber data from the same database.

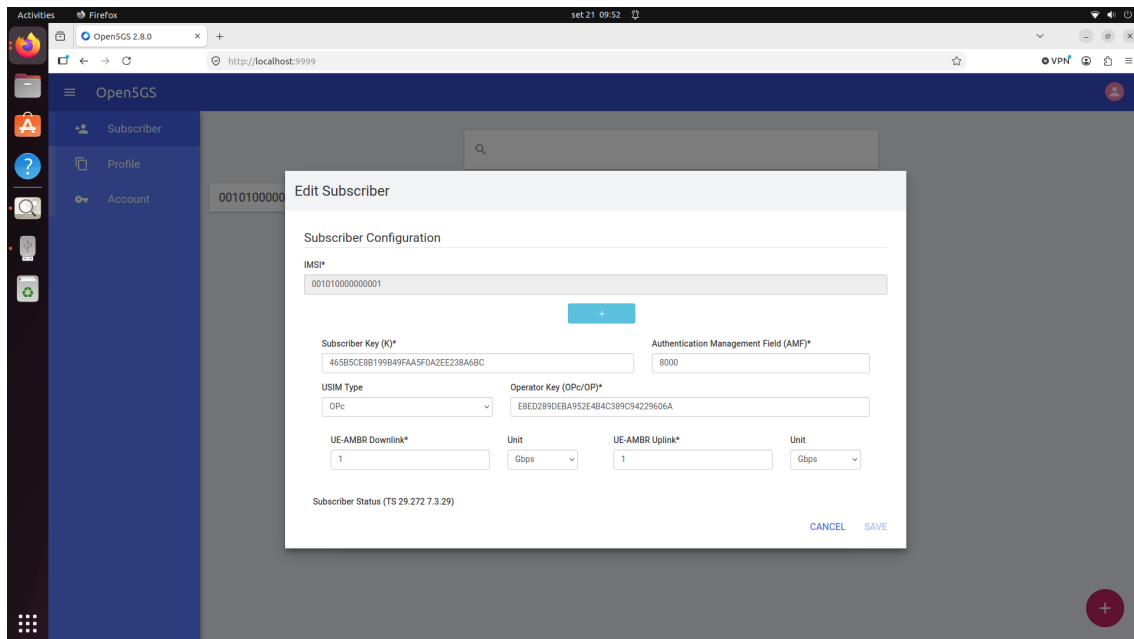


Figure A.5: The Open5GS WebUI's subscriber form, filled in for the testbed's UE.

A.3 4G RAN Machine (Machine 4)

This section sets up the 4G RAN: the drivers for the Nuand bladeRF xA4 and srsRAN 4G. The hardware itself, including the BT-200 and BT-100 amplifiers on the board's RX and TX ports, is described in [Section 4.2](#).

A.3.1 Setting up the Nuand drivers and firmware for the bladeRF xA4

Nuand provides official Ubuntu package repositories (PPAs) that simplify the installation of the bladeRF software on Ubuntu systems. Through these repositories, users can quickly install both the required host tools and libraries and the necessary firmware for the FPGA using apt, without needing to build everything manually from source. To enable the PPA:

```
sudo add-apt-repository ppa:nuandllc/bladerf
sudo apt update
```

After enabling the PPA, we can install the bladeRF software and firmware using the following commands:

```
sudo apt install bladerf
sudo apt install bladerf-fpga-hostedxa4
```

To bridge the connection between the bladeRF xA4 and the RAN, SoapySDR will be used, which will be covered in the next section ([A.3.2](#)). For this to work, the header files for the bladeRF drivers need to be installed:

```
sudo apt install libbladerf-dev
```

With the bladeRF drivers installed, we can proceed to the installation of SoapySDR, which will allow us to use the bladeRF xA4 as the radio interface for the RAN.

A.3.2 Installing SoapySDR

The SoapySDR component comprises two parts: the SoapySDR library, which is the main interface for all types of SDR devices, and the SoapyBladeRF plug-in, which provides specific support for the bladeRF xA4.

First, we need to install the necessary dependencies:

```
sudo apt install -y cmake g++ libpython3-dev python3-numpy swig
```

Afterward, we can clone the SoapySDR repository and build the library:

```
git clone https://github.com/pothosware/SoapySDR.git
cd SoapySDR

mkdir build
cd build
cmake ..
make -j`nproc`
sudo make install -j`nproc`
sudo ldconfig
```

To verify the installation of the SoapySDR library, we can run the following command:

```
SoapySDRUtil --info
```

After installing SoapySDR, we can proceed to install the SoapyBladeRF plug-in through their repository:

```
git clone https://github.com/pothosware/SoapyBladeRF.git
cd SoapyBladeRF

mkdir build
cd build
cmake ..
make
sudo make install
```

With both the SoapySDR library and the SoapyBladeRF plug-in installed, the bladeRF xA4 should now be recognized as an available SDR device when running the following command:

```
SoapySDRUtil --probe="driver=bladerf"
```

This command should output the details of the board, thus confirming that the bladeRF xA4 is properly configured and ready to be used as the radio interface for the 4G RAN. With the SDR configuration complete, we can now proceed to install and configure srsRAN 4G.

A.3.3 srsRAN 4G

First, install the dependencies for srsRAN 4G:

```
sudo apt-get install build-essential cmake libfftw3-dev libmbedtls-dev
libboost-program-options-dev libconfig++-dev libsctp-dev
```

Then, clone the srsRAN repository and build the software:

```
git clone https://github.com/srsRAN/srsRAN_4G.git
cd srsRAN_4G
mkdir build
cd build
cmake ../
make
make test
```

Proceed to install srsRAN 4G:

```
sudo make install
srsran_install_configs.sh user
```

The second command installs the default configuration files in `~/.config/srsran/`. Only the eNodeB files are used, since the core is provided by Open5GS rather than by srsRAN's own EPC. The configuration used in this dissertation is kept in the project's repository [31] (`enb.conf`, `rr.conf`, `sib.conf` and `rb.conf`). The lines that differ from the default files point the eNodeB to the core, set its own address, and set the radio parameters:

```
# enb.conf

[enb]

enb_id = 0x19B
mcc = 001
mnc = 01
mme_addr = 10.0.0.10
gtp_bind_addr = 10.0.0.40
s1c_bind_addr = 10.0.0.40
n_prb = 50

[rf]

tx_gain = 66
```

```
rx_gain = 60
device_name = bladeRF

# rr.conf (cell list)
cell_id = 0x01;
tac = 0x0007;
pci = 1;
dl_earfcn = 3350;
```

This completes the one-time setup of the 4G RAN machine. The next steps to run the network are given in [Section A.5](#).

A.4 5G RAN Machine (Machine 2)

This section sets up the 5G RAN: the drivers for the NI B2901, the GPSDO, and the srsRAN Project gNodeB. The hardware itself is described in [Section 4.2](#).

A.4.1 Installing UHD

The B2901 is driven through UHD, the USRP Hardware Driver. The testbed ran UHD version 4.10.0.0, built from source at that release rather than installed from the Ubuntu repositories, which may provide a different version. The OCUDU installation guide names UHD 4.6.0.0 as its requirement; the later 4.10.0.0 release worked with the gNodeB used in this dissertation. First, install the build dependencies:

```
sudo apt-get install autoconf automake build-essential ccache cmake
cpufrequtils doxygen ethtool g++ git inetutils-tools libboost-all-dev
libncurses-dev libusb-1.0-0 libusb-1.0-0-dev libusb-dev python3-dev
python3-mako python3-numpy python3-requests python3-scipy
python3-setuptools python3-ruamel.yaml libgrpc++-dev libprotobuf-dev
protobuf-compiler-grpc
```

Then clone the UHD repository.

```
git clone https://github.com/EttusResearch/uhd.git
cd uhd
```

The UHD host software is built from the host directory of the repository:

```
cd host
mkdir build
cd build
cmake ../
make
make test
sudo make install
sudo ldconfig
```

The B2901 loads its FPGA image from the host each time it is powered on, so the images matching the installed UHD version must be downloaded:

```
sudo uhd_images_downloader
```

Finally, with the B2901 connected over USB 3.0, check that UHD detects it and that it reports the board's details without errors:

```
uhd_find_devices
uhd_usrp_probe
```

The second command loads the FPGA image onto the board, so it takes a few seconds to complete the first time it is run after the board is powered on.

A.4.2 Setting up the GPSDO

The B2901 synchronizes its frequency and time references to the Leo Bodnar GPSDO, which in turn disciplines its oscillator to the GPS satellites. The GPSDO provides two signals,

each connected to the matching input on the rear of the B2901, as shown in Figure 4.4:

- a 1PPS (one pulse per second) signal, on Output 1, connected to the PPS IN port, which aligns the board's time to the start of each second;
- a 10 MHz reference clock, on Output 2, connected to the REF IN port, from which the board derives all of its sampling and carrier frequencies.

The gNodeB YAML file is configured to use both through the `clock: external` and `sync: external` settings (Section A.4.4). The GPSDO is powered over USB from Machine 2, and the same USB connection is used to configure it and to check its lock.

A.4.2.1 Placing the GPS antenna

The GPSDO can only discipline its oscillator while it receives signal from the GPS satellites, so its antenna needs a clear view of the sky. In this testbed, the antenna was placed on the windowsill of the lab, as shown in Figure A.6. After power-on, the receiver locks onto the signal from the satellites after a few minutes.



Figure A.6: The GPSDO's GPS antenna, placed on the windowsill outside the lab window for a clear view of the sky.

A.4.2.2 Installing lbgpsdo

The GPSDO is configured and monitored with `lbgpsdo` [17], an open-source command-line utility for Leo Bodnar devices. It is written in Python and depends on the `hid` package, so it is installed in its own virtual environment:

```
git clone https://github.com/hamarituc/lbgpsdo.git
cd lbgpsdo
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

The utility provides two scripts: `lbe142x.py`, for the LBE-1421 used in this testbed, and `lbgpsdo.py`, for the older Leo Bodnar GPSDO models. With the GPSDO connected, confirm that it is detected:

```
./lbe142x.py list
```

A.4.2.3 Configuring the outputs

Set Output 1 to the 1PPS signal and Output 2 to 10 MHz, and save the settings to the device, so that they persist when it is powered off:

```
./lbe142x.py modify --pps-enable --f2 10000000 --save
```

A.4.2.4 Checking the lock

Before starting the gNodeB, check that the GPSDO is locked:

```
sudo ./lbe142x.py detail
```

The output shows the device's status and the settings of each output, as shown in Figure A.7. The GPSDO is ready once it reports both satellite lock (`SAT lock`), meaning

it is receiving enough satellites, and PLL lock, meaning its output is disciplined to them. The output settings confirm the configuration from the previous step: 1PPS on Output 1 and 10 MHz on Output 2.

```
ran@ran:~/lbgpsdo$ sudo ./lbe142x.py detail
Device information
-----
VID, PID:      0x1dd2:0x2444
Device:        /dev/hidraw3
Product:       LBE-1421 GPSDO Locked Clock Source
Manufacturer: Leo Bodnar Electronics
S/N:           0C7BB80F412A
Firmware:      1.9

Device status
-----
SAT lock:      LOCKED
PLL lock:      LOCKED
Antenna:       OK
Mode:          PLL

Output settings
-----
Output 1:      1 PPS level: NORMAL
Output 2:      10000000 Hz level: NORMAL
ran@ran:~/lbgpsdo$
```

Figure A.7: Output of `lbe142x.py detail` for the LBE-1421, reporting both satellite and PLL lock, and the two outputs set to 1PPS and 10 MHz.

A.4.3 srsRAN Project (OCUDU)

The srsRAN Project gNodeB is now developed under the name OCUDU [25]. UHD must be installed, as described in Section A.4.1, before the gNodeB is built. The build checks for UHD only when it is configured, and includes support for the B2901 only if UHD is found at that point.

A gNodeB built without UHD pre-installed compiles and installs without errors, but will not detect the radio when started, and installing UHD afterward does not fix it. The gNodeB has to be rebuilt from the `cmake` step onwards. With UHD in place, install the build dependencies:

```
sudo apt-get install cmake make gcc g++ pkg-config libmbdtk-dev
libscrt-dev libyaml-cpp-dev libgtest-dev
```

Then clone the repository:

```
git clone https://gitlab.com/ocudu/ocudu.git
cd ocudu
```

Build the software, run its tests, and install it:

```
mkdir build
cd build
cmake ../
make -j $(nproc)
make test -j $(nproc)
sudo make install
```

The build takes several minutes. Once installed, the gNodeB is available system-wide as the `gnb` command, which is used to start the 5G RAN in Section [A.5.3](#).

A.4.4 Configuring the gNB

Unlike srsRAN 4G, the srsRAN Project gNodeB reads a single YAML configuration file, passed to it through the startup command. The file used in this dissertation is kept in the project's repository [\[31\]](#) as `gnb.yaml`, and is shown below, as used in Round 3. It has five parts:

- `cu_cp`: the connection to the AMF on the core machine, and the tracking area, PLMN and slice the gNodeB supports, which must match the core configuration.
- `ru_sdr`: the radio. The B2901 is driven through UHD, takes both its clock and its time reference from the GPSDO (external), and runs at the transmit and receive gains discussed in Chapter [5](#).
- `cell_cfg`: the cell itself, in band n78 with a 20 MHz channel and 30 kHz subcarrier spacing, using the 2:2 TDD pattern.

- log and metrics: logging of all layers at info level, with the per-second metrics written into the log, as described in Section 4.6.
- pcap: packet captures of the MAC layer and of the NGAP signaling with the core.

```
cu_cp:
  inactivity_timer: 7200
  amf:
    addr: 10.0.0.10
    port: 38412
    bind_addr: 10.0.0.20
    supported_tracking_areas:
      - tac: 7
      plmn_list:
        - plmn: "00101"
      tai_slice_support_list:
        - sst: 1

ru_sdr:
  device_driver: uhd
  device_args: type=b200
  srate: 23.04
  clock: external
  sync: external
  otw_format: sc12
  tx_gain: 89.75
  rx_gain: 60

cell_cfg:
  dl_arfcn: 627340
  band: 78
  channel_bandwidth_MHz: 20
```

```
common_scs: 30
plmn: "00101"
tac: 7
pci: 1
tdd_ul_dl_cfg:
  dl_ul_tx_period: 5
  nof_dl_slots: 2
  nof_dl_symbols: 5
  nof_ul_slots: 2
  nof_ul_symbols: 5
q_rx_lev_min: -70

log:
  filename: /tmp/gnb.log
  all_level: info
  hex_max_size: 0

metrics:
  autostart_stdout_metrics: true
  enable_log: true
  enable_verbose: true

pcap:
  mac_enable: true
  mac_filename: /tmp/gnb_mac.pcap
  ngap_enable: true
  ngap_filename: /tmp/gnb_ngap.pcap
```

Note that the log and packet captures are written to /tmp, which is cleared when the machine restarts, so they must be copied elsewhere after each test session.

This completes the one-time setup of the 5G RAN machine. The steps repeated each

time the network is started are given in Section [A.5](#).

A.5 Running the Network

The previous sections cover setup, which is done once. This section covers the steps repeated every time the network is started. They assume the core has been configured and the subscriber added (Section [A.2](#)); since the Open5GS network functions run as system services, the core is already running whenever the core machine is on.

A.5.1 Preparing the RAN Machines

To ensure the best performance on the RAN side, the srsRAN team provides a performance script to optimize the system for real-time processing. This script should be executed on each RAN machine before starting its RAN:

```
# Download and execute the srsRAN performance tuning script
BASE=https://raw.githubusercontent.com/srsran/srsRAN_Project/main/scripts
curl -fsSL $BASE/srsran_performance | bash
```

A.5.2 Starting the 4G Network

To start the 4G network, start the eNodeB on Machine 4:

```
sudo srsenb
```

The eNodeB connects to the MME at 10.0.0.10. Once it reports the S1 connection as established, the 4G RAN is up and running, and the UE can connect to the network.

A.5.3 Starting the 5G Network

Before starting the 5G network, confirm that the GPSDO has GPS lock (Section [A.4.2](#)), as the B2901 takes both its clock and its time reference from it, and the gNodeB will not transmit correctly without them. Then start the gNodeB on Machine 2, passing it the configuration file from Section [A.4.4](#):

```
sudo gnb -c gnb.yaml
```

On start-up, the gNodeB opens an Stream Control Transmission Protocol (SCTP) connection to the AMF at 10.0.0.10 and exchanges an NG Setup Request and Response with it. The following lines in `/tmp/gnb.log` show a successful start:

```
[CU-CP] N2: Connection to AMF on 10.0.0.10:38412 was established
[NGAP ] Tx PDU: NGSetupRequest
[NGAP ] Rx PDU: NGSetupResponse
[CU-CP] Connected to AMF. Supported PLMNs: 00101
[GNB ] ==== gNB started ===
```

The same exchange can be confirmed on the core machine, where the AMF reports the gNodeB as connected in its log file, `/var/log/open5gs/amf.log`:

```
[amf] INFO: gNB-N2 accepted[10.0.0.20] in master_sm module
[amf] INFO: [Added] Number of gNBs is now 1
```

Once the gNodeB has started, the 5G RAN is up and running, and the UE can connect to the network. With the configuration used in this dissertation, the gNodeB also prints its per-second metrics to the console as soon as a UE is attached. To stop the gNodeB, press `Ctrl+C`.

The log and packet captures in `/tmp` should then be copied elsewhere, as noted in Section [A.4.4](#).

A.5.4 Verifying the Connection

With either RAN running, the UE should find the network under its Mobile Network (or Cellular) settings, when searching for mobile networks, listed under the testbed's PLMN (00101). Once the UE is connected, the registration can be seen in the Core's logs. For the 5G network, the AMF log, `/var/log/open5gs/amf.log`, records the registration of the

subscriber, and the SMF log, `/var/log/open5gs/smf.log`, the IP address assigned to it:

```
[amf] INFO: InitialUEMessage (../src/amf/ngap-handler.c:668)
[gmm] INFO: Registration request (../src/amf/gmm-sm.c:1678)
[gmm] INFO: [imsi-001010000000001] Registration complete
        (../src/amf/gmm-sm.c:3165)
[smf] INFO: UE SUPI[imsi-001010000000001] DNN[internet] IPv4[10.45.0.3]
        IPv6[] (../src/smf/npcf-handler.c:647)
```

For the 4G network, this information is found in the MME log, `/var/log/open5gs/mme.log`, and again in the SMF log:

```
[mme] INFO: InitialUEMessage (../src/mme/s1ap-handler.c:628)
[emm] INFO: [001010000000001] Attach request (../src/mme/emm-sm.c:488)
[smf] INFO: UE IMSI[001010000000001] APN[internet] IPv4[10.45.0.7] IPv6[]
        (../src/smf/s5c-handler.c:312)
[emm] INFO: [001010000000001] Attach complete (../src/mme/emm-sm.c:1603)
```

The address 10.45.0.3 assigned to the UE belongs to the 10.45.0.0/16 pool of the core, and it can be used to test the connection from the core machine, for example with ping 10.45.0.3. The throughput tests of this dissertation were run with `iperf3`, with the server on the core machine and the client on the UE.

On the core machine, start the server, writing its results to a JSON file, which records the round-trip time of each TCP stream in downlink tests. A separate file should be used for each test location, as `iperf3` appends to an existing file instead of replacing it:

```
iperf3 -s -J --logfile results_location_A.json
```

On the UE, the tests are run from the CellularLab app [6], pointing its `iperf3` client to the core machine's `ogstun` address, 10.45.0.1. The parameters used are a duration of 20 seconds and 8 parallel streams, with a target rate of 100 Mbps per stream in the case

of UDP tests. The app runs these as standard `iperf3` commands and records the exact command at the top of each result it exports. For a UDP downlink test, for example:

```
iperf3 -c 10.45.0.1 -p 5201 -t 20 -i 1 -P 8 -R -u -b 100M -d -V
```

Here, `-R` selects the reverse (downlink) direction, `-u -b 100M` specifies a UDP test at 100 Mbps per stream, and `-d -V` enables debug and verbose output. TCP tests omit `-u -b 100M`, and uplink tests omit `-R`. Figure A.8 shows the app itself, with the parameters of a TCP uplink test filled in.

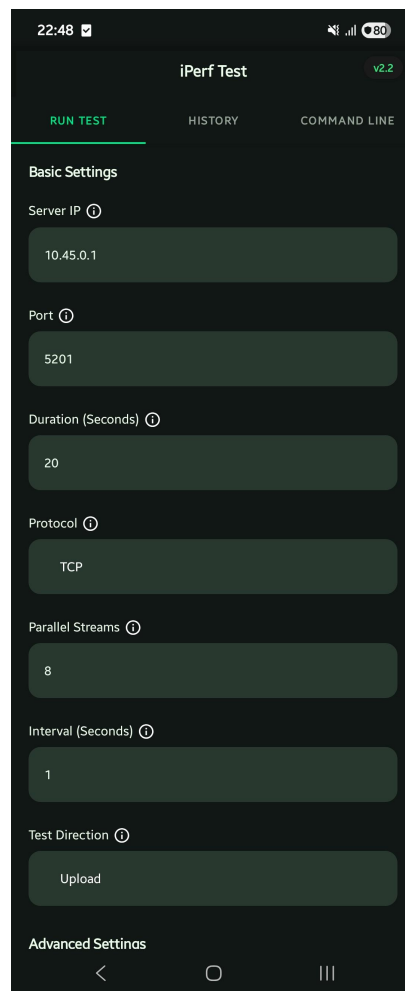


Figure A.8: The CellularLab app on the UE, configured for a TCP uplink test: the server address and port of the `iperf3` server on the core machine, a duration of 20 seconds, 8 parallel streams, and a reporting interval of one second.

A.6 Troubleshooting

This section briefly lists the problems encountered while building and testing the testbed, and how each one was identified and solved.

The gNodeB or eNodeB does not detect the radio.

If UHD, in the 5G case, or libbladeRF and SoapySDR, in the 4G case, are installed after the RAN software has been built, that software is compiled without support for the SDR and will not find it, even though `uhd_find_devices` or `SoapySDRUtil --probe="driver=bladerf"` lists it. Both select their RF drivers when they are built, so the RAN has to be rebuilt from the CMake step onwards: Section [A.4.3](#) for the gNodeB, Section [A.3.3](#) for the eNodeB.

The gNodeB starts, but the UE does not find the network.

Check if the UE supports the band and channel configured in the gNodeB's YAML file (Section [A.4.4](#)). Also check if the UE's operating system allows the use of the PLMN configured in the Core and RAN, as some phones restrict the use of non-standard PLMNs. If the UE is not compatible with the network, it will not find it.

The UE cannot stay connected to the network.

Check that the GPSDO reports both satellite and PLL lock (Section [A.4.2](#)), and that the gNodeB is properly configured to use it. Without the GPSDO, the radio's clock and timing reference are not disciplined, and the UE may fail to synchronize with the cell. Also check the gain settings on the gNodeB or eNodeB, as too low a gain may prevent the UE from maintaining a stable connection. If needed, perform a gain sweep to find the optimal values for the test location.

The RAN cannot connect to the core.

If the gNodeB or eNodeB reports that the connection to the AMF or MME was refused, check that the core machine is reachable at 10.0.0.10 (Section [A.1](#)), that the AMF and MME are configured to listen on that address, and that their services are running, with `systemctl status open5gs-amfd` or `open5gs-mmed`.

The UE is rejected when registering.

An authentication failure in the AMF or MME log means that the subscriber data in

the WebUI does not match the SIM. Check the IMSI, K, and OPc values against those of the eSIM ([Appendix B](#)).

The UE connects, but has no internet access.

The IP forwarding and NAT rules of [Section A.2.5](#) are not kept across a restart of the core machine and have to be applied again after every reboot, unless they are made persistent.

PROVISIONING eSIM

In this appendix, we will provision an eSIM profile for the testbed and connect to the network. We will create an eSIM profile through a provisioning server and follow the steps to activate it on the UE. Once the eSIM is activated, it can be used to connect to the network and transmit/receive data.

B.1 Creating an eSIM profile

To create an eSIM profile for the testbed, we will use Simlessly [34], specifically their Remote SIM Provisioning (RSP) tool, which allows custom eSIM profile creation. This platform requires an account to access the provisioning services. Once logged in, the user is greeted with a dashboard, shown in Figure B.1, where we can manage our eSIM profiles and view their status.

From the dashboard, we can click the “Generate AC” button to create a new eSIM profile. In the prompt (Figure B.2), we fill in the details for the eSIM profile, provided in Table B.1.

Key	Value
ICCID	89001010000000000001
IMSI	0010100000000001
KI	465B5CE8B199B49FAA5F0A2EE238A6BC
OPc	E8ED289DEBA952E4B4C389C94229606A

Table B.1: eSIM profile details.

Click on “Generate” to create the eSIM profile. Once generated, back on the dashboard, click on the QR code icon in the “AC QR code” column to display the Activation Code.

B.1. CREATING AN ESIM PROFILE

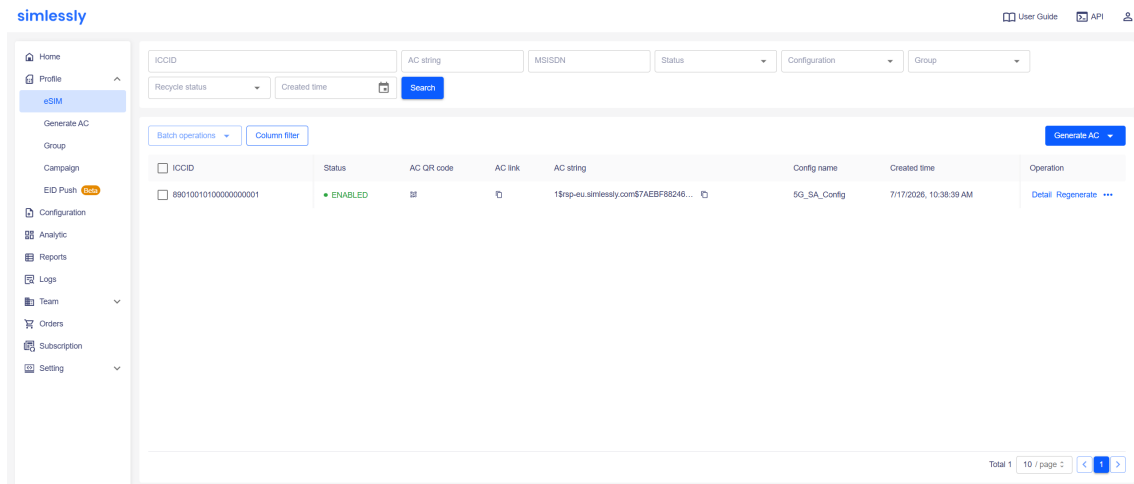


Figure B.1: Simlessly Dashboard.

Generate AC

Only for generating AC with a "Regular"/"Private LTE" configuration

* ICCID

89001010000000000001

* IMSI

0010100000000001

* KI

465B5CE8B199B49FAA5F0A2EE238A6BC

* OPC

E8ED289DEBA952E4B4C389C94229606A

* Configuration

5G_SA_Config

* AC format

☒ String ☒ Picture ☒ Link

Generate

Figure B.2: Generating an eSIM profile.



Figure B.3: eSIM Activation Code.

B.2 Installing the profile on the UE

With the Activation Code displayed, the profile is installed on the phone by scanning it. On the UE, we open the SIM manager, choose to add an eSIM, and scan the QR code shown in Figure B.3. The phone then asks for confirmation, downloads the profile from the provisioning server, and lists it in the SIM manager once it is installed. Figure B.4 shows the three steps as they appear on the UE.

Once the profile is enabled and selected for mobile data, the UE attaches to the testbed's network as described in Appendix A.

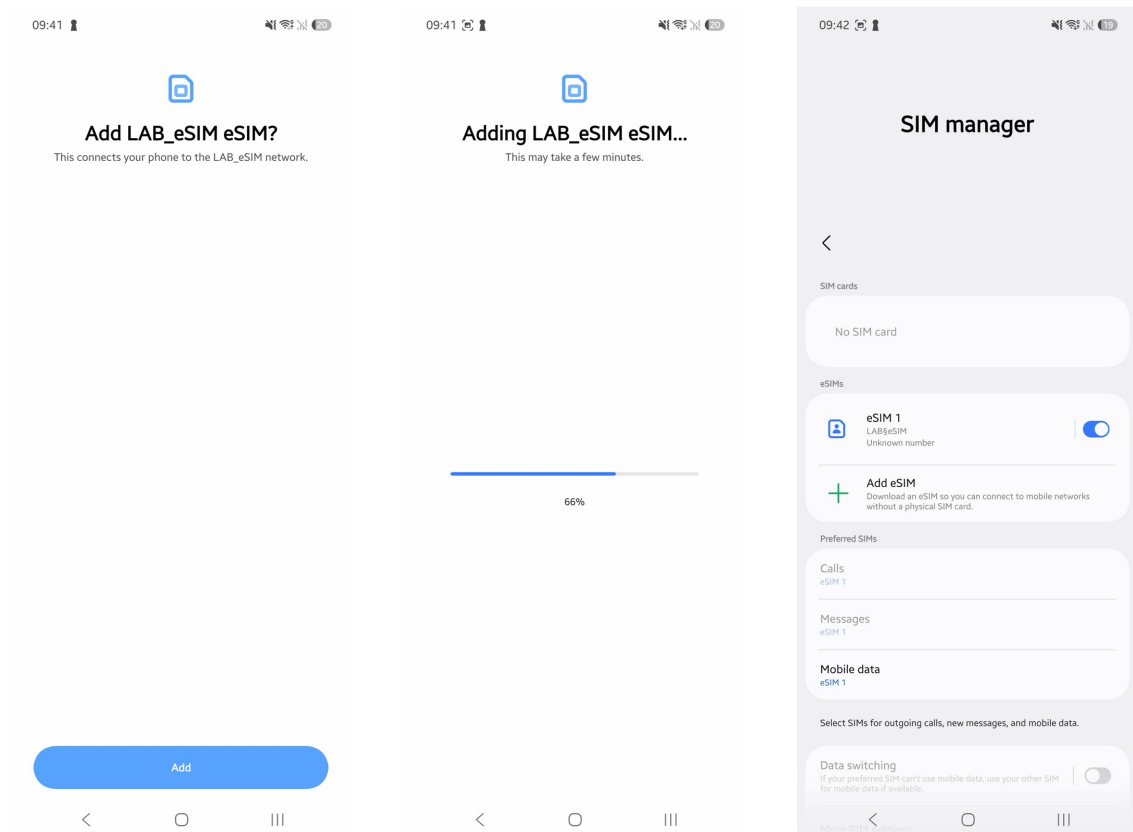


Figure B.4: Installing the eSIM profile on the UE. Left: the phone asks to confirm the LAB_eSIM profile. Center: the profile being downloaded from the provisioning server. Right: the SIM manager with the profile installed and enabled, and mobile data set to use it.

COLOPHON



This document was typeset with lua $\text{\LaTeX}$ and the **novathesis** template (v8.3.0), set in New PX (Palatino), with New PX Math for math.

J. M. Lourenço. *novathesis: A LaTeX Template for Academic Theses and Dissertations*. Version 8.3.0. NOVA University Lisbon. 2026. DOI: [10.5281/zenodo.21938603](https://doi.org/10.5281/zenodo.21938603) This is *not* a PDF/A archival copy.

Build 12cc9022, September 28, 2026.

